UNIVERSIDADE PAULISTA – UNIP
CAMPUS SÃO JOSÉ DO RIO PRETO
CURSO DE CIÊNCIA DA COMPUTAÇÃO

EDUARDO ZANELLA BENTO

DETECÇÃO AUTOMÁTICA DE VOZ SINTÉTICA UTILIZANDO TÉCNICAS DE *MACHINE LEARNING*

São José do Rio Preto
2026

EDUARDO ZANELLA BENTO

DETECÇÃO AUTOMÁTICA DE VOZ SINTÉTICA UTILIZANDO TÉCNICAS DE *MACHINE LEARNING*

Trabalho de Conclusão de Curso apresentado ao Curso de Ciência da Computação da Universidade Paulista – UNIP, Campus São José do Rio Preto, como requisito parcial para obtenção do título de Bacharel em Ciência da Computação.

Orientador: Prof. Anderson Fola

São José do Rio Preto

2026

FOLHA DE APROVAÇÃO

EDUARDO ZANELLA BENTO

DETECÇÃO AUTOMÁTICA DE VOZ SINTÉTICA UTILIZANDO TÉCNICAS DE *MACHINE LEARNING*

Trabalho de Conclusão de Curso apresentado ao Curso de Ciência da Computação da Universidade Paulista – UNIP, Campus São José do Rio Preto, como requisito parcial para obtenção do título de Bacharel em Ciência da Computação.

Aprovado em: ____/____/2026

BANCA EXAMINADORA

_____

Prof. Anderson Fola – Orientador

Universidade Paulista – UNIP

_____

Prof(a). _____

Universidade Paulista – UNIP

_____

Prof(a). _____

Universidade Paulista – UNIP

RESUMO

O avanço das técnicas de síntese de voz baseadas em aprendizado profundo, em especial a combinação entre modelos de *Text-to-Speech* (TTS) e de *Voice Conversion* (VC), tornou possível a geração de áudios sintéticos com alto grau de fidelidade às vozes humanas reais, ampliando riscos como falsificação de identidade, *vishing* e disseminação de desinformação. Diante desse cenário, este trabalho investigou se modelos clássicos baseados em características acústicas manuais mantêm desempenho preditivo competitivo e viabilidade computacional, frente a uma Rede Neural Convolucional (CNN) na detecção de voz sintética, dentro do mesmo ambiente experimental. Foram comparados dois classificadores clássicos, *Random Forest* (RF) e *Support Vector Machine* (SVM), alimentados por Coeficientes Cepstrais de Frequência Mel (MFCC), Taxa de Cruzamento por Zero (ZCR) e Centroide Espectral, e uma CNN aplicada a espectrogramas Log-Mel. O universo do experimento foi a partição de avaliação do ASVspoof 2021 (*Automatic Speaker Verification Spoofing and Countermeasures Challenge*), subconjunto *Logical Access* (LA), com 148.176 áudios, dividida internamente em treino, validação e teste. Os três modelos tiveram a mesma amostra final de treinamento no braço principal, uma subamostra estratificada de 30.000 áudios fixada *a priori* como orçamento experimental comum, depois de um pré-processamento comum que incluiu a remoção do silêncio. O limiar de decisão de cada modelo foi escolhido apenas na validação, o conjunto de teste foi avaliado uma única vez e as diferenças entre os modelos foram comparadas por *bootstrap* pareado. No teste, a CNN obteve F1-macro de 0,8883 e taxa de erro igual (EER) de 0,0737; o SVM, 0,7981 e 0,1411; e o RF, 0,7210 e 0,1946, com diferenças estatisticamente distinguíveis nas duas métricas. Quanto ao custo computacional, medido como tempo de inferência ponta a ponta de um áudio isolado (soma das medianas das etapas), o SVM levou 4,92 ms e o RF, 10,23 ms, contra 16,80 ms da CNN na unidade central de processamento (CPU) e 4,58 ms na unidade de processamento gráfico (GPU). Conclui-se que, neste protocolo, os modelos clássicos não mantiveram desempenho preditivo competitivo (F1-macro e EER) e que a vantagem deles em latência de inferência depende do *hardware*: sem GPU, eles apresentam menor latência ponta a ponta por áudio isolado, e, com GPU, a CNN e o SVM tiveram tempos da mesma ordem de grandeza, sem ordenação estável.

Palavras-chave: Detecção de voz sintética. *Deepfakes* de áudio. *Machine learning*. MFCC. Redes Neurais Convolucionais.

ABSTRACT

The advancement of deep learning-based speech synthesis techniques, particularly the combination of Text-to-Speech (TTS) and Voice Conversion (VC) models, has enabled the generation of high-fidelity synthetic audio closely resembling real human voices, expanding threats such as identity fraud, vishing, and disinformation. In this context, this work investigated whether classical models based on manually extracted acoustic features maintain competitive predictive performance and computational feasibility, when compared to a Convolutional Neural Network (CNN) for synthetic voice detection within the same experimental environment. Two classical classifiers, Random Forest (RF) and Support Vector Machine (SVM), fed by Mel-Frequency Cepstral Coefficients (MFCC), Zero Crossing Rate (ZCR) and Spectral Centroid, were compared with a CNN applied to Log-Mel spectrograms. The experimental universe was the evaluation partition of the ASVspoof 2021 (Automatic Speaker Verification Spoofing and Countermeasures Challenge) dataset, Logical Access (LA) subset, with 148,176 audio files, internally split into training, validation and test sets. The three models shared the same final training sample in the main arm, a stratified subsample of 30,000 audio files fixed *a priori* as a common experimental budget, after a common preprocessing stage that included silence removal. The decision threshold of each model was selected on the validation set only, the test set was evaluated a single time, and the differences between models were assessed by paired bootstrap. On the test set, the CNN achieved a macro F1-score of 0.8883 and an equal error rate (EER) of 0.0737; the SVM, 0.7981 and 0.1411; and the RF, 0.7210 and 0.1946, with statistically distinguishable differences in both metrics. Regarding computational cost, measured as end-to-end inference time for a single audio file (sum of the per-stage medians), the SVM took 4.92 ms and the RF 10.23 ms, against 16.80 ms for the CNN on a central processing unit (CPU) and 4.58 ms on a graphics processing unit (GPU). Under this protocol, the classical models did not maintain competitive predictive performance (macro F1-score and EER), and their advantage in inference latency depends on the hardware: without a GPU they have lower end-to-end latency per single audio file, while with a GPU the CNN and the SVM had inference times of the same order of magnitude, with no stable ordering.

Keywords: Synthetic voice detection. Audio deepfakes. Machine learning. MFCC. Convolutional Neural Networks.

LISTA DE FIGURAS

Figura 1 – Fluxograma do protocolo experimental 30
Figura 2 – Partição dos dados, braços do experimento e ponto em que cada modelo usa o treino 42
Figura 3 – Correlação absoluta de cada característica com a proporção de fala, antes e depois do mascaramento do *padding* (fase *eval*) 55
Figura 4 – Curva de aprendizado do RF ajustado na validação, com as características congeladas e o limiar selecionado em cada ponto 57
Figura 5 – Espectrogramas Log-Mel crus do piloto do gerador, com três exemplos *bonafide* (acima) e três *spoof* (abaixo), de banda larga e estreita; a linha tracejada marca o fim dos *frames* válidos 60
Figura 6 – F1-macro no conjunto interno de parada antecipada, por época, nas seis configurações da grade da CNN (a estrela marca a melhor época da configuração escolhida) 62
Figura 7 – Curvas de treino da configuração escolhida da CNN: perda ponderada no treino interno (27.000) e no conjunto interno (3.000), à esquerda, e F1-macro e EER no conjunto interno, à direita 62
Figura 8 – Curvas ROC dos três modelos do braço principal na validação (n = 22.226), com o EER marcado no cruzamento com a reta FPR = FNR 68
Figura 9 – Matrizes de confusão dos quatro modelos no teste (n = 22.227), com o limiar selecionado na validação: contagem e percentual da classe real em cada célula 70
Figura 10 – F1-macro e EER dos quatro modelos na validação e no teste, com o limiar selecionado na validação 71
Figura 11 – EER por sistema de ataque (A07 a A19) dos três modelos do braço principal na validação (n = 22.226) 74
Figura 12 – F1-macro e EER por *codec* dos três modelos do braço principal na validação (n = 22.226), com o limiar do protocolo 76
Figura 13 – Custo de inferência nos quatro arranjos: por áudio isolado (só a predição e *pipeline* completo) e em lote (só a predição), em escala logarítmica 79
Figura 14 – Importância das características no RF principal: por permutação na validação e por redução de impureza 80

LISTA DE QUADROS

Quadro 1 – Comparativo dos Trabalhos Relacionados 26
Quadro 2 – Características acústicas do ramo clássico e parâmetros de extração 35
Quadro 3 – Definição do espectrograma Log-Mel e do protocolo da CNN 37
Quadro 4 – Artefatos congelados do experimento e respectivos *hashes* MD5 51
Quadro 5 – As 12 asserções que validaram o gerador de espectrogramas no piloto 59
Quadro 6 – Problemas técnicos encontrados na execução: sintoma, detecção e controle permanente 64

LISTA DE TABELAS

Tabela 1 – Composição do universo do experimento (fase *eval*) por *codec* e por classe 32
Tabela 2 – Composição do universo do experimento (fase *eval*) por classe e por sistema de ataque 32
Tabela 3 – Arquitetura da CNN adotada, por etapa 40
Tabela 4 – Espaço de busca e valor escolhido, por modelo 46
Tabela 5 – Resultado das buscas aleatórias de hiperparâmetros no treino do braço principal 56
Tabela 6 – Dispersão do F1-macro na validação por fonte de variação, com os hiperparâmetros vencedores fixos 58
Tabela 7 – Grade de configurações da CNN, avaliada no conjunto interno de parada antecipada (3.000 exemplos) 61
Tabela 8 – Desempenho dos quatro modelos na validação (n = 22.226), com o limiar selecionado nesse conjunto 67
Tabela 9 – Desempenho dos quatro modelos no teste (n = 22.227), com o limiar selecionado na validação 69
Tabela 10 – Acurácia e métricas por classe dos quatro modelos no teste (n = 22.227) 69
Tabela 11 – Variação do F1-macro e do EER da validação para o teste, por modelo, com o ruído de amostragem heurístico da diferença 71
Tabela 12 – Diferenças pareadas entre os modelos do braço principal na validação e no teste: Δ observado e IC95 do *bootstrap* pareado 72
Tabela 13 – Custo da subamostragem para o RF: diferença pareada entre o RF de referência (103.723) e o RF principal (30.000), na validação e no teste 73
Tabela 14 – Resumo do EER por sistema de ataque na validação (n = 22.226), por modelo do braço principal 74
Tabela 15 – Distância agregada de EER entre os modelos do braço principal na validação e piso de ruído heurístico 75
Tabela 16 – F1-macro e EER por banda de *codec* na validação, por modelo do braço principal 76
Tabela 17 – Tempo de inferência ponta a ponta de um áudio isolado (lote unitário), por etapa, nos quatro arranjos, em milissegundos (mediana) 77
Tabela 18 – Vazão em lote da predição nos quatro arranjos, a partir da representação já extraída (mediana) 78

Tabela 19 – As 10 características de maior importância por permutação no RF principal, com a importância e a posição por redução de impureza 81

LISTA DE ABREVIATURAS E SIGLAS

ASVspoof	*Automatic Speaker Verification Spoofing and Countermeasures Challenge*
AUC	*Area Under the Curve* (área sob a curva)
CEO	*Chief Executive Officer* (diretor executivo)
CNN	*Convolutional Neural Network* (Rede Neural Convolucional)
CPU	*Central Processing Unit* (unidade central de processamento)
CQCC	*Constant-Q Cepstral Coefficients* (Coeficientes Cepstrais de Q Constante)
CQT	*Constant-Q Transform*
CRLF	*Carriage Return Line Feed* (quebra de linha do Windows)
cuBLAS	*CUDA Basic Linear Algebra Subroutines* (biblioteca de álgebra linear da NVIDIA para GPU)
CUDA	*Compute Unified Device Architecture*
cuDNN	*CUDA Deep Neural Network library* (biblioteca da NVIDIA para redes neurais profundas em GPU)
DCT	*Discrete Cosine Transform* (Transformada Discreta do Cosseno)
DF	*DeepFake* (tarefa de detecção de *deepfakes* do ASVspoof 2021)
DFT	*Discrete Fourier Transform* (Transformada Discreta de Fourier)
DL	*Deep Learning* (aprendizado profundo)
EER	*Equal Error Rate* (Taxa de Erro Igual)
FFT	*Fast Fourier Transform* (Transformada Rápida de Fourier)
FinCEN	*Financial Crimes Enforcement Network*
FLAC	*Free Lossless Audio Codec*
FNR	*False Negative Rate* (Taxa de Falsos Negativos)
FPR	*False Positive Rate* (Taxa de Falsos Positivos)
GB	*Gradient Boosting*
GPU	*Graphics Processing Unit* (unidade de processamento gráfico)
HTK	*Hidden Markov Model Toolkit*
IC95	Intervalo de confiança de 95%
k-NN	*k-Nearest Neighbour* (k Vizinhos Mais Próximos)
LA	*Logical Access* (acesso lógico)
LCNN	*Light Convolutional Neural Network* (Rede Neural Convolucional Leve)
LFCC	*Linear Frequency Cepstral Coefficients* (Coeficientes Cepstrais de Frequência Linear)

LLM	*Large Language Model* (modelo de linguagem de grande escala)
MD5	*Message-Digest Algorithm 5*
MFCC	*Mel-Frequency Cepstral Coefficients* (Coeficientes Cepstrais de Frequência Mel)
MLP	*Multi-Layer Perceptron* (Perceptron Multicamadas)
NaN	*Not a Number* (valor não numérico)
PA	*Physical Access* (acesso físico)
RBF	*Radial Basis Function* (função de base radial)
ReLU	*Rectified Linear Unit* (unidade linear retificada)
RF	*Random Forest*
RMS	*Root Mean Square* (valor quadrático médio)
ROC	*Receiver Operating Characteristic*
STFT	*Short-Time Fourier Transform* (Transformada de Fourier de Tempo Curto)
SVM	*Support Vector Machine* (Máquina de Vetores de Suporte)
TF32	*TensorFloat-32*
TTS	*Text-to-Speech*
VAD	*Voice Activity Detection* (detecção de atividade de voz)
VC	*Voice Conversion* (conversão de voz)
ZCR	*Zero Crossing Rate* (Taxa de Cruzamento por Zero)

SUMÁRIO

1 INTRODUÇÃO 13
1.1 Objetivos 14
1.1.1 Objetivo Geral 14
1.1.2 Objetivos Específicos 14
1.2 Justificativa 14
1.3 Metodologia 15
1.4 Descrição da Estrutura do Trabalho 16
2 FUNDAMENTAÇÃO TEÓRICA 18
2.1 Síntese de Voz e *Deepfakes* de Áudio 18
2.2 Processamento Digital de Sinais de Áudio 19
2.3 Características Acústicas (*Features*) para Classificação 20
2.3.1 Coeficientes Cepstrais de Frequência Mel (MFCC) 20
2.3.2 Taxa de Cruzamento por Zero (ZCR) e Centroide Espectral 21
2.3.3 Espectrogramas 21
2.4 Algoritmos de *Machine Learning* para Classificação 22
2.4.1 *Random Forest* e *Support Vector Machine* (SVM) 22
2.4.2 Redes Neurais Convolucionais (CNN) 22
2.5 Trabalhos Relacionados e Estado da Arte 23
3 METODOLOGIA EXPERIMENTAL 28
3.1 Visão Geral do Protocolo Experimental 28
3.2 Conjunto de Dados e Universo do Experimento 30
3.3 Pré-Processamento 33
3.4 Ramo Clássico: Extração de Características 35
3.5 Ramo Profundo: Espectrograma Log-Mel e Arquitetura da CNN 37
3.6 Partição dos Dados, Braço Duplo e Validação Cruzada 41
3.7 Ajuste de Hiperparâmetros 43
3.8 Protocolo de Avaliação 46
3.9 Protocolo de Medição do Tempo de Inferência 48
3.10 Ambiente Computacional e Reprodutibilidade 49
4 DESENVOLVIMENTO 53
4.1 Organização do Código e Princípio da Fonte Única 53
4.2 *Pipeline* de Características e Lote Único de Reextração 54

4.3 Treinamento de *Random Forest* e SVM 56
4.4 Geração dos Espectrogramas e Treinamento da CNN 58
4.5 Problemas Encontrados e Controles de Integridade 63
5 RESULTADOS E DISCUSSÃO 67
5.1 Resultados na Validação 67
5.2 Resultados no Teste Lacrado 68
5.3 Da Validação para o Teste 70
5.4 Comparação Estatística: *Bootstrap* Pareado 72
5.5 Custo da Subamostragem: o Braço de Referência 73
5.6 Análise por Ataque e por *Codec* 74
5.7 Custo Computacional 77
5.8 Importância das Características no *Random Forest* 79
5.9 Discussão: Resposta à Pergunta de Pesquisa 81
5.10 Limitações 84
6 CONCLUSÃO 90
6.1 Considerações Finais 90
6.2 Trabalhos Futuros 91
REFERÊNCIAS 93

1 INTRODUÇÃO

Nos últimos anos, o avanço das técnicas de Inteligência Artificial, especialmente aquelas baseadas em aprendizado profundo (*deep learning*, DL), tem possibilitado a geração de conteúdos sintéticos cada vez mais realistas (Goodfellow; Bengio; Courville, 2016). No contexto do processamento de sinais de áudio, esse avanço se manifesta por meio de sistemas capazes de sintetizar vozes humanas com alto grau de fidelidade (Wang, Y. *et al.*, 2017) e, mais recentemente, de reproduzir a voz de um locutor específico a partir de poucos segundos de gravação, a chamada clonagem de voz (Wang, C. *et al.*, 2023). Avanços recentes em arquiteturas de *Text-to-Speech* (TTS) e *Voice Conversion* (VC) elevaram o realismo da fala sintética a patamares que tornam a sua diferenciação perceptual cada vez mais difícil (Singh *et al.*, 2026).

Embora tais tecnologias apresentem aplicações legítimas, como assistentes virtuais, acessibilidade e dublagem automatizada, seu uso indevido tem levantado preocupações significativas, especialmente no que diz respeito à segurança da informação, fraudes e disseminação de desinformação (Müller *et al.*, 2022). A capacidade de gerar vozes sintéticas muito difíceis de distinguir das vozes humanas reais abre espaço para ataques como *voice phishing (vishing)* e falsificação de identidade (Bateman, 2020; FinCEN, 2024), além de enfraquecer a confiança em gravações como evidência (Chesney; Citron, 2019).

Diante desse cenário, surge a necessidade de desenvolver mecanismos capazes de diferenciar automaticamente entre áudios gerados por humanos e aqueles produzidos por sistemas de inteligência artificial (Liu *et al.*, 2023). Essa tarefa configura um problema relevante na área de Ciência da Computação, envolvendo conceitos de processamento de sinais, aprendizado de máquina e análise de padrões acústicos. Estudos recentes têm comparado abordagens clássicas, como *Random Forest*, *Support Vector Machine* (SVM) e *Gradient Boosting*, com modelos baseados em redes profundas no contexto do *dataset* do desafio ASVspoof (*Automatic Speaker Verification Spoofing and Countermeasures Challenge*), edição de 2021 (Bhukya; Raj; Kumar, 2025), evidenciando a pertinência de uma comparação sistemática entre essas duas famílias de métodos.

Considerando o avanço das tecnologias de síntese de voz e os riscos associados ao seu uso indevido, este trabalho busca responder à seguinte questão: modelos clássicos (SVM e *Random Forest*) baseados em características acústicas manuais mantêm desempenho preditivo competitivo e viabilidade computacional, quando comparados a Redes Neurais Convolucionais

(CNNs) na detecção de *deepfakes* de áudio dentro do mesmo ambiente experimental? Neste trabalho, o desempenho preditivo é avaliado principalmente pelo F1-macro (média não ponderada do F1 das duas classes) e pela taxa de erro igual (EER), e não pela acurácia isolada, porque o desbalanceamento de 9:1 entre as classes *spoof* e *bonafide* faz com que a acurácia seja dominada pela classe majoritária (Seção 3.8).

1.1 Objetivos

1.1.1 Objetivo Geral

Desenvolver e avaliar modelos de *machine learning* capazes de classificar áudios como pertencentes a vozes humanas ou vozes sintéticas, a partir da análise de características acústicas extraídas dos sinais de áudio.

1.1.2 Objetivos Específicos

- Investigar técnicas de processamento de áudio e representação de sinais sonoros;
- Estudar características acústicas relevantes, como MFCC (*Mel-Frequency Cepstral Coefficients*), taxa de cruzamento por zero e características espectrais;
- Selecionar e preparar um conjunto de dados adequado contendo exemplos de vozes reais e sintéticas;
- Implementar métodos de extração de características a partir dos sinais de áudio;
- Treinar e avaliar diferentes modelos de *machine learning*, incluindo *Random Forest*, *Support Vector Machine* (SVM) e Redes Neurais Convolucionais (CNN);
- Comparar o desempenho dos modelos com base em métricas como acurácia, precisão, *recall*, F1-macro, taxa de erro igual (EER), área sob a curva característica de operação do receptor (ROC-AUC) e tempo de inferência;
- Analisar criticamente os resultados obtidos, identificando limitações e possíveis melhorias.

1.2 Justificativa

A crescente sofisticação das técnicas de geração de voz sintética, impulsionada pela maturação de sistemas TTS de última geração e pelo aprimoramento de arquiteturas de *Voice Conversion*, torna cada vez mais difícil distinguir conteúdos autênticos de conteúdos artificiais (Singh *et al.*, 2026), o que representa um desafio relevante para áreas como segurança digital, autenticação biométrica e verificação de identidade (Bateman, 2020; FinCEN, 2024; Müller *et al.*, 2022).

Do ponto de vista acadêmico, este trabalho contribui para a compreensão de como características acústicas podem ser utilizadas para identificar padrões distintivos entre vozes reais e sintéticas. Estudos recentes, como o de Bhukya, Raj e Kumar (2025), evidenciam que classificadores clássicos (SVM, *Random Forest*, *Gradient Boosting*, entre outros) podem alcançar resultados expressivos quando combinados com características acústicas adequadas, motivando uma comparação criteriosa frente a abordagens baseadas em redes profundas. Nesse sentido, a principal contribuição deste estudo é responder empiricamente à seguinte questão de pesquisa: modelos clássicos (SVM e *Random Forest*) baseados em características acústicas manuais mantêm desempenho preditivo competitivo e viabilidade computacional, quando comparados a Redes Neurais Convolucionais (CNNs) na detecção de *deepfakes* de áudio dentro do mesmo ambiente experimental? Adicionalmente, o trabalho permite a avaliação comparativa de diferentes algoritmos de aprendizado de máquina aplicados a esse problema, enriquecendo a literatura sobre detecção de *deepfakes* de áudio (Liu *et al.*, 2023).

Sob a perspectiva prática, os resultados deste estudo podem subsidiar o desenvolvimento de sistemas mais robustos de detecção de fraudes baseadas em áudio, contribuindo para mitigar riscos associados ao uso indevido de tecnologias de síntese de voz. Assim, o trabalho se justifica tanto pela relevância científica quanto pela aplicabilidade prática do problema investigado.

1.3 Metodologia

Este trabalho foi conduzido por meio de um estudo experimental de natureza comparativa, no qual técnicas de *machine learning* foram aplicadas à classificação de áudios em duas categorias: voz humana e voz sintética. Como base experimental, foi utilizado o conjunto de dados ASVspoof 2021, especificamente o subconjunto *Logical Access* (LA), por se tratar de um conjunto de referência consolidado para a detecção de ataques de *spoofing* baseados em síntese de voz e conversão de voz (Liu *et al.*, 2023). A escolha desse subconjunto justificou-se por ele incorporar diferentes condições de *codec* e de transmissão relevantes à avaliação de detectores e por ser amplamente utilizado como *benchmark* na literatura recente (Bhukya; Raj; Kumar, 2025; Zhang *et al.*, 2025). Como o ASVspoof 2021 LA não dispõe de partições próprias de treino e de desenvolvimento (Liu *et al.*, 2023), o universo do experimento foi delimitado aos 148.176 áudios da fase de avaliação oficialmente pontuada do desafio (*eval*), divididos internamente em treino, validação e teste, nas proporções de 70%, 15% e 15%.

Todos os áudios passaram por um mesmo pré-processamento, composto por normalização da amplitude, remoção de trechos silenciosos por meio de *Voice Activity*

Detection (VAD) e padronização da duração em 4,0 s. A partir dele, foram construídos dois ramos de representação: no ramo clássico, características acústicas extraídas manualmente (coeficientes MFCC, Taxa de Cruzamento por Zero e Centroide Espectral) alimentaram os classificadores *Random Forest* e SVM; no ramo profundo, espectrogramas Log-Mel alimentaram uma Rede Neural Convolucional (CNN). Os três modelos foram treinados na mesma subamostra estratificada de 30.000 áudios do conjunto de treino, e um segundo *Random Forest*, treinado no conjunto de treino completo, foi utilizado apenas para medir o custo dessa subamostragem. Os hiperparâmetros do *Random Forest* e do SVM foram ajustados por busca aleatória (*Random Search*) com validação cruzada estratificada em cinco partições, restrita ao conjunto de treino, e os da CNN, por uma grade reduzida de configurações. O desbalanceamento entre as classes foi tratado por ponderação de classes nos três modelos.

A avaliação adotou o F1-macro como critério principal, complementado por acurácia, precisão, *recall*, EER, ROC-AUC e tempo de inferência ponta a ponta. O limiar de decisão de cada modelo foi escolhido apenas no conjunto de validação, e o conjunto de teste foi avaliado uma única vez, com os limiares já fixados. A comparação entre os modelos foi feita por *bootstrap* pareado sobre as mesmas amostras de avaliação. Além das métricas globais, os resultados foram analisados por sistema de ataque e por *codec*, e a relevância das características acústicas foi estimada no *Random Forest*. O Capítulo 3 descreve o protocolo completo e apresenta, com as respectivas justificativas, os pontos em que a execução divergiu do planejamento inicial do trabalho.

1.4 Descrição da Estrutura do Trabalho

O presente trabalho está organizado da seguinte forma:

- Capítulo 1 (Introdução): apresenta a contextualização do tema, o problema e a pergunta de pesquisa, os objetivos, a justificativa e uma visão geral da metodologia;
- Capítulo 2 (Fundamentação Teórica): aborda os conceitos de processamento de áudio, as características acústicas, os algoritmos de aprendizado de máquina e os trabalhos relacionados;
- Capítulo 3 (Metodologia Experimental): descreve o conjunto de dados, o pré-processamento, os dois ramos de representação (características manuais e espectrogramas Log-Mel), a partição dos dados, o ajuste de hiperparâmetros e os protocolos de avaliação e de medição de tempo;
- Capítulo 4 (Desenvolvimento): relata a implementação e a execução do experimento, os controles de integridade adotados e os problemas encontrados;

- Capítulo 5 (Resultados e Discussão): apresenta o desempenho dos modelos na validação e no teste, a comparação estatística, as análises por ataque e por *codec* e o custo computacional, discute a resposta à pergunta de pesquisa e declara as limitações;
- Capítulo 6 (Conclusão): sintetiza os principais achados e propõe trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Síntese de Voz e *Deepfakes* de Áudio

A síntese de voz, amplamente conhecida pela sigla TTS (*Text-to-Speech*), refere-se ao processo por meio do qual sistemas computacionais transformam texto escrito em fala humana artificial. Os primeiros sistemas de síntese produziam vozes nitidamente artificiais, o que limitava sua aceitação em interações cotidianas. Com a maturação das redes neurais profundas, contudo, observou-se um avanço significativo na qualidade da fala gerada, possibilitando a reprodução de aspectos prosódicos como expressividade e ritmo (Goodfellow; Bengio; Courville, 2016).

Nesse contexto evolutivo, surgiram modelos que se consolidaram como marcos da área, como o Tacotron (Wang, Y. *et al.*, 2017) e o WaveNet (Oord *et al.*, 2016), que cumprem papéis diferentes na cadeia de síntese. O Tacotron aprende a gerar, a partir do texto, uma representação espectral da fala, da qual a forma de onda é obtida por uma etapa posterior de reconstrução, o algoritmo de Griffin-Lim no artigo original. O WaveNet, por sua vez, é um modelo generativo que produz diretamente a forma de onda do áudio, amostra a amostra, na função que cabe a um *vocoder*. A inovação central desses sistemas reside na capacidade de aprender essas etapas a partir de dados, com mínima interferência humana. O resultado é uma fala com fluidez e entonação próximas das observadas em locutores reais, o que ampliou aplicações legítimas, como assistentes de voz, ferramentas de acessibilidade e a produção de conteúdo criativo e de entretenimento (Negreiro, 2025; Shi *et al.*, 2025; Singh *et al.*, 2026). Mais recentemente, novos sistemas TTS, incluindo arquiteturas baseadas em *Large Language Models* (LLMs) e modelos não autorregressivos, têm elevado ainda mais o realismo da fala sintética (Singh *et al.*, 2026).

Em contrapartida, a alta qualidade da fala sintética viabilizou também o surgimento dos chamados *deepfakes* de áudio. Enquanto a síntese de voz para fins legítimos busca apoiar usuários por meio de assistentes virtuais e ferramentas de acessibilidade, o *deepfake* sonoro emprega a mesma base tecnológica para fins ilícitos. A partir de amostras curtas da voz de uma pessoa real, sistemas recentes conseguem reproduzir com alta similaridade a voz do locutor, em alguns casos com apenas três segundos de gravação (Wang, C. *et al.*, 2023), e produzir *deepfakes* de áudio cada vez mais sofisticados e difíceis de detectar (Negreiro, 2025). Em um caso de fraude relatado, em que criminosos aparentemente usaram clonagem de voz para se passar por um CEO (*Chief Executive Officer*) alemão, o subordinado britânico afirmou ter reconhecido o leve sotaque e a melodia da voz do chefe ao telefone (Bateman, 2020).

Essa facilidade na manipulação da identidade vocal traz implicações relevantes para a autenticidade de gravações reais. Ataques de *voice phishing (vishing)*, nos quais criminosos se passam por outra pessoa, como um familiar ou um executivo da empresa, para induzir transferências financeiras (Bateman, 2020; FinCEN, 2024), inserem-se em um quadro mais amplo: desde 2023, o *Financial Crimes Enforcement Network*, FinCEN (2024), observa aumento de relatos de atividade suspeita que descrevem o uso de *deepfakes* em fraudes contra instituições financeiras e seus clientes. É nesse contexto de vulnerabilidade que o desenvolvimento de ferramentas de detecção robustas se torna essencial, atuando como uma linha de defesa para preservar a confiabilidade nas comunicações digitais (Müller *et al.*, 2022; Zhang *et al.*, 2025).

2.2 Processamento Digital de Sinais de Áudio

A voz humana, em sua essência, é um fenômeno físico contínuo, caracterizado por variações de pressão do ar que se propagam até o sistema auditivo. Para que sistemas computacionais, incluindo arquiteturas de aprendizado profundo (Goodfellow; Bengio; Courville, 2016), possam analisar e extrair padrões dessa onda sonora, é necessário traduzir o som para o domínio digital. Esse processo de conversão fundamenta-se em duas etapas principais: a amostragem (*sampling*) e a quantização.

A amostragem define a frequência com que o sinal sonoro contínuo é medido ao longo do tempo. No contexto da detecção de *deepfakes*, a preservação de uma alta taxa de amostragem é uma decisão arquitetônica relevante (Zhang *et al.*, 2025), uma vez que a perda de banda e a degradação introduzidas pela compressão por *codecs* e pela transmissão reduzem o desempenho dos detectores (Shi *et al.*, 2025). A quantização, por sua vez, complementa esse processo mapeando as medições temporais para valores numéricos discretos, permitindo o armazenamento e a interpretação computacional da intensidade da onda.

Capturar o áudio com qualidade, contudo, resolve apenas parte do problema. O sinal de fala possui caráter não estacionário: suas características (energia, tom e frequências predominantes) variam em frações de segundo, dependendo da consoante ou sílaba pronunciada. A análise da gravação como um todo mascararia tais nuances. Para contornar esse aspecto, aplica-se a técnica de janelamento (*windowing*), que segmenta o sinal em quadros temporais curtos (em geral, entre 20 e 30 milissegundos), suficientemente breves para que o comportamento espectral do som possa ser considerado aproximadamente estacionário.

A partir dessas janelas, aplica-se a Transformada de Fourier, ferramenta clássica do processamento de sinais que decompõe a onda no domínio do tempo em suas componentes de

frequência. Aplicando-se essa operação de forma sucessiva sobre as janelas, obtém-se a representação do áudio sob a forma de espectrogramas.

O espectrograma fornece uma representação visual do conteúdo sonoro, mapeando o tempo no eixo horizontal, as frequências no eixo vertical e a energia sonora pela intensidade de cores. Conforme apontado pela literatura recente (Liu *et al.*, 2023; Majid, 2026), alimentar modelos de *machine learning* com espectrogramas tem se mostrado uma abordagem efetiva para a identificação de assinaturas acústicas associadas à fala sintética, contribuindo para a classificação automática de áudios em cenários de detecção de *deepfakes*.

2.3 Características Acústicas (*Features*) para Classificação

2.3.1 Coeficientes Cepstrais de Frequência Mel (MFCC)

A extração de características corresponde à etapa em que se passa da onda bruta para representações mais informativas e adequadas ao algoritmo de classificação. Nesse contexto, os Coeficientes Cepstrais de Frequência Mel (MFCC) ocupam posição central no processamento de voz. O diferencial dos MFCCs decorre de sua inspiração perceptual: em vez de analisar o som de forma puramente linear, eles aproximam o modo como a cóclea do ouvido humano percebe as frequências.

O cálculo dos MFCCs inicia-se com a aplicação da Transformada de Fourier e prossegue com a aplicação da Escala Mel, de natureza logarítmica. Tal escolha decorre do fato de que a audição humana é mais sensível a variações em frequências baixas, perdendo precisão em frequências mais elevadas. A aplicação de filtros baseados nessa escala condensa os dados, e os coeficientes resultantes representam aspectos do envelope espectral associados às características do trato vocal, relacionadas, em última análise, ao timbre do locutor.

Na detecção de *deepfakes*, essa técnica tem se mostrado eficaz, conforme sustentado pela literatura recente (Bhukya; Raj; Kumar, 2025; Zhang *et al.*, 2025). De acordo com os trabalhos da área, modelos geradores baseados em redes neurais profundas, como o Tacotron, podem apresentar dificuldades em reproduzir, com fidelidade, certas transições acústicas e a complexidade física do trato vocal humano, o que tende a se manifestar em forma de divergências estatísticas detectáveis em representações cepstrais. Os coeficientes de ordem mais baixa descrevem o envelope espectral, e os de ordem mais alta tendem a representar variações espectrais mais finas, fornecendo a algoritmos classificadores (como SVM e *Random Forest*) um vetor de características denso, no qual essas divergências entre fala humana e fala sintética podem ser exploradas como critério de discriminação.

2.3.2 Taxa de Cruzamento por Zero (ZCR) e Centroide Espectral

Enquanto o MFCC analisa, em geral, a forma e o timbre do som, outras características concentram-se no comportamento da onda em escalas microscópicas. A Taxa de Cruzamento por Zero (ZCR, do inglês *Zero Crossing Rate*) mede a frequência com que a onda sonora cruza o eixo zero de amplitude em um determinado intervalo de tempo. Sons fortemente ruidosos, como as consoantes fricativas surdas da fala humana (por exemplo, os fonemas associados às letras /s/ e /f/), geram transições rápidas e, consequentemente, ZCR mais elevado. Por responder a componentes de alta frequência, o ZCR pode captar ruídos ou distorções que o processo de síntese introduza nessa região do espectro, o que motivou sua inclusão entre as características do ramo clássico.

Em paralelo, o Centroide Espectral atua no domínio da frequência e funciona como o "centro de massa" do espectro sonoro, indicando matematicamente onde se concentra a maior parte da energia acústica e estando associado à percepção de "brilho" da voz. Em áudios sintéticos, diferenças na distribuição de energia ao longo do espectro podem deslocar o centroide em relação ao de vozes reais, o que justifica testá-lo como característica discriminativa; neste trabalho, sua contribuição foi medida na Seção 5.8.

2.3.3 Espectrogramas

Diferentemente dos descritores anteriores, que condensam o áudio em vetores numéricos, os espectrogramas representam o som como uma imagem bidimensional. O eixo horizontal corresponde ao tempo, o eixo vertical mapeia as frequências e a intensidade de cor de cada *pixel* indica a energia sonora associada a determinado instante. Essa conversão é fundamental para abordagens contemporâneas de detecção, pois a representação tempo-frequência permite aplicar convoluções bidimensionais para explorar padrões locais do espectrograma.

Ao representar a voz matricialmente como uma imagem, viabiliza-se a utilização de Redes Neurais Convolucionais (CNNs). Conforme sustentado pela literatura recente (Majid, 2026; Zhang *et al.*, 2025), CNNs têm sido aplicadas a espectrogramas com o objetivo de aprender automaticamente representações discriminativas a partir das chamadas impressões digitais espectro-temporais associadas à fala sintética. De acordo com esses estudos, tal abordagem permite a identificação de padrões correlacionados a *deepfakes* (como divergências em altas frequências, blocos repetitivos ou descontinuidades entre fonemas) sem a necessidade de engenharia manual de *features*. Resultados obtidos em iniciativas como o desafio ASVspoof

(Liu *et al.*, 2023) sustentam a relevância da formulação do problema como uma tarefa de classificação sobre representações tempo-frequência.

2.4 Algoritmos de *Machine Learning* para Classificação

2.4.1 *Random Forest* e *Support Vector Machine* (SVM)

O *Random Forest* (Floresta Aleatória) opera segundo a lógica de *ensemble*: durante o treinamento, são construídas múltiplas árvores de decisão independentes; no momento da classificação, cada árvore emite um voto, e a predição final corresponde à classe majoritária. O SVM (*Support Vector Machine*), por sua vez, atua de forma geométrica, projetando os dados em um espaço multidimensional e buscando o hiperplano ótimo que separa as amostras das classes em análise com a maior margem possível.

Entre as vantagens do *Random Forest* estão a tendência a reduzir o *overfitting* em relação a árvores de decisão isoladas e o tratamento adequado de bases de dados moderadamente desbalanceadas. Como limitação, modelos com grande número de árvores podem se tornar custosos para predições em tempo real. Já o SVM destaca-se pela eficiência em espaços de alta dimensionalidade, como os vetores produzidos por extratores de características acústicas, e pela capacidade de tratar problemas não linearmente separáveis por meio de funções de *kernel*. Em contrapartida, o SVM tende a apresentar custo elevado de treinamento em conjuntos de dados de grande porte.

Na detecção de *deepfakes*, esses algoritmos são empregados como classificadores tabulares aplicados sobre vetores de características acústicas extraídas dos áudios. Conforme apontam Bhukya, Raj e Kumar (2025) e Zhang *et al.* (2025), tais algoritmos têm sido utilizados na literatura tanto como *baselines* comparativas, quanto como soluções competitivas em determinadas combinações de *features*, oferecendo respostas binárias com bom grau de interpretabilidade e custo computacional reduzido em relação a redes profundas.

2.4.2 Redes Neurais Convolucionais (CNN)

A arquitetura de uma CNN inspira-se na organização do córtex visual biológico. Ela é composta por camadas de convolução intercaladas com camadas de redução (*pooling*). Nas camadas de convolução, matrizes numéricas denominadas filtros deslizam sobre os dados de entrada, executando operações que identificam, nas camadas iniciais, padrões simples (como bordas) e, em camadas mais profundas, padrões progressivamente mais complexos.

Em vez de tratar o áudio como sequência numérica unidimensional, a CNN recebe os dados no formato de espectrogramas. Tal conversão transforma o áudio em uma imagem (cujos

eixos correspondem ao tempo e à frequência e cuja intensidade representa a energia sonora), permitindo que a classificação seja abordada com convoluções bidimensionais, técnica amplamente empregada em reconhecimento de imagens.

A CNN processa o espectrograma e aprende, de maneira automática, padrões associados a artefatos sintéticos. Conforme sustentado pela literatura recente, abordagens baseadas em CNN têm apresentado desempenho competitivo, e em diversos cenários superior, ao de métodos clássicos na detecção automática de voz sintética (Liu *et al.*, 2023; Majid, 2026; Zhang *et al.*, 2025). Ressalta-se, contudo, que tal vantagem é condicionada a fatores como o volume e a qualidade dos dados de treinamento, a capacidade de generalização frente a domínios e *codecs* não vistos, e o custo computacional envolvido (Müller *et al.*, 2022; Shi *et al.*, 2025), aspectos que motivam a comparação experimental sistemática proposta no presente trabalho.

2.5 Trabalhos Relacionados e Estado da Arte

A detecção de *deepfakes* de áudio é um campo de pesquisa que ganhou tração significativa nos últimos anos, impulsionado tanto pela evolução dos métodos de síntese quanto pela urgência em combater seus usos maliciosos. Nesta seção, são revisados trabalhos selecionados sobre o tema, evidenciando as abordagens adotadas, os avanços alcançados e as lacunas que ainda persistem.

Um dos marcos mais importantes para a comunidade científica foi o desafio ASVspoof, uma competição bienal dedicada ao estudo de ataques de *spoofing* contra sistemas de verificação automática de locutor. Em sua edição de 2021, documentada por Liu *et al.* (2023), o desafio introduziu pela primeira vez uma tarefa específica voltada à detecção de *deepfakes* de áudio, reconhecendo que o problema já havia ultrapassado as fronteiras da segurança biométrica para se tornar uma ameaça social mais ampla. A edição contou com a participação de 54 equipes e trouxe inovações importantes em relação às anteriores: os participantes não receberam dados de treinamento compatíveis com os dados de avaliação, simulando condições reais nas quais a natureza do áudio falsificado é desconhecida. Além disso, os dados de avaliação da tarefa de *deepfake* foram processados com diferentes *codecs* de compressão com perda, reproduzindo o cenário típico de áudios compartilhados em redes sociais e plataformas de mídia. Apesar dessas dificuldades adicionais, os melhores sistemas alcançaram taxas de erro iguais (EER) próximas a 1,32% na tarefa de acesso lógico (Yamagishi *et al.*, 2021), demonstrando o progresso substancial da área. No entanto, os próprios organizadores reconheceram uma limitação persistente: as soluções de detecção, embora resilientes aos efeitos de compressão, ainda

carecem de capacidade de generalização quando expostas a conjuntos de dados de origens diferentes daqueles usados no treinamento.

Em uma direção complementar, Bhukya, Raj e Kumar (2025) realizaram um estudo sobre o *dataset* ASVspoof 2021 utilizando um conjunto de classificadores tradicionais e baseados em *deep learning*, incluindo *k-Nearest Neighbour* (k-NN), *Support Vector Machine* (SVM), *Random Forest* (RF), *Gradient Boosting* (GB), AdaBoost, XGBoost, *Multi-Layer Perceptron* (MLP) e arquiteturas profundas customizadas. Os autores empregaram uma combinação híbrida de características acústicas (como MFCCs, *Mel-Spectrograms*, Cromagramas, Contraste Espectral e características Tonnetz) sobre as três tarefas de avaliação do *dataset*: LA, PA (*Physical Access*) e DF (*DeepFake*). Os resultados reportados indicam acurácias entre 90% e 96% para os classificadores clássicos avaliados, evidenciando que algoritmos clássicos podem alcançar desempenho expressivo quando alimentados por conjuntos cuidadosamente construídos de *features* acústicas. Esse trabalho fornece evidências relevantes para sustentar a comparação direta entre abordagens clássicas e profundas dentro de um mesmo protocolo experimental, motivação central também adotada pelo presente estudo.

A fragilidade de generalização identificada no ASVspoof 2021 foi justamente o foco central do trabalho de Müller *et al.* (2022), que realizou uma investigação sistemática sobre o desempenho de detectores de *deepfake* fora de seus ambientes controlados de treinamento. Os autores reimplementaram e avaliaram de forma uniforme diversas arquiteturas da literatura, incluindo modelos baseados em redes residuais (ResNet), *Transformers* e redes convolucionais mais leves, padronizando etapas de pré-processamento e hiperparâmetros para isolar os fatores que realmente contribuem para o sucesso da detecção. Um achado particularmente relevante foi a identificação de que o uso de representações espectrais baseadas em CQT (*Constant-Q Transform*) ou em espectrogramas logarítmicos supera consistentemente representações baseadas na escala Mel, com uma melhoria média de 37% na taxa de erro. Contudo, o resultado mais impactante do estudo emergiu quando os modelos treinados no *benchmark* ASVspoof foram testados em um conjunto de dados inédito, composto por 37,9 horas de gravações reais de celebridades e políticos coletadas da internet, das quais 17,2 horas eram *deepfakes*. A degradação de desempenho observada chegou a até 1.000%, sugerindo que a comunidade pode ter ajustado suas soluções de forma excessiva ao *benchmark* ASVspoof, e que a detecção de *deepfakes* em condições reais é consideravelmente mais difícil do que os resultados laboratoriais faziam supor.

Em uma perspectiva mais ampla, Zhang *et al.* (2025) publicaram um *survey* abrangente que mapeia o estado da arte da detecção de *deepfakes* de áudio, cobrindo desde os métodos de

geração (TTS e conversão de voz) até as abordagens de detecção mais recentes. O trabalho organiza o campo em três grandes eixos: a extração de características no *frontend* (incluindo *features* artesanais como MFCC, LFCC [*Linear Frequency Cepstral Coefficients*] e CQCC [*Constant-Q Cepstral Coefficients*], além de representações aprendidas por modelos de aprendizado autossupervisionado), os modelos de classificação no *backend* (como LCNN [*Light Convolutional Neural Network*], ResNet, *Transformers* e redes baseadas em grafos) e os sistemas fim a fim que dispensam etapas manuais de extração de características. Os autores destacam que, embora os modelos baseados em aprendizado profundo tenham demonstrado desempenho expressivo nos *benchmarks* existentes, com alguns sistemas ultrapassando o limiar de 99% de acurácia em bases como o ASVspoof 2019-LA, os desafios de generalização, robustez a ataques adversariais e detecção de *deepfakes* parciais permanecem como fronteiras abertas. O *survey* também chama atenção para tópicos emergentes como a preservação de privacidade na detecção, a explicabilidade dos modelos e a equidade (*fairness*) nos sistemas de classificação, sinalizando que o campo ainda possui ampla margem para evolução.

Complementando a dimensão técnica, Bateman (2020) trouxe uma perspectiva aplicada ao analisar, em documento de trabalho (*working paper*), os cenários de ameaça que *deepfakes* e mídias sintéticas representam para o sistema financeiro. O autor inclui o *voice phishing* (*vishing*), no qual vozes clonadas são usadas para se passar por executivos ou familiares da vítima, entre as técnicas prioritárias, embora avalie que, à época, a ameaça financeira era baixa e baseada em poucos casos documentados. Entre as recomendações, estão ações internas das instituições, como reavaliar métodos de autenticação de clientes baseados em voz ou face, além de ações do setor e de outros atores, e o autor cita tecnologias *antispam* e de *antispoofing* como contramedidas possíveis. Em linha com essa preocupação, o FinCEN (2024) registrou, desde 2023, aumento de relatos de atividade suspeita envolvendo *deepfakes* e apontou, entre as boas práticas, a autenticação multifator.

A análise conjunta desses trabalhos sugere um panorama claro: enquanto os avanços em modelos de detecção são notáveis, entre os trabalhos revisados nesta pesquisa, a maior parte concentra-se em arquiteturas de *deep learning* de alta complexidade, frequentemente avaliadas em condições controladas e com pouca atenção à comparação direta entre abordagens clássicas e profundas dentro de um mesmo *pipeline* experimental. Entre eles, os que utilizam *Random Forest* ou SVM tendem a empregá-los como *baselines* rápidas, sem explorar de forma aprofundada quais características acústicas são mais discriminativas para esses classificadores. Por outro lado, os que empregam CNNs frequentemente adotam espectrogramas como entrada sem contrastar seus resultados com os obtidos a partir de *features* artesanais tradicionais

processadas por algoritmos mais simples. É nessa lacuna que o presente trabalho se posiciona. Ao propor uma comparação sistemática entre classificadores clássicos (*Random Forest* e SVM) alimentados por *features* acústicas extraídas manualmente (MFCC, ZCR e Centroide Espectral) e uma Rede Neural Convolucional (CNN) operando sobre espectrogramas, este estudo busca oferecer uma visão integrada que permita avaliar, dentro de um mesmo protocolo experimental, quais combinações de técnicas e representações são mais eficazes para a tarefa de detecção de voz sintética.

Quadro 1 – Comparativo dos Trabalhos Relacionados

Autor/Ano	**Problema Abordado**	**Solução Proposta**	**Principais Resultados**	**Diferencial deste Trabalho**
Bateman (2020)	Ameaças de *deepfakes* ao sistema financeiro (*vishing*)	Análise qualitativa de cenários de ameaça e recomendações	Ameaça considerada baixa em 2020; reavaliar autenticação por voz ou face	Este trabalho fornece um *pipeline* técnico de detecção automática que pode embasar soluções para o contexto financeiro identificado
Liu *et al.* (2023); Yamagishi *et al.* (2021)	Detecção de *spoofing* e *deepfakes* de áudio em condições reais	Desafio ASVspoof 2021 com modelos baseados em *deep learning* e espectrogramas	EER de 1,32% em acesso lógico; limitação de generalização entre *datasets*	Este trabalho compara explicitamente abordagens clássicas (RF, SVM) com CNN dentro de um mesmo protocolo experimental
Müller *et al.* (2022)	Generalização de detectores de *deepfake* fora do ambiente de treino	Avaliação sistemática de ResNet, *Transformers* e CNNs com diferentes representações espectrais	Degradação de até 1.000% ao testar em dados reais; CQT supera Mel em 37%	Este trabalho avalia *features* artesanais (MFCC, ZCR) vs. espectrogramas para CNN, contribuindo para a análise de representações discriminativas
Bhukya; Raj; Kumar (2025)	Detecção de locuções falsificadas (*spoofed utterances*) no ASVspoof 2021 (LA, PA e DF)	Comparação de classificadores clássicos (k-NN, SVM, RF, GB, AdaBoost, XGBoost, MLP) e DL com *features* híbridas (MFCC, *Mel-Spectrograms*, Tonnetz, etc.)	Acurácia entre 90% e 96% para classificadores clássicos	Este trabalho avalia, no mesmo protocolo, RF e SVM com *features* manuais (MFCC, ZCR, centroide) frente a CNN com espectrogramas, com foco em viabilidade computacional

(continua)

(continuação)

Autor/Ano	Problema Abordado	Solução Proposta	Principais Resultados	Diferencial deste Trabalho
Zhang *et al.* (2025)	Mapeamento do estado da arte em detecção de *deepfakes* de áudio	*Survey* de métodos *frontend* (MFCC, LFCC, CQCC) e *backend* (LCNN, ResNet, *Transformers*)	Modelos profundos superam 99% de acurácia em ASVspoof 2019-LA; desafios de *fairness* e explicabilidade persistem	Este trabalho implementa e compara empiricamente abordagens de diferentes complexidades, com foco em aplicabilidade prática

Fonte: Elaborado pelo autor (2026).

A partir da análise dos trabalhos correlatos, nota-se que a literatura adota predominantemente a Taxa de Erro Igual (EER, do inglês *Equal Error Rate*), a Acurácia e o F1-score como métricas padrão para avaliar o desempenho de modelos de detecção de *deepfakes* de áudio. Para que os resultados fossem reportados nas mesmas métricas adotadas na literatura, o protocolo experimental incluiu essas dimensões; a comparação direta com os valores publicados, porém, é limitada pelas diferenças de protocolo, como discutido na Seção 5.9. Adicionalmente, o tempo de inferência computacional foi registrado, com o objetivo de comparar a viabilidade operacional entre os modelos clássicos (*Random Forest* e SVM) e a Rede Neural Convolucional (CNN), permitindo discutir não apenas qual abordagem oferece maior poder preditivo, mas também qual delas apresenta o melhor compromisso entre desempenho e custo computacional para aplicações reais.

Dessa forma, o trabalho contribui para preencher uma lacuna metodológica identificada entre os trabalhos revisados, fornecendo subsídios para pesquisadores e desenvolvedores que precisam escolher entre abordagens de diferentes níveis de complexidade para aplicações práticas de segurança digital.

3 METODOLOGIA EXPERIMENTAL

Este capítulo descreve o protocolo experimental construído para responder à pergunta de pesquisa apresentada na Seção 1.2: se modelos clássicos alimentados por características acústicas manuais mantêm desempenho preditivo competitivo e viabilidade computacional, frente a uma rede neural convolucional, dentro do mesmo ambiente experimental. A Seção 3.1 apresenta a visão geral do protocolo, resumida na Figura 1. As Seções 3.2 a 3.5 tratam dos dados, do pré-processamento comum e dos dois ramos de representação; as Seções 3.6 a 3.8, da partição dos dados, do ajuste de hiperparâmetros e do protocolo de avaliação; e as Seções 3.9 e 3.10, da medição do tempo de inferência e do ambiente computacional.

O capítulo descreve o desenho do experimento, isto é, o que foi feito e por quê. A execução, com os controles de integridade e os problemas encontrados, é relatada no Capítulo 4, e os resultados, no Capítulo 5. Os parâmetros centrais do protocolo foram lidos do arquivo de configuração do experimento, e os parâmetros próprios de uma única etapa ficaram declarados no código dessa etapa (Seção 4.1). Todo valor numérico citado provém de um artefato versionado no repositório do trabalho; a correspondência entre cada tabela, quadro ou figura e o seu artefato está registrada em `docs/execucao/RASTREABILIDADE_TABELAS_FIGURAS.md`. O repositório está disponível em https://github.com/EduardoZanellaBento/Tcc_Deepfake.

3.1 Visão Geral do Protocolo Experimental

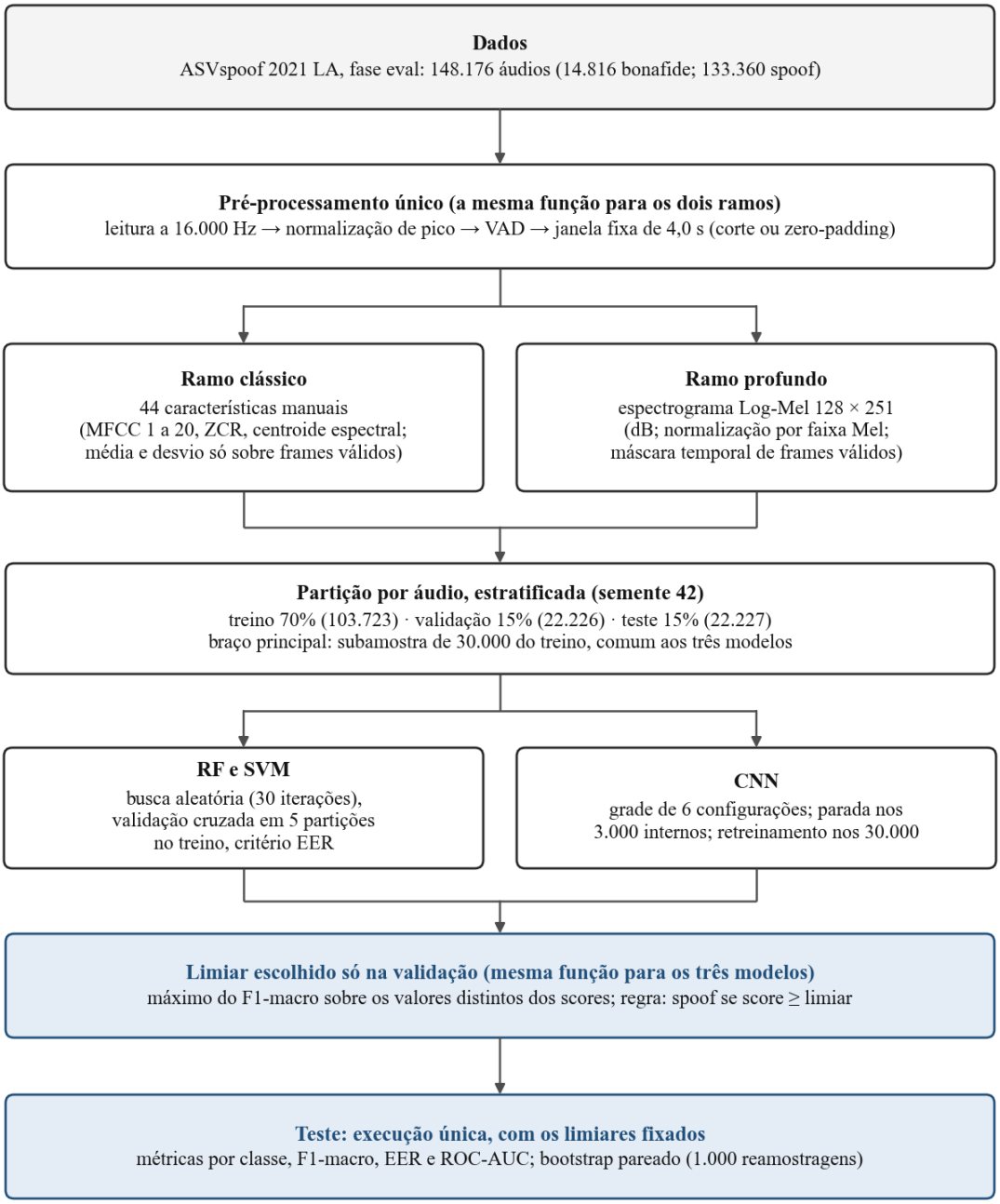
O trabalho consistiu num estudo experimental de natureza comparativa, no qual três classificadores foram treinados e avaliados sob um mesmo protocolo: *Random Forest* (RF), máquina de vetores de suporte (SVM) com *kernel* de função de base radial (RBF) e uma rede neural convolucional (CNN). O desenho partiu do princípio de que tudo o que não fosse objeto da comparação deveria ser controlado ou explicitamente declarado. Por isso, os áudios passaram por um único pré-processamento, implementado numa única função do código, cuja saída alimentou dois ramos de representação. No ramo clássico, cada áudio foi resumido em 44 características acústicas manuais, entregues ao RF e ao SVM; no ramo profundo, o mesmo sinal foi convertido num espectrograma Log-Mel de formato fixo, entregue à CNN.

Com esse arranjo, a diferença entre os dois *pipelines* ficou reduzida principalmente à forma de representação e de aprendizado: no ramo clássico, as características acústicas foram resumidas e classificadas por RF e SVM; na CNN, a estrutura tempo-frequência foi preservada e a representação, aprendida pela rede. Os demais elementos do protocolo foram

compartilhados. A partição dos dados, gravada em disco como artefato versionado, foi a mesma para os três modelos, e os três tiveram a mesma amostra final de treinamento no braço principal, uma subamostra estratificada de 30.000 áudios (Seção 3.6).

A avaliação também seguiu uma régua única. As funções de avaliação, de seleção do limiar e de cálculo do EER foram reunidas num único módulo do código e compartilhadas pelos três *pipelines*, de modo que a regra de decisão, a escolha do limiar e o cálculo das métricas não pudessem divergir entre eles. O limiar de cada modelo foi escolhido apenas no conjunto de validação, e o conjunto de teste foi reservado para uma única execução final, com os limiares já fixados (Seção 3.8). A Figura 1 resume a sequência completa, dos dados ao teste. O princípio de fonte única que a sustenta no código, segundo o qual nenhuma etapa foi reimplementada em mais de um lugar, é detalhado na Seção 4.1.

Figura 1 – Fluxograma do protocolo experimental



Fonte: Elaborado pelo autor (2026).

3.2 Conjunto de Dados e Universo do Experimento

A base experimental foi o subconjunto *Logical Access* (LA) do ASVspoof 2021, que reúne áudios de fala genuína e de fala gerada artificialmente, transmitidos por diferentes *codecs* e canais. O próprio conjunto de dados rotula as duas classes como *bonafide* (fala genuína) e *spoof* (fala sintetizada ou convertida), e este trabalho adotou os mesmos rótulos. O arquivo de chaves do subconjunto lista 181.566 áudios, distribuídos em três fases: *eval* (148.176), *progress*

(16.464) e *hidden* (16.926). O universo do experimento foi delimitado à fase *eval*, o conjunto oficialmente pontuado do desafio: 148.176 áudios, dos quais 14.816 *bonafide* e 133.360 *spoof*, uma razão de 9,0 áudios *spoof* para cada *bonafide*.

As fases *progress* e *hidden* foram excluídas por controle metodológico. A fase *hidden* contém áudios cujo silêncio já foi cortado na origem, conforme indicado no arquivo de chaves, isto é, submetidos a um pré-processamento distinto do aplicado aos demais, o que contaminaria qualquer análise baseada na proporção de fala retida pelo detector de atividade de voz (Seção 3.3). A homogeneidade do universo foi verificada por *script*: todos os 148.176 áudios da fase *eval* estão marcados, no arquivo de chaves, como não cortados. As consequências dessa delimitação são discutidas na Seção 5.10.

O planejamento inicial do trabalho mencionava apenas o ASVspoof 2021 LA; a delimitação à fase *eval* foi uma precisão introduzida na execução. Ela se liga a uma característica do conjunto: o ASVspoof 2021 LA não disponibiliza partições próprias de treino e de desenvolvimento com as condições de *codec* e de transmissão da avaliação (Liu *et al.*, 2023). Para treinar, ajustar e avaliar os três modelos dentro de um mesmo conjunto, foi necessária uma partição interna do universo, descrita na Seção 3.6.

Registra-se também uma divergência em relação ao protocolo oficial do desafio. No ASVspoof 2021, os sistemas avaliados na partição LA são treinados com as partições de treino e de desenvolvimento do ASVspoof 2019 LA, cujos ataques (A01 a A06) não passam por *codecs* de transmissão, e testados no 2021 LA, cujos ataques (A07 a A19) estão em maioria ausentes do treino (Liu *et al.*, 2023; Yamagishi *et al.*, 2021). Esse protocolo mede a generalização a ataques e a canais não vistos. Este trabalho não o seguiu: a escolha do ASVspoof 2021 LA, com partição interna 70/15/15, foi feita no planejamento inicial e se justificou pelo fato de o conjunto incorporar diferentes condições de *codec* e de transmissão relevantes à avaliação de detectores e de ser usado como *benchmark* na literatura recente (Seção 1.3). O desenho responde à pergunta de pesquisa deste trabalho, que é comparar famílias de modelos no mesmo ambiente, com as mesmas condições de ataque e de *codec* presentes no treino dos três. O que esse desenho não permite afirmar é discutido na Seção 5.10, e o protocolo oficial fica como trabalho futuro (Seção 6.2).

A Tabela 1 apresenta a composição do universo por *codec* e por classe. Os sete *codecs* dividem-se em dois grupos: banda estreita (alaw, gsm, pstn e ulaw), com teto próximo de 4 kHz, que somam 85.860 áudios (57,9% do universo), e banda larga (g722, none e opus), com 62.316 áudios (42,1%). A proporção de *bonafide* é praticamente a mesma em todos os *codecs*, entre 9,9% e 10,0%. Essa composição fundamentou duas decisões: não limitar a banda das

representações a 4 kHz, o que destruiria sinal real acima dessa frequência em 42,1% dos dados (Seções 3.4 e 3.5), e reportar as métricas também por *codec* (Seção 5.6).

Tabela 1 – Composição do universo do experimento (fase *eval*) por *codec* e por classe

Codec	**Banda**	***Bonafide***	***Spoof***	**Total**	**% do universo**
alaw	estreita	1.924	17.512	19.436	13,12
gsm	estreita	2.356	21.164	23.520	15,87
pstn	estreita	1.938	17.446	19.384	13,08
ulaw	estreita	2.356	21.164	23.520	15,87
Subtotal	**estreita**	8.574	77.286	85.860	57,94
g722	larga	1.937	17.438	19.375	13,08
none	larga	1.949	17.472	19.421	13,11
opus	larga	2.356	21.164	23.520	15,87
Subtotal	**larga**	6.242	56.074	62.316	42,06
Total		14.816	133.360	148.176	100,00

Fonte: Elaborado pelo autor (2026).
Nota: Banda estreita: teto próximo de 4 kHz (telefonia). Os percentuais das linhas são calculados sobre o total do universo. Banda de cada *codec* conforme `scripts/diagnostico_por_codec.py`.

Os treze sistemas de ataque, identificados de A07 a A19, aparecem em proporções praticamente iguais, entre 6,85% e 7,01% do universo cada um (Tabela 2). A composição por ataque e por *codec* foi usada como critério de estratificação da subamostra de treino (Seção 3.6) e, na análise dos resultados, como eixo de diagnóstico (Seção 5.6).

Tabela 2 – Composição do universo do experimento (fase *eval*) por classe e por sistema de ataque

Classe	**Ataque**	**Áudios**	**% do universo**
spoof	A07	10.238	6,91
spoof	A08	10.368	7,00
spoof	A09	10.152	6,85
spoof	A10	10.318	6,96
spoof	A11	10.276	6,93
spoof	A12	10.259	6,92
spoof	A13	10.302	6,95
spoof	A14	10.234	6,91
spoof	A15	10.235	6,91
spoof	A16	10.390	7,01
spoof	A17	10.239	6,91
spoof	A18	10.148	6,85
spoof	A19	10.201	6,88

(continua)

(continuação)

Classe	Ataque	Áudios	% do universo
spoof	**Subtotal**	133.360	90,00
bonafide	-	14.816	10,00
Total		148.176	100,00

Fonte: Elaborado pelo autor (2026).

3.3 Pré-Processamento

Todos os áudios passaram pela mesma cadeia de pré-processamento, implementada numa única função do código e chamada tanto pela extração de características quanto pela geração de espectrogramas. A cadeia teve quatro etapas, nesta ordem: leitura do arquivo como sinal monofônico a 16.000 Hz; normalização de amplitude pelo pico; remoção de silêncio por detecção de atividade de voz (VAD, do inglês *Voice Activity Detection*); e padronização da duração em 4,0 s, por corte do excesso ou por preenchimento com zeros (*zero-padding*) no fim do sinal. Os arquivos do ASVspoof já estão a 16.000 Hz, e a reamostragem atuaria apenas sobre um arquivo com outra taxa. A saída de cada áudio foi um vetor de 64.000 amostras, acompanhado de dois valores de diagnóstico: a parcela do áudio original retida pelo VAD, chamada daqui em diante de proporção de fala, e o número de amostras de áudio real anteriores ao *padding*.

A normalização de pico divide o sinal pelo seu maior valor absoluto, levando-o à faixa [−1, 1]. Seu propósito foi remover o ganho de gravação como variável espúria, já que, sem ela, um modelo poderia aprender o volume em vez dos artefatos de síntese. O pico foi preferido a normalizações por energia (RMS, do inglês *Root Mean Square*) por ser simples e determinístico e por não alterar a forma de onda, apenas a escala. A normalização precedeu o VAD porque o detector utilizado, da biblioteca webrtcvad, opera sobre amostras inteiras de 16 bits: com o sinal normalizado, a conversão para inteiro aproveita toda a faixa dinâmica, e as decisões de fala e silêncio ficam mais confiáveis em gravações de baixo volume.

O VAD usou agressividade 2, numa escala de 0 (mais conservador) a 3 (mais agressivo), sobre quadros de 30 ms. A conversão para inteiros serviu apenas para as decisões do detector: o áudio sem silêncio foi remontado a partir do sinal original em ponto flutuante, preservando a precisão para as etapas seguintes. Se o detector descartasse todos os quadros, o áudio original era mantido e a proporção de fala registrada como zero, o que sinaliza o caso sem interromper o *pipeline*.

O VAD precedeu a padronização da duração para que o silêncio natural do áudio fosse removido antes de o sinal ser cortado ou completado até o tamanho-alvo. Na ordem inversa, o

clipe voltaria a ter duração variável após o VAD, e o *padding* ficaria fragmentado no meio do sinal. A reinserção de zeros depois da remoção do silêncio não é contraditória, porque o valor do *padding* é determinístico (zero) e ele serve apenas para igualar o formato do vetor, embora a quantidade de *padding* varie de um áudio para outro, conforme a duração de fala válida que sobra depois do VAD. Ainda assim, para que ele não fosse tomado como informação, foi excluído da agregação nos dois ramos (Seções 3.4 e 3.5). A duração de 4,0 s foi fixada por decisão de protocolo e não foi tratada como eixo experimental (ver Seção 5.10).

Três grandezas de diagnóstico foram gravadas junto às características, mas mantidas fora da matriz de entrada dos modelos: a proporção de fala, o número de *frames* válidos e o número total de *frames* (igual a 251 em todos os áudios). As três medem quantidade de fala e de silêncio, sem relação com o timbre, e o VAD retém proporções muito diferentes nas duas classes. No universo de 148.176 áudios, a proporção de fala média foi de 62,94% no *bonafide* e de 84,65% no *spoof*, uma diferença de 21,71 pontos percentuais; os áudios *bonafide* tinham, portanto, mais silêncio removido pelo VAD. Incluir essas grandezas na entrada daria aos modelos um atalho pela duração da fala, sem relação com os artefatos de síntese. A exclusão foi feita num ponto único do código, a função que seleciona as colunas de entrada dos modelos.

A diferença na proporção de fala entre as classes tem a mesma direção da relatada por Müller *et al.* (2021), cuja análise principal foi feita no ASVspoof 2019 LA: os áudios *bonafide* têm, em média, silêncio inicial e final mais longo que os *spoof*. Nesse conjunto, um modelo treinado apenas com a duração do silêncio inicial chegou a EER de 15,12% na partição de avaliação (Tabela 1 do artigo), e o RawNet2, um detector estabelecido, passou de EER de 3,61% para 15,50% quando o silêncio foi removido (Seção 5.3 do artigo). Os autores sugerem que detectores anteriores podem ter aprendido a distinguir fala real de fala sintética apoiando-se, sem intenção, nessa diferença de silêncio em vez de artefatos de síntese. Na mesma Tabela 1, o modelo baseado no silêncio inicial teve EER de 19,93% no ASVspoof 2021 LA e de 17,42% na tarefa *DeepFake* (DF) da mesma edição, números da fase *progress* copiados da plataforma do desafio; a conclusão do artigo apresenta a extensão do fenômeno ao ASVspoof 2021 como resultado preliminar. O achado fornece a justificativa de literatura para as duas escolhas deste trabalho: remover o silêncio no pré-processamento e manter a proporção de fala fora das entradas.

Neste trabalho, o atalho foi medido num diagnóstico anterior à delimitação do universo, numa partição que ainda reunia as fases *eval* e *progress* (sem a *hidden*). Uma árvore de decisão rasa, treinada apenas com a proporção de fala, alcançou F1-macro de 0,6503 e EER de 0,1933 na validação daquela partição. Os valores não se comparam diretamente aos do Capítulo 5,

obtidos em outra partição, mas mostram que essa variável, sozinha, já separava as classes muito acima do acaso.

3.4 Ramo Clássico: Extração de Características

No ramo clássico, cada áudio pré-processado foi transformado num vetor de 44 características acústicas, extraídas com a biblioteca librosa (McFee *et al.*, 2015): os 20 primeiros coeficientes cepstrais de frequência Mel (MFCC), a taxa de cruzamento por zero (ZCR) e o centroide espectral, as mesmas características discutidas nas Seções 2.3.1 e 2.3.2. O MFCC (Davis; Mermelstein, 1980) descreve o envelope espectral: os primeiros coeficientes representam aspectos do envelope espectral associados às características do trato vocal, e os mais altos tendem a representar variações espectrais mais finas, nas quais artefatos de síntese podem aparecer. A ZCR responde a ruído e a fricativas de alta frequência, e o centroide indica o "centro de massa" do espectro. O Quadro 2 lista as características e seus parâmetros.

As três séries foram calculadas sobre janelas de 400 amostras (25 ms a 16 kHz), a duração curta em que a fala pode ser tratada como aproximadamente estacionária (Seção 2.2), com passo de 256 amostras (16 ms) e transformada rápida de Fourier (FFT) de 512 pontos. A janela de 400 amostras foi passada explicitamente também ao centroide espectral, para o qual a biblioteca usaria, por omissão, uma janela igual ao tamanho da FFT; assim, as três séries tiveram a mesma resolução temporal. Cada série (uma por coeficiente, no caso do MFCC) foi resumida no tempo pela média e pelo desvio-padrão, o que converte um número variável de janelas num vetor de tamanho fixo, com $20 \times 2 + 2 + 2 = 44$ valores. As características foram gravadas sem padronização; a padronização, quando necessária, foi ajustada dentro do *pipeline* de cada modelo, apenas com os dados de treino (Seção 3.7), para evitar vazamento de informação.

Quadro 2 – Características acústicas do ramo clássico e parâmetros de extração

Grupo	**Descritores**	**Parâmetros de extração**	**Qtde.**
MFCC	coeficientes 1 a 20: média e desvio-padrão	20 coeficientes; FFT de 512 pontos; janela de 400 amostras (25 ms); passo de 256 amostras (16 ms); banco de filtros padrão da biblioteca: 128 filtros Mel de 0 a 8.000 Hz, escala Mel de Slaney (não HTK, *Hidden Markov Model Toolkit*), filtros normalizados por área	40
ZCR	média e desvio-padrão	janela de 400 amostras; passo de 256 amostras	2

(continua)

(continuação)

Grupo	Descritores	Parâmetros de extração	Qtde.
Centroide espectral	média e desvio-padrão	FFT de 512 pontos; janela de 400 amostras; passo de 256 amostras	2
Total (entrada dos modelos)		agregação só sobre os *frames* válidos de cada áudio (máscara de *padding*)	**44**
Diagnóstico (fora da entrada)	proporção de fala; número de *frames* válidos; número total de *frames*	gravadas para auditoria e excluídas da entrada dos modelos	3

Fonte: Elaborado pelo autor (2026).
Nota: Escala Mel e normalização dos filtros pelos valores padrão de `librosa.filters.mel` (`htk`, `norm`; librosa 0.11.0).

A agregação foi mascarada: média e desvio-padrão foram calculados apenas sobre os *frames* válidos de cada áudio, isto é, aqueles cujo centro cai dentro do trecho de áudio real, antes do *padding*. Com a convenção de janelas centradas da biblioteca, o *frame i* está centrado na amostra $i \times 256$, e o número de *frames* válidos é o menor inteiro maior ou igual à razão entre o número de amostras válidas e 256, limitado ao total de 251 *frames*. Esse cálculo foi implementado numa única função do código, usada também pela CNN (Seção 3.5). A decisão de mascarar foi tomada antes da extração definitiva das características (Seção 4.2) e apoiou-se em dois argumentos.

O primeiro argumento, e o principal, é de validade de medida. No universo, a mediana foi de 120 *frames* válidos em 251, e 53,07% dos áudios tinham mais da metade do vetor preenchida por *padding*. Uma checagem A/B controlada, sobre 200 áudios agregados com e sem a máscara, mostrou que incluir o *padding* deslocava as características em 30,3% na mediana e em até 47,03%, com alteração em 44 das 44 características. Uma estatística tirada de um vetor que é metade zeros descreve, em boa parte, a formatação do vetor em vez do áudio.

O segundo argumento é que essa distorção era assimétrica entre as classes, com assimetria padronizada de até 0,47 em módulo, no desvio-padrão do 1º coeficiente MFCC. A assimetria não decorre de uma diferença na quantidade de *padding*, pois a fração média de *padding* foi de 47,11% no *bonafide* e de 47,25% no *spoof*, uma diferença de 0,14 ponto percentual; ela vem do conteúdo acústico de cada classe. A proporção de fala, que difere muito entre as classes, mede outra coisa e quase não se correlaciona com o número de *frames* válidos ($r = 0{,}145$).

A definição de *frame* válido pelo centro implica uma aproximação de ±1 *frame* na fronteira entre áudio e *padding*. Como a janela (400 amostras) é maior que o passo (256), janelas vizinhas se sobrepõem: o primeiro *frame* excluído ainda alcança até 200 amostras de áudio real, e o último *frame* incluído pode conter até 200 amostras de zeros. A aproximação foi mantida

deliberadamente, porque a mesma definição, calculada pela mesma função, foi usada nos dois ramos. A máscara da CNN marca exatamente as mesmas posições que a agregação do ramo clássico (Seção 3.5), e a paridade entre os ramos, que é o que a comparação exige, fica preservada.

O banco de filtros Mel (*filterbank*) interno do MFCC foi o padrão da biblioteca, com 128 filtros sobre a FFT de 512 pontos. O comportamento numérico dessa combinação, que motivou uma escolha diferente no ramo profundo (Seção 3.5), é discutido na Seção 5.10.

3.5 Ramo Profundo: Espectrograma Log-Mel e Arquitetura da CNN

No ramo profundo, o mesmo sinal pré-processado foi convertido num espectrograma Log-Mel de formato fixo; o gerador de espectrogramas chama a mesma função de pré-processamento, sem reimplementar o VAD nem o *padding*. A definição do espectrograma e do protocolo da CNN foi fechada em sete decisões, registradas com a evidência que motivou cada uma e tomadas antes da geração dos espectrogramas. O Quadro 3 reúne os valores adotados, incluindo os valores padrão da biblioteca que foram registrados explicitamente, para que uma atualização não os alterasse sem aviso.

Quadro 3 – Definição do espectrograma Log-Mel e do protocolo da CNN

Aspecto	**Valor adotado**
formato da entrada	128 faixas Mel × 251 *frames*, sem interpolação
tratamento do *padding*	espectrograma completo, com agregação mascarada dentro da rede
normalização	média e desvio por faixa Mel, só do treino de cada estágio e só sobre *frames* válidos
taxa de amostragem	16.000 Hz
tamanho da FFT	1.024 pontos (513 *bins*)
janela de análise	400 amostras (25 ms)
passo entre janelas	256 amostras (16 ms)
filtros Mel e faixa de frequência	128 filtros, de 0 a 8.000 Hz
tipo de espectrograma e janelas	espectrograma de potência (expoente 2,0); janelas centradas
escala de amplitude	decibéis, com referência 1,0 e piso de 80 dB abaixo do máximo de cada exemplo
parada antecipada e treino final	27.000 / 3.000 para a parada; retreinamento nos 30.000
desbalanceamento	entropia cruzada com 2 saídas, peso 9:1 a favor do *bonafide*
valores padrão da biblioteca registrados	janela de Hann; bordas completadas com zeros; escala Mel de Slaney (não HTK); filtros normalizados por área; piso de potência de 10^{-10} antes do logaritmo

Fonte: Elaborado pelo autor (2026).
Nota: Valores padrão da biblioteca registrados na última linha: `window` = `hann`, `pad_mode` = `constant`, `htk` = `False`, `norm` = `slaney`, `amin` = `1e-10`.

A janela de análise foi mantida em 400 amostras (25 ms) e o passo em 256 amostras, os mesmos do ramo clássico, com 128 filtros Mel entre 0 e 8.000 Hz. O tamanho da FFT, porém, foi de 1.024 pontos, contra 512 no ramo clássico. Mantidos os mesmos 25 ms de análise, a FFT de 1.024 pontos aumenta a densidade de amostragem da DFT (transformada discreta de Fourier) por *zero-padding* (513 *bins* de frequência em vez de 257), o que permite representar o banco de 128 filtros Mel de maneira numericamente mais adequada. Nenhuma informação nova é criada, porque a resolução física continua determinada pela janela. Com 257 *bins*, 61 dos 128 filtros leriam no máximo dois *bins* e 12 pares de filtros compartilhariam o mesmo *bin* de pico, um regime degenerado sobre o qual a biblioteca não emite aviso; com 513 *bins*, resta um único filtro com até dois *bins* (o mais baixo, estreito por construção da escala Mel junto à frequência mínima de 0 Hz) e nenhum pico compartilhado. O número de *frames* não depende do tamanho da FFT: 251 = 1 + ⌊64.000 / 256⌋ nos dois ramos.

O limite superior de 8.000 Hz, a frequência de Nyquist para 16 kHz, foi mantido em vez de se limitar a banda a 4 kHz, porque 42,1% do universo é de banda larga e tem sinal real acima dessa frequência (Seção 3.2). Nos áudios de banda estreita, as faixas Mel centradas acima de cerca de 4 kHz, 28 das 128, tendem a ficar num platô constante, efeito do *codec*, e não do parâmetro.

O espectrograma de potência (expoente 2,0) foi convertido para decibéis com referência 1,0 e piso de 80 dB, os mesmos valores que a função de MFCC da biblioteca aplica internamente. A referência de 1,0 evitou introduzir uma segunda normalização, relativa ao máximo do próprio espectrograma, e manteve paridade com a transformação que o MFCC já aplica; o ganho das gravações já havia sido removido antes, pela normalização de pico anterior ao VAD (Seção 3.3). A alternativa de tomar como referência o máximo de cada espectrograma seria, ela própria, uma normalização por exemplo (invariante ao ganho, como verificado na auditoria das decisões), o que contradiria a normalização global por estatísticas do treino adotada neste trabalho. Já o piso de 80 dB é relativo ao máximo de cada exemplo: ele fica 80 dB abaixo do máximo daquele espectrograma, de modo que a região de *padding* se torna um platô constante dentro de cada exemplo, com valor que varia entre exemplos (ver Seção 5.10).

Cada espectrograma teve formato de 128 faixas Mel por 251 *frames*, exatamente o número de *frames* produzido pelo *pipeline*; não houve interpolação para uma largura de 256, que inventaria *frames* e desalinharia o eixo temporal em relação ao ramo clássico. Os tensores foram gravados sem normalização, e a normalização foi aplicada no momento da carga: média e desvio-padrão por faixa Mel (dois vetores de 128 valores), calculados somente sobre o treino disponível em cada estágio (27.000 exemplos na fase de parada antecipada, do inglês *early*

stopping, e 30.000 no retreinamento final) e somente sobre os *frames* válidos de cada exemplo. Validação e teste nunca entraram nesse cálculo. As estatísticas de cada estágio foram gravadas como artefato versionado, com o conjunto de origem, o número de exemplos, o *hash* da lista de identificadores e a semente. A normalização por faixa, em vez de um escalar global, iguala a escala entre faixas de energia média muito diferente sem apagar o contraste temporal que a convolução lê.

Como apresentado na Seção 2.5, Müller *et al.* (2022) obtiveram, na detecção de *deepfakes* de áudio, desempenho superior com representações baseadas na transformada Q constante (CQT) ou em espectrogramas logarítmicos do que com representações na escala Mel. A escolha do Log-Mel neste trabalho não se apoiou em desempenho esperado, e sim na paridade com o ramo clássico: o espectrograma da CNN usou a mesma convenção de escala Mel, as mesmas frequências centrais dos 128 filtros e a mesma conversão para decibéis que o MFCC do ramo clássico. Assim, a diferença entre os ramos ficou na forma de representação e de aprendizado, com pré-processamento comum e representações construídas a partir de convenções acústicas alinhadas (FFT de 512 pontos e MFCC com DCT [transformada discreta do cosseno] no ramo clássico; FFT de 1.024 pontos e Log-Mel de 128 × 251 na CNN); adotar a CQT daria à CNN uma representação que o RF e o SVM não tiveram. O uso da CQT fica como trabalho futuro (Seção 6.2).

A CNN adotada foi uma rede pequena, de quatro blocos convolucionais (Goodfellow; Bengio; Courville, 2016), cada um formado por convolução 3 × 3, com preenchimento de uma posição e sem viés, normalização em lote (Ioffe; Szegedy, 2015), ativação ReLU (*Rectified Linear Unit*) e *max pooling* 2 × 2, com 32, 64, 128 e 128 canais, num total de 240.866 parâmetros treináveis (Tabela 3). Cada bloco reduz à metade os eixos de frequência e de tempo. Depois dos blocos, o eixo de frequência foi agregado por média integral, já que as 128 faixas Mel são válidas em todo exemplo, e o eixo do tempo por agregação global mascarada (*masked global pooling*): soma das ativações apenas nas posições válidas, dividida pelo número de posições válidas daquele exemplo. O vetor resultante, de 128 valores, passou por uma camada de *dropout* (com taxa 0,0 na configuração final, Seção 3.7) e por uma camada densa com duas saídas. A camada de achatamento (*Flatten*) foi descartada: além de inflar o número de parâmetros, ela amarraria a rede às posições do *padding*, permitindo-lhe aprender onde o áudio termina, justamente a variável espúria que o ramo clássico havia excluído.

Tabela 3 – Arquitetura da CNN adotada, por etapa

Etapa	Operação	Saída (canais × Mel × tempo)	Parâmetros
Entrada	Log-Mel normalizado + máscara de 251 posições	1 × 128 × 251	0
1º bloco	conv. 3 × 3 (32 canais) → normalização em lote → ReLU → *max pooling* 2 × 2	32 × 64 × 125	352
2º bloco	conv. 3 × 3 (64 canais) → normalização em lote → ReLU → *max pooling* 2 × 2	64 × 32 × 62	18.560
3º bloco	conv. 3 × 3 (128 canais) → normalização em lote → ReLU → *max pooling* 2 × 2	128 × 16 × 31	73.984
4º bloco	conv. 3 × 3 (128 canais) → normalização em lote → ReLU → *max pooling* 2 × 2	128 × 8 × 15	147.712
Agregação	média na frequência + agregação global mascarada no tempo	128	0
Dropout	taxa 0,0 (configuração final)	128	0
Saída	camada densa 128 → 2 (*logits*); *softmax* → P(*spoof*)	2	258
Total			**240.866**

Fonte: Elaborado pelo autor (2026).

A máscara temporal nasceu do mesmo número de *frames* válidos do ramo clássico, calculado pela mesma função, e foi reduzida, a cada bloco, pela mesma operação que reduz o eixo do tempo do mapa de ativação: um *max pooling* unidimensional com o mesmo núcleo e o mesmo passo. O comprimento do eixo temporal passou, assim, de 251 para 125, 62, 31 e 15 posições, e os comprimentos da máscara e do mapa coincidem por construção, sem depender de uma fórmula escrita à parte; uma asserção dentro da passagem direta da rede cobra essa igualdade. A redução segue uma semântica de teto: uma posição reduzida é válida se qualquer das posições originais que ela agrega era válida, coerente com o arredondamento para cima na contagem de *frames* válidos. Ao final, cada posição agrega 16 *frames* originais. A máscara atua apenas no tempo, porque só existe *padding* no tempo, e a invariância da saída ao conteúdo do *padding* foi verificada antes do treino (Seção 4.4).

A máscara protege a agregação temporal, mas não as demais operações da rede. A normalização em lote calcula média e variância sobre o mapa de ativação inteiro, *padding* incluído, e, pela semântica de teto, uma posição de fronteira formada em sua maior parte por *padding* entra na agregação com o mesmo peso das demais. As duas convenções foram registradas junto com a definição da CNN, e seus efeitos possíveis sobre os resultados são discutidos na Seção 5.10.

O desbalanceamento de 9,0:1 entre as classes, o mesmo no treino, na subamostra, na validação e no teste, foi tratado na função de perda: entropia cruzada com duas saídas e peso de cada classe igual ao total de exemplos dividido pelo produto entre o número de classes e o

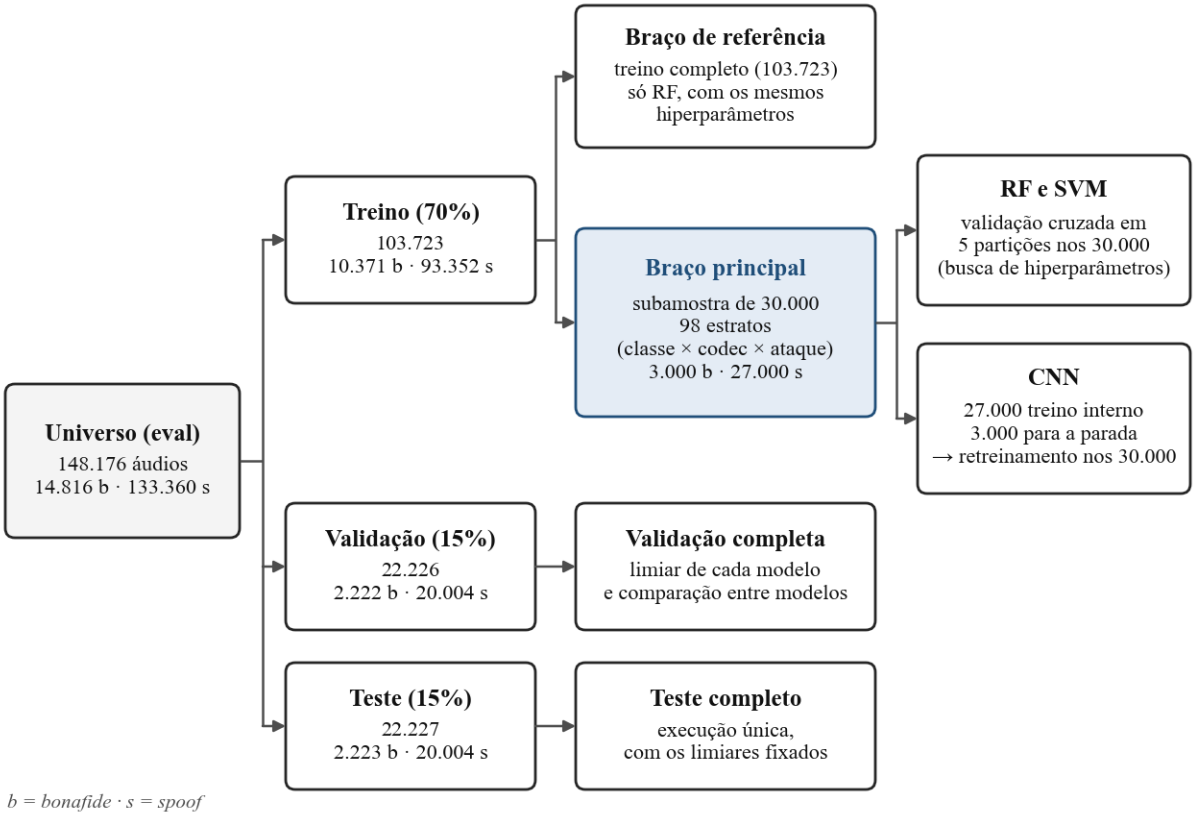
número de exemplos da classe, a mesma lógica da ponderação balanceada do scikit-learn. Os pesos foram derivados das contagens reais de cada conjunto de treino (2.700 *bonafide* e 24.300 *spoof* nos 27.000 da fase de seleção; 3.000 e 27.000 nos 30.000 do retreinamento), o que resultou em peso 5,0 para o *bonafide* e 0,5556 para o *spoof*, razão de 9:1. Não houve sobreamostragem nem subamostragem por classe. A formulação evita uma armadilha: com a convenção do projeto (*bonafide* = 0, *spoof* = 1), a alternativa aparentemente natural, uma perda binária de saída única com peso 9 para a classe positiva, daria o peso maior ao *spoof*, que é a classe majoritária, e inverteria a intenção.

O *score* da CNN foi a probabilidade de *spoof* dada pela *softmax* das duas saídas. O planejamento inicial do trabalho propunha tratar o desbalanceamento com pesos de classe (*class weights*) na função de perda dos classificadores. No executado, a ponderação na função de perda é literal apenas para a CNN; o RF e o SVM trataram o desbalanceamento pelo peso de classe dos próprios classificadores (Seção 3.7).

3.6 Partição dos Dados, Braço Duplo e Validação Cruzada

A partição foi feita por áudio, uma única vez, e gravada em disco como arquivo versionado, de modo que os três modelos usaram exatamente os mesmos conjuntos. O universo foi dividido de forma estratificada pela classe, com semente 42, em treino (70%, 103.723 áudios), validação (15%, 22.226) e teste (15%, 22.227), e a razão entre *spoof* e *bonafide* ficou em 9,0:1 nos três conjuntos (Figura 2). A estratificação impede que a proporção entre as classes oscile por acaso entre os subconjuntos, o que, com esse desbalanceamento, comprometeria a comparação entre eles. Foram necessários três conjuntos porque decisões tomadas olhando um conjunto o desgastam como estimativa de generalização. A validação absorveu esse desgaste (comparação de modelos e escolha de limiar), e o teste ficou reservado para a prova final, usada uma única vez.

Figura 2 – Partição dos dados, braços do experimento e ponto em que cada modelo usa o treino



Fonte: Elaborado pelo autor (2026).

Os três modelos do braço principal foram treinados sobre uma subamostra de 30.000 áudios do conjunto de treino. O tamanho foi fixado *a priori*, no desenho do experimento, como um orçamento experimental comum aos três modelos, tendo em vista a escalabilidade do SVM com *kernel* RBF, cujo custo de treinamento cresce entre $O(n^2)$ e $O(n^3)$ com o número de exemplos. Com esse orçamento, a busca de hiperparâmetros com validação cruzada em cinco partições coube no cronograma do trabalho; como a pergunta de pesquisa exige o mesmo ambiente experimental, os três modelos tiveram a mesma amostra final de treinamento no braço principal, não só o SVM. O SVM não foi treinado no conjunto de treino completo de 103.723 áudios, de modo que não se afirma que esse treino seria inviável, apenas que ele não foi executado; sua realização fica como trabalho futuro (Seção 6.2), e o custo da subamostragem em desempenho é discutido na Seção 5.10.

A subamostra foi estratificada por classe, *codec* e ataque em 98 estratos, com alocação proporcional pelo método do maior resto, piso de um exemplo por estrato (não acionado) e semente 42. Ela reproduziu a composição do treino com desvio máximo de 0,0018 ponto percentual por estrato e razão de exatamente 9,0:1 (3.000 *bonafide* e 27.000 *spoof*).

O experimento teve, por isso, dois braços. O braço principal (RF, SVM e CNN treinados na mesma subamostra de 30.000) é o que responde à pergunta de pesquisa. O braço de referência consistiu apenas num RF treinado no conjunto de treino completo (103.723 áudios), com a mesma configuração de hiperparâmetros escolhida no braço principal, sem nova busca. Esse RF não é concorrente do SVM nem da CNN: ele existe apenas para quantificar quanto desempenho a subamostragem custou. Nos dois braços, a validação e o teste permaneceram completos.

A validação cruzada estratificada em cinco partições foi usada apenas dentro do treino, como parte da busca de hiperparâmetros do RF e do SVM, sobre os 30.000 áudios do braço principal (Seção 3.7). A CNN seguiu outro arranjo: a subamostra foi dividida uma única vez, com a mesma estratificação em 98 estratos e semente 42, em 27.000 exemplos de treino interno e 3.000 de parada antecipada. O conjunto interno de 3.000 serviu para interromper o treino e para escolher a configuração; em seguida, a configuração escolhida foi retreinada nos 30.000 completos, pelo número de épocas definido na fase anterior. Esse retreinamento final (*refit*) removeu uma assimetria, pois sem ele a CNN final teria sido treinada em 27.000 exemplos, enquanto RF e SVM usaram 30.000. A validação externa não foi usada para parada antecipada em fase nenhuma; se fosse, a CNN teria usado a validação duas vezes (para decidir quando parar e para escolher o limiar), privilégio que os modelos clássicos não tiveram.

O planejamento inicial do trabalho propunha a validação cruzada estratificada em cinco partições para reduzir a variância associada à escolha de uma única partição. No executado, ela ficou restrita ao treino, como descrito acima. O papel de medir a incerteza da comparação foi cumprido por outros instrumentos: o *bootstrap* pareado sobre a validação e o teste (Seção 3.8), que reamostra a avaliação com o mesmo vetor de índices para todos os modelos, e duas análises de estabilidade feitas na validação, com o RF treinado com cinco sementes e com RF e SVM treinados em quatro subamostras de 30.000 (a oficial e três alternativas, geradas pela mesma função com as sementes 43, 44 e 45). A limitação decorrente de a partição ser aleatória por áudio, sem separar os ataques entre treino e avaliação, é discutida na Seção 5.10.

3.7 Ajuste de Hiperparâmetros

O RF (Breiman, 2001) e o SVM tiveram hiperparâmetros ajustados por busca aleatória, ou *Random Search* (Bergstra; Bengio, 2012), com 30 iterações e validação cruzada estratificada em cinco partições sobre os 30.000 áudios do braço principal, com semente 42, na implementação do scikit-learn (Pedregosa *et al.*, 2011). O critério da busca foi o EER, com a ROC-AUC registrada ao lado. O EER foi preferido ao F1-macro porque independe de limiar:

buscar pelo F1-macro no limiar padrão de 0,50 favoreceria configurações com *scores* centrados nesse valor, em vez das que melhor ordenam os exemplos, enquanto o limiar do protocolo só seria escolhido depois, na validação (Seção 3.8). A Tabela 4 apresenta o espaço de busca e o valor escolhido para cada hiperparâmetro.

No RF, duas dimensões do espaço de busca foram incluídas por decisão de protocolo. A primeira, o número mínimo de amostras por folha entre 5 e 20, impediu folhas puras. Numa folha pura, a probabilidade estimada é 1 para a classe presente, qualquer que seja o peso de classe, e a ponderação de classes fica neutralizada na saída do modelo; esse efeito havia sido diagnosticado no RF de linha de base, que ordenava bem os exemplos mas decidia mal no limiar de 0,50 (Seção 4.3). A segunda foi o peso de classe, com a variante que recalcula os pesos em cada amostra *bootstrap* das árvores comparada à ponderação balanceada simples. As demais dimensões cobriram o número de árvores, a profundidade máxima, o mínimo de amostras para dividir um nó e o número de características consideradas em cada divisão.

O SVM (Cortes; Vapnik, 1995) foi montado como um *pipeline* com padronização (média zero e desvio-padrão unitário) seguida do classificador, de modo que a média e o desvio de cada característica fossem ajustados apenas nos dados de treino de cada partição da validação cruzada, sem vazamento. O RF dispensa essa etapa, por ser invariante a transformações monotônicas de escala. A busca cobriu o parâmetro de regularização C, em escala log-uniforme entre 0,1 e 100; o parâmetro γ do *kernel*, sorteado com probabilidade de 50% como a heurística de escala da biblioteca (*scale*) e, nos demais casos, em escala log-uniforme entre 0,0001 e 0,1; e o peso de classe (balanceado ou nenhum). O *score* do SVM foi o valor da função de decisão, real e centrado em zero; a calibração de probabilidades foi dispensada, o que torna o treino do SVM determinístico.

O *kernel* do SVM foi fixado em RBF, sem busca do tipo de *kernel*, embora o planejamento inicial do trabalho previsse incluí-lo no espaço de busca. A fixação ocorreu na implementação, e o motivo não foi registrado na época; a justificativa a seguir foi redigida depois, a partir de evidência que já constava dos resultados da busca. O RBF é a escolha inicial recomendada para classificação com SVM (Hsu; Chang; Lin, 2003) e, para valores muito baixos de γ, pode aproximar um regime de decisão próximo ao linear sob determinadas condições (Keerthi; Lin, 2003). A busca de γ, em escala logarítmica até 0,0001, incluiu valores nessa região: os sete candidatos com $\gamma \leq 0{,}001$ obtiveram EER médio de validação cruzada entre 0,187 e 0,290, contra 0,153 do melhor candidato. Isso é um indício, não uma comparação de *kernels*, já que os *kernels* polinomial e sigmoide não foram avaliados (ver Seção 5.10).

Na CNN, a busca foi uma grade curta de seis configurações em duas fases. Na primeira, duas arquiteturas foram combinadas com *dropout* de 0 e de 0,3, com taxa de aprendizado de 0,001: a arquitetura A, com quatro blocos e canais 32-64-128-128 (240.866 parâmetros), e a B, com três blocos e canais 16-32-64 (23.538 parâmetros). Na segunda, as duas arquiteturas foram treinadas com taxa de 0,0003, no valor de *dropout* com maior F1-macro médio na primeira fase, regra fixada antes de rodar a grade. Ficaram fixos o otimizador Adam (Kingma; Ba, 2015), o lote de 128 exemplos, o núcleo de 3 × 3, a normalização em lote em todos os blocos e a ausência de agendamento da taxa de aprendizado e de *weight decay*. Todas as configurações partiram do mesmo estado do gerador aleatório, com semente 42.

Cada configuração foi treinada nos 27.000 exemplos do treino interno, com parada antecipada pelo F1-macro no conjunto interno de 3.000, avaliado ao fim de cada época, com paciência de 8 épocas e teto de 60. A paciência longa foi deliberada: com apenas 300 exemplos *bonafide* nos 3.000, parte da oscilação do F1-macro entre épocas é variância de estimativa, e uma paciência curta subestimaria a melhor época. Venceu a configuração de maior F1-macro nos 3.000, na melhor época de cada uma; diferenças menores que 0,005, valor fixado antes de rodar a grade, não seriam tomadas como decisivas. Na CNN, a seleção usou o F1-macro, diferentemente do RF e do SVM, que usaram o EER, porque ali se escolhiam ao mesmo tempo a configuração e a época de parada, e a parada precisa de um número que reflita a decisão binária do modelo. A assimetria é intencional e foi registrada junto com a definição da CNN.

O planejamento inicial do trabalho propunha busca aleatória também na CNN, sobre a taxa de aprendizado, o número de filtros e o *dropout*. Os três eixos foram explorados, mas numa grade curta, por causa do custo de cada treino: nesta CNN, um treino levou de 12,0 a 17,8 minutos na unidade de processamento gráfico (GPU), contra segundos de um ajuste do RF. Diante disso, optou-se por reduzir o espaço de busca, diretriz adotada no trabalho para os casos em que o ajuste de hiperparâmetros se mostrasse caro. A grade completa consumiu 93,2 minutos. O agendamento da taxa de aprendizado, o *weight decay*, o tamanho do núcleo, a presença da normalização em lote e o tamanho do lote ficaram como espaço não explorado.

A configuração vencedora foi retreinada nos 30.000 exemplos da subamostra por um número fixo de épocas, igual à melhor época da fase de seleção (37), sem parada antecipada e sem acesso à validação externa, com as estatísticas de normalização e os pesos da função de perda recalculados sobre os 30.000. A opção de manter o número de épocas, em vez do número de atualizações de gradiente, é discutida na Seção 5.10.

Tabela 4 – Espaço de busca e valor escolhido, por modelo

Modelo	Hiperparâmetro	Espaço explorado	Escolhido
RF	número de árvores	100; 200; 300; 500	300
RF	profundidade máxima	sem limite; 10; 20; 30	30
RF	mínimo de amostras para dividir	2; 5; 10	10
RF	mínimo de amostras por folha	inteiros de 5 a 20 (faixa do protocolo)	5
RF	características por divisão	raiz quadrada do total; $\log_2$ do total; 30% do total	raiz quadrada do total
RF	peso de classe	balanceado; balanceado por amostra *bootstrap* (faixa do protocolo)	balanceado
SVM	*kernel*	RBF, fixo (sem busca)	RBF
SVM	C	log-uniforme em [0,1; 100]	24,53
SVM	γ	heurística *scale* (50%) ou log-uniforme em [0,0001; 0,1]	*scale*
SVM	peso de classe	balanceado; nenhum	balanceado
CNN	arquitetura	A: 4 blocos, 32-64-128-128; B: 3 blocos, 16-32-64	A
CNN	*dropout*	0; 0,3	0
CNN	taxa de aprendizado	0,001; 0,0003	0,001
CNN	épocas do retreinamento	parada por paciência de 8, teto de 60	37

Fonte: Elaborado pelo autor (2026).
Nota: RF e SVM: busca aleatória com 30 iterações, validação cruzada estratificada em 5 partições sobre os 30.000 do braço principal, critério EER. CNN: grade de 6 configurações em duas fases, critério F1-macro no conjunto interno de 3.000; fixos: Adam, lote de 128, núcleo 3 × 3.

3.8 Protocolo de Avaliação

Todo modelo produziu um *score* contínuo, no qual valores maiores indicam mais evidência de *spoof*: a probabilidade de *spoof* estimada pelo RF, calculada com um único processo para que o resultado fosse reprodutível bit a bit; o valor da função de decisão do SVM, real e centrado em zero; e a probabilidade de *spoof* dada pela *softmax* da CNN. A decisão binária seguiu uma regra única para os três modelos (*spoof* se *score* ≥ limiar), implementada numa única função. A função de avaliação recebe o limiar e deriva internamente as decisões, de modo que nenhum *pipeline* possa aplicar, por acidente, outra regra, como a escolha da classe de maior probabilidade feita pelo método de predição padrão da biblioteca, que diverge da regra adotada quando o *score* coincide exatamente com o limiar.

O limiar de cada modelo foi escolhido somente no conjunto de validação, como o valor que maximiza o F1-macro. Os candidatos foram os valores distintos dos próprios *scores*, em vez de uma grade fixa: sob a regra ≥, qualquer limiar entre dois *scores* consecutivos produz exatamente a mesma partição, de modo que os valores observados formam o conjunto de

candidatos completo e mínimo, e o procedimento não depende da escala do *score* (o limiar do SVM, por exemplo, é negativo). Quando mais de um candidato atingia o máximo, adotava-se o menor. Escolhido na validação, o limiar foi apenas aplicado nos demais conjuntos. A seleção de limiar tornou-se regra do protocolo depois do diagnóstico do RF de linha de base (Seção 4.3): com peso de classe e limiar padrão de 0,50, esse modelo ordenava bem os exemplos, mas decidia mal, porque a distribuição dos seus *scores* estava deslocada para o lado *spoof*. Sem uma regra comum, a comparação favoreceria o modelo com os *scores* mais bem centrados.

As métricas dependentes de limiar e o EER foram calculados pela mesma função para os três modelos: acurácia; precisão, revocação (*recall*) e F1 de cada classe; F1-macro, a média não ponderada dos F1 das duas classes; e a taxa de erro igual (EER), com o *spoof* como classe positiva. A ROC-AUC foi registrada ao lado. O EER foi tomado na curva ROC, no vértice em que a taxa de falsos negativos (FNR) e a de falsos positivos (FPR) estão mais próximas, sem interpolação, como a média entre as duas taxas nesse ponto. Por independer de limiar, o EER complementa o F1-macro e é a métrica padrão da literatura do ASVspoof; os valores deste trabalho, porém, não se comparam diretamente aos do desafio, por diferença de protocolo (Seções 5.9 e 5.10). Em relação ao planejamento inicial do trabalho, que listava acurácia, precisão, *recall* e F1, o executado acrescentou o EER, a ROC-AUC e o tempo de inferência (Seção 3.9) e adotou o F1-macro como critério de decisão.

O F1-macro foi preferido à acurácia como critério por causa do desbalanceamento. No teste, um classificador trivial que respondesse sempre *spoof* acertaria 90,00% dos áudios (20.004 *spoof* em 22.227), com *recall* de *bonafide* nulo e F1-macro de 0,4737, valores calculados com a mesma função de avaliação. O RF do braço principal, com o limiar escolhido pelo protocolo, teve acurácia de 0,8940, abaixo desse classificador trivial, e F1-macro de 0,7210. O limiar que maximiza o F1-macro troca alguns acertos na classe majoritária por *recall* na minoritária, ganho que a acurácia não registra. A acurácia foi reportada, mas não decidiu nenhuma comparação.

O conjunto de teste foi usado uma única vez, com os limiares lidos dos registros de validação de cada modelo e sem nenhum treino nessa etapa. A regra que protege o teste está no código, além do texto: a função que carrega os modelos ajustados recusa um limiar cujo registro não indique que foi selecionado na validação, e o *script* de avaliação do teste recusa uma segunda execução. Os *scores* do teste foram gravados, e toda análise posterior leu esse arquivo, sem pontuar o teste de novo.

A comparação entre modelos foi feita por *bootstrap* pareado (Efron; Tibshirani, 1993), na validação e no teste. Como os três modelos são avaliados exatamente nas mesmas amostras,

seus erros são correlacionados, e comparar intervalos de confiança individuais superestimaria o ruído da comparação. Em cada uma das 1.000 reamostragens, com semente 42, um único vetor de índices foi sorteado e aplicado aos *scores* de todos os modelos, com os limiares fixos no valor do protocolo; guardou-se a diferença entre os modelos em F1-macro e em EER, e o intervalo de confiança de 95% (IC95) foi dado pelos percentis 2,5 e 97,5 dessas diferenças. Um intervalo que não contém zero indica diferença distinguível neste protocolo; um intervalo que contém zero indica apenas que não foi possível demonstrar diferença distinguível, o que não equivale a demonstrar que os modelos são equivalentes. O *bootstrap* reamostra a avaliação, de modo que a variabilidade do treino não entra nesse intervalo.

Com esse instrumento, a palavra "competitivo" da pergunta de pesquisa recebeu uma definição operacional. Em desempenho preditivo (F1-macro e EER), considerou-se que um modelo clássico só poderia ser tido como competitivo frente à CNN se o IC95 do *bootstrap* pareado da diferença entre os dois contivesse zero em F1-macro ou em EER, isto é, se não fosse possível demonstrar diferença distinguível neste protocolo em pelo menos uma das duas métricas. Essa condição é necessária, mas não suficiente: um IC95 que contém zero indica apenas que a diferença não foi demonstrada, e não prova equivalência, que exigiria uma margem de equivalência definida previamente, o que este trabalho não fez. Essa escolha é permissiva com os modelos clássicos, porque basta que o IC95 contenha zero em uma das métricas para que a condição necessária seja satisfeita; em contrapartida, para concluir que um modelo clássico não é competitivo, a diferença precisa ser distinguível nas duas, de modo que essa conclusão não depende da métrica escolhida. Em custo computacional, não se adotou um limiar único: a comparação foi feita por regime de uso, separando a latência de um áudio isolado, a vazão em lote e a execução com e sem GPU (Seção 3.9).

3.9 Protocolo de Medição do Tempo de Inferência

O custo computacional foi medido como tempo de inferência ponta a ponta por áudio, com lote unitário, que corresponde ao cenário em que um áudio chega e precisa ser classificado. O tempo incluiu todas as etapas do caminho real de decisão: leitura e normalização do áudio, VAD e *padding*, geração da representação (as 44 características ou o espectrograma Log-Mel) e predição. A medição usou as mesmas funções do *pipeline* de produção, sobre uma amostra de 200 áudios da validação, sorteada com semente 42; cada tempo foi a mediana de 10 repetições, depois de 3 execuções de aquecimento descartadas, reportando-se também o mínimo e o máximo. A base comum de cada ramo (leitura, VAD e representação) foi medida uma vez e somada à predição de cada modelo.

Como RF e SVM rodam na unidade central de processamento (CPU) e a CNN foi treinada em GPU, o protocolo reportou quatro arranjos: RF em CPU, SVM em CPU, CNN em GPU e CNN em CPU. Medir tudo em CPU penalizaria artificialmente a CNN, e medir cada modelo apenas no seu *hardware* natural compararia máquinas em vez de modelos. O par CNN-GPU e CNN-CPU permite examinar se a vantagem dos modelos clássicos está em serem mais rápidos em igualdade de *hardware* ou em não exigirem GPU. Os modelos clássicos rodaram a inferência com um único processo, e a CNN em CPU, com uma única *thread*. Os quatro arranjos foram medidos na mesma execução, porque tempos medidos em momentos diferentes compariam também o estado da máquina.

Na GPU, dois cuidados foram obrigatórios. Como a execução em CUDA (*Compute Unified Device Architecture*) é assíncrona, o processador aguardou explicitamente o término das operações da GPU antes e depois do trecho cronometrado; sem essa sincronização, mede-se o tempo de enfileirar as operações, não o de executá-las. Além disso, o dispositivo foi aquecido por 2 s antes do cronômetro, porque as 3 execuções descartadas do protocolo não bastam para tirar a placa do estado de baixo consumo depois de um trecho longo em CPU (Seção 4.5). A inferência da CNN rodou no mesmo regime determinístico do resto do trabalho, com a seleção automática de algoritmos do cuDNN (*CUDA Deep Neural Network library*) desligada, o que torna a medida conservadora.

A vazão em lote foi medida separadamente da latência, porque as duas respondem a cenários de uso diferentes. Nesse caso, mediu-se apenas a predição, a partir da representação já extraída: RF e SVM predisseram a validação inteira (22.226 áudios) numa única chamada; a CNN, em lotes de 128, cobriu a validação inteira em GPU (22.226 áudios) e uma amostra de 1.280 áudios em CPU, reportando-se a taxa por áudio.

Por fim, a variação dos tempos entre execuções foi medida quando os artefatos de inferência da CNN foram regenerados: os tempos de predição da CNN, medidos em duas execuções feitas em momentos diferentes na mesma máquina, variaram entre 4,1% e 13,7% de uma execução para a outra. Por isso, diferenças de tempo dentro dessa faixa, isto é, de até 13,7%, foram tratadas como insuficientes para sustentar uma ordenação estável entre os modelos, sem que isso constitua teste de equivalência. Todas as medições foram feitas num único equipamento (ver Seção 5.10).

3.10 Ambiente Computacional e Reprodutibilidade

O experimento foi executado numa única máquina, com sistema operacional Windows 11, Python 3.10.11, processador AMD (registrado como “AMD64 Family 25 Model 97”) e

GPU NVIDIA GeForce RTX 5060 Ti. Essa GPU, de arquitetura Blackwell, tem capacidade de computação 12.0. A escolha do PyTorch (Paszke *et al.*, 2019) foi orientada por essa GPU e pelo sistema operacional: o TensorFlow não oferece suporte nativo a GPU no Windows depois da versão 2.10 (TensorFlow, 2026), enquanto o PyTorch passou a oferecer suporte à arquitetura Blackwell, com pacotes pré-compilados para CUDA 12.8, a partir da versão 2.7 (PyTorch Team, 2025). Foram usados o PyTorch 2.11.0 com CUDA 12.8 e cuDNN 9.19, o scikit-learn 1.7.2 (Pedregosa *et al.*, 2011), o librosa 0.11.0 (McFee *et al.*, 2015), o NumPy 2.2.6 e o pandas 2.3.3. Os arquivos de registro dos experimentos indicam "Windows 10" no campo de sistema operacional. Isso decorre da forma como o Python 3.10 identifica o sistema: a função `platform.release()` retorna "10" também no Windows 11. A *build* registrada no lote de extração, 10.0.26200, corresponde ao Windows 11.

A leitura dos áudios foi feita diretamente com a biblioteca soundfile, sem passar pela função de carregamento do librosa, para que eventuais exceções do decodificador chegassem intactas ao registro de erros, em vez de mascaradas por um mecanismo alternativo de leitura. A versão do soundfile foi fixada em 0.12.1, porque, neste ambiente experimental, as versões 0.13 e 0.14 para Windows, que embutem uma versão da libsndfile, apresentaram falha determinística no decodificador do formato FLAC (*Free Lossless Audio Codec*) em cerca de 44% dos arquivos FLAC válidos do conjunto. A integridade dos arquivos foi confirmada pelo resumo MD5 (*Message-Digest Algorithm 5*) registrado no cabeçalho STREAMINFO de cada arquivo e por um decodificador independente, conforme registrado no arquivo de dependências do projeto.

A aleatoriedade foi controlada por uma semente única, 42, lida do arquivo de configuração e aplicada ao Python, ao NumPy e, explicitamente, ao parâmetro de semente de cada componente do scikit-learn, sem depender do estado global. Na CNN, fixar a semente não basta, porque operações da GPU somam em ponto flutuante numa ordem que pode variar entre execuções. Foram usadas quatro travas (sementes do PyTorch, algoritmos determinísticos do cuDNN, desativação da seleção automática de algoritmos e configuração do espaço de trabalho do cuBLAS [*CUDA Basic Linear Algebra Subroutines*] antes da primeira chamada CUDA), com o modo determinístico estrito do PyTorch, que interrompe a execução diante de uma operação sem versão determinística. O modo estrito esteve ativo no retreinamento e na inferência da CNN. A predição do RF foi feita com um único processo, porque a soma paralela das árvores não é reprodutível bit a bit.

O formato TF32 (*TensorFloat-32*) da GPU foi desligado antes de qualquer *score* da CNN. Com ele ativo, o *score* passa a depender do tamanho do lote e difere do cálculo em precisão simples numa ordem de grandeza comparável ao espaçamento entre *scores* vizinhos;

como o limiar é escolhido sobre os valores distintos dos *scores* (Seção 3.8), isso poderia deslocá-lo. A decisão foi tomada durante a validação da CNN (Seção 4.5), e o registro do teste confirma o TF32 desligado.

Os artefatos que definem o experimento foram congelados e identificados por *hash* MD5 (Quadro 4). As definições das características e dos espectrogramas foram assinadas em arquivos de metadados, conferidos por guardas que abortam qualquer retomada de extração sob outra definição. A CNN final e o SVM foram versionados no repositório, os RF foram mantidos em cópia externa, e os MD5 de todos os modelos foram registrados num arquivo versionado, assim como as versões exatas das bibliotecas do ambiente. Por fim, uma guarda de reprodução comparou campo a campo os artefatos de resultado regenerados com cópias anteriores, excluídas apenas as medidas de relógio: os oito artefatos do ramo clássico (busca, modelos ajustados, estabilidade, ablação e curva de aprendizado) e os quatro que dependem da inferência da CNN, entre eles o da comparação estatística, reproduziram. Os achados dessas verificações são relatados na Seção 4.5.

Quadro 4 – Artefatos congelados do experimento e respectivos *hashes* MD5

Artefato	**Conteúdo**	**MD5**
matriz de características	148.176 linhas × 50 colunas (44 características + identificação e diagnóstico)	`51b2f439bf6f1e10237acbc620bb92d9`
partição	70/15/15 por áudio	`9143f0c7b83ec2db4aa144ed5deb3402`
subamostra do braço principal	30.000 áudios do treino	`654cb796b738512388b28e15ffb14a9d`
divisão interna da CNN	27.000 / 3.000	`4dd2246cf8b24e4a06517d4a3b2b37b7`
normalização da CNN final	média e desvio por faixa Mel (30.000 exemplos)	`348d1df7f1251ab7670dbee2b180ea14`
CNN final	pesos da rede	`9e91f9ca0ae5da912b4c576d995863fb`
SVM do braço principal	modelo ajustado	`b2bd1f782559e71e61895a535842d9d0`

Fonte: Elaborado pelo autor (2026), a partir de `results/metricas/teste_lacrado.json`, `results/metricas/split_interno_cnn.json` e `docs/execucao/HASHES_MODELOS.txt`; arquivos, na ordem das linhas: `data/features/features.csv`, `data/processed/split.csv`, `data/processed/subamostra_30k.csv`, `data/processed/split_interno_cnn.csv`, `data/espectrogramas/normalizacao_cnn_30k.json`, `models/cnn_final_30k.pt` e `models/svm_tuned_principal.joblib`.

Os *hashes* MD5 do Quadro 4 foram calculados sobre os arquivos na forma em que foram gerados, com a quebra de linha do Windows (CRLF), e o repositório foi configurado para entregar essa mesma forma em qualquer sistema operacional ao ser clonado, de modo que os *hashes* também conferem fora do Windows. A matriz de características, primeira linha do

quadro, não está no repositório, porque os arquivos de dados volumosos do projeto foram mantidos fora do controle de versão; o MD5 permite conferir uma cópia obtida à parte.

4 DESENVOLVIMENTO

Este capítulo relata a execução do protocolo descrito no Capítulo 3, na ordem em que as etapas foram realizadas: o que foi verificado antes de cada operação de custo alto, o que deu errado e como foi detectado, e o que ficou congelado ao fim de cada etapa. Cada etapa terminou com um artefato versionado, que a etapa seguinte leu sem alterar. A Seção 4.1 trata da organização do código; a Seção 4.2, do diagnóstico e da extração definitiva das características; a Seção 4.3, do treinamento do RF e do SVM; a Seção 4.4, da geração dos espectrogramas e do treinamento da CNN; e a Seção 4.5, dos problemas técnicos encontrados e dos controles que protegeram o conjunto de teste. Os resultados comparativos ficam para o Capítulo 5.

4.1 Organização do Código e Princípio da Fonte Única

Os parâmetros centrais do protocolo (a semente 42, as 30 iterações da busca aleatória, as 10 repetições e os 3 descartes da cronometragem, a duração de 4,0 s e os parâmetros do espectrograma) foram lidos de um único arquivo de configuração. Os parâmetros próprios de uma única etapa, como a paciência e a grade da CNN, o número de reamostragens do *bootstrap* e os parâmetros do VAD, ficaram como constantes nomeadas no início do *script* que os usa ou como valores padrão declarados na própria função. Com os parâmetros centrais num só lugar, um parâmetro declarado na configuração e não consumido pelo código aparece como divergência. Foi desse modo que se descobriu que a cronometragem media apenas a predição, embora a configuração declarasse incluir a extração das características, e o custo ponta a ponta passou a ser medido à parte (Seção 3.9).

O código seguiu o princípio da fonte única: cada operação usada por mais de um modelo ou *script* (pré-processamento, contagem de *frames* válidos, lista das características de entrada, regra de decisão, seleção do limiar, métricas, carregamento de cada modelo ajustado com o seu limiar, cronometragem e sementes) foi implementada uma vez e importada pelos demais. A cópia de código é a origem mais comum de divergência silenciosa entre quem grava um resultado e quem o lê, e a pergunta de pesquisa depende de os três modelos serem medidos pela mesma régua; por isso, a validação da CNN importou as funções de limiar, de avaliação e de EER do mesmo módulo que produziu os números do RF e do SVM. Além disso, nenhum artefato gerado recebeu edição à mão, e cada registro de resultado guarda a semente, os hiperparâmetros, as versões das bibliotecas e os *hashes* MD5 dos artefatos congelados de que depende.

4.2 *Pipeline* de Características e Lote Único de Reextração

A primeira etapa examinou o *pipeline* de características (Seções 3.3 e 3.4). Como a proporção de fala retida pelo VAD difere muito entre as classes, suspeitou-se de um atalho pela duração: uma árvore de decisão treinada só com essa grandeza, numa partição anterior à delimitação do universo, obteve F1-macro de 0,6503 e EER de 0,1933 na validação daquela partição. A proporção de fala ficou, por isso, fora das entradas, como coluna de diagnóstico, junto com o número de *frames* válidos e o número total de *frames*.

Restava saber se o atalho estava nas próprias características, então agregadas sobre os 4,0 s inteiros, zeros incluídos. Com as características desse *pipeline*, ainda anterior ao mascaramento, um RF treinado e avaliado com as classes pareadas em dez faixas da proporção de fala (arranjo em que a proporção de fala deixa de separar as classes) obteve EER de 0,1631, e o RF de linha de base, treinado no treino completo, passou de EER de 0,1818 na validação completa para 0,2102 no conjunto pareado. Num piloto com 2.000 áudios, agregados com e sem os *frames* de *padding*, o RF teve F1-macro de 0,5068 com a máscara contra 0,4905 sem ela, resultado tratado como indicativo pelo tamanho da amostra, e a recomendação foi mascarar o *padding* na agregação temporal.

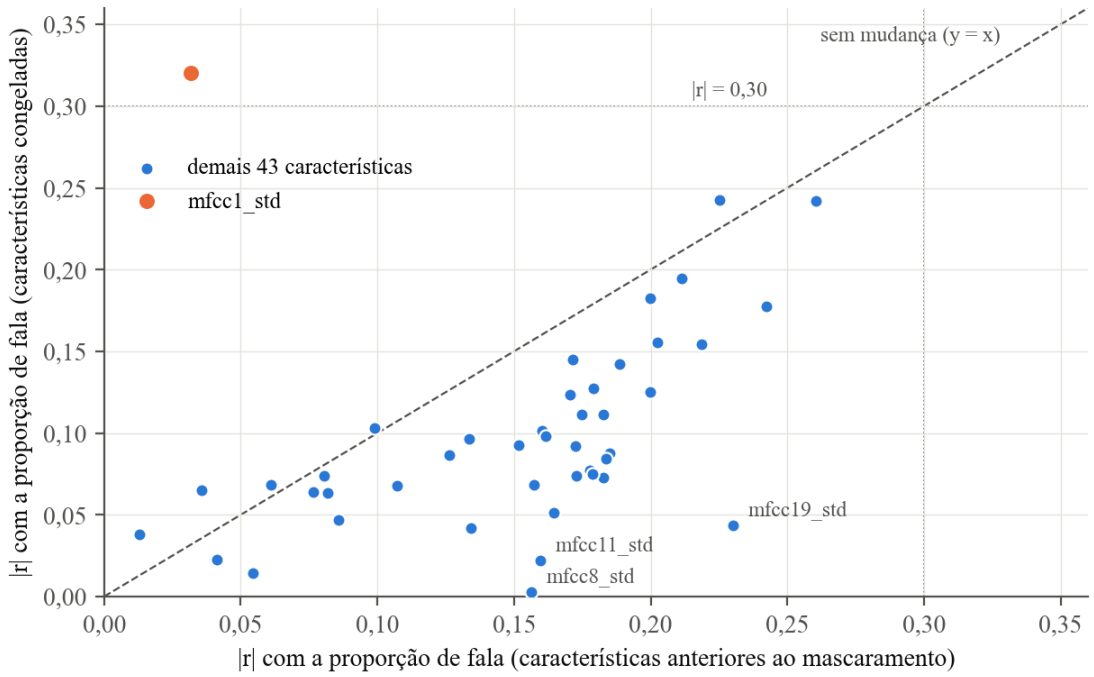
O que precisou de correção foi o mecanismo, não a decisão. A recomendação original supunha que o *bonafide*, por perder mais sinal no VAD, recebia mais *padding*. A checagem sobre 200 áudios desmentiu a inferência (fração média de *padding* de 0,4480 no *bonafide* e de 0,4523 no *spoof*, com proporção de fala de 0,6398 contra 0,8537), e a medição no universo a confirmou (Seção 3.4): a proporção de fala é relativa ao áudio original, enquanto o *padding* depende da duração absoluta que sobra. A recomendação foi mantida pelo argumento de validade de medida (mediana de 120 *frames* válidos em 251, isto é, cerca de metade do vetor em *padding*), e o documento recebeu um adendo com a correção, sem apagar o texto original.

A extração definitiva aplicou, numa única execução, as quatro mudanças decididas (janela do centroide igualada à das demais séries, universo restrito à fase *eval*, *padding* mascarado e colunas de diagnóstico fora das entradas), e só foi disparada depois de uma checagem obrigatória em duas partes. Um teste A/B controlado sobre 200 áudios provou que o mascaramento estava ativo: as 44 características mudaram, com deslocamento relativo mediano de 30,3% e máximo de 47,03%. Em seguida, o código de produção rodou sobre 500 áudios, e o arquivo resultante foi validado (50 colunas, 44 de entrada, só a fase *eval* e 251 *frames* totais em todos os áudios). A guarda contra retomada sob outra definição de característica e a legibilidade dos 148.176 arquivos também foram testadas antes do disparo.

A extração cobriu os 148.176 áudios, com os mesmos identificadores da partição e a definição assinada num arquivo de metadados (mascaramento ativo, 20 coeficientes MFCC, FFT de 512 pontos, passo de 256 e janela de 400 amostras). Partição e subamostra foram preservadas em vez de regeradas, porque a extração mudou os valores das características e não o conjunto de arquivos, e a preservação foi conferida (identificadores idênticos, as 30.000 linhas da subamostra recuperadas e *hashes* MD5 inalterados). As 44 características foram então congeladas, com o arquivo identificado pelo MD5 `51b2f439bf6f1e10237acbc620bb92d9`.

Com as características congeladas, a correlação com a proporção de fala foi recalculada (Figura 3). A média de $|r|$ sobre as 44 características caiu de 0,150 para 0,101, com a queda concentrada nas características de desvio-padrão, cujo valor antes crescia com a quantidade de zeros. Na direção oposta, o desvio-padrão do 1º coeficiente MFCC (`mfcc1_std`) passou de $|r|$ = 0,032 para 0,320 e tornou-se a única característica acima de 0,3, o que exigiu um segundo adendo à recomendação. Contra o número de *frames* válidos, a maior correlação ficou em 0,14. A leitura desse resíduo e a ablação que o testou estão na Seção 5.8.

Figura 3 – Correlação absoluta de cada característica com a proporção de fala, antes e depois do mascaramento do *padding* (fase *eval*)



Fonte: Elaborado pelo autor (2026).
Nota: Cada ponto é uma das 44 características; nos rótulos, o sufixo `_std` indica o desvio-padrão e o número indica o coeficiente MFCC (`mfcc1_std` é o desvio-padrão do 1º coeficiente). Os pontos abaixo da diagonal tracejada (y = x) indicam características cuja correlação absoluta com a proporção de fala diminuiu depois do mascaramento; os pontos acima dela, que aumentou. As linhas pontilhadas marcam $|r|$ = 0,30 em cada eixo.

4.3 Treinamento de *Random Forest* e SVM

O RF de linha de base foi treinado nos 30.000 áudios do braço principal, ainda sobre as características anteriores ao mascaramento, com a decisão pelo método de predição padrão da biblioteca, isto é, pelo limiar implícito de 0,50. Na validação, o resultado expôs um problema de decisão, não de ordenação: F1-macro de 0,4980, *recall* de *bonafide* de apenas 0,0243 e acurácia de 0,9025, contra 0,9000 de um classificador trivial que responde sempre *spoof*. Com as árvores crescendo até folhas puras, o peso de classe era neutralizado na probabilidade estimada, os *scores* tinham granularidade grossa e o limiar ótimo ficava longe de 0,50. Desse diagnóstico vieram as três correções incorporadas antes da busca: o número mínimo de amostras por folha entre 5 e 20, a ponderação balanceada por amostra *bootstrap* como alternativa de peso de classe e a seleção do limiar na validação como regra comum aos três modelos (Seção 3.8).

As buscas seguiram o desenho da Seção 3.7 (Tabela 5). A do RF levou 648,6 s, e dois valores da configuração vencedora caíram na borda do espaço explorado, o mínimo de amostras por folha no limite inferior (5) e o mínimo para dividir um nó no maior valor da lista (10), o que o registro da busca marca como dimensões censuradas (Seção 5.10). No SVM, cujo treino cresce entre $O(n^2)$ e $O(n^3)$ com o número de exemplos, o *script* cronometrou o ajuste de uma partição (5,3 s) e projetou o custo da busca antes de dispará-la: 0,04 h, contra um orçamento de 6,0 h. Um estouro reduziria o número de iterações, nunca a subamostra; como a projeção coube com folga, a busca rodou as 30 iterações previstas, em 392,0 s.

Tabela 5 – Resultado das buscas aleatórias de hiperparâmetros no treino do braço principal

Modelo	Configuração vencedora	EER na validação cruzada (média ± dp)	ROC-AUC na validação cruzada (média ± dp)	Tempo da busca (s)
RF	300 árvores; profundidade máxima 30; raiz quadrada do número de características em cada divisão; mínimo de 5 amostras por folha e de 10 para dividir um nó; peso de classe balanceado	0,2059 ± 0,0117	0,8725 ± 0,0095	648,6
SVM (RBF)	C = 24,53; γ pela heurística de escala da biblioteca; peso de classe balanceado	0,1528 ± 0,0076	0,9224 ± 0,0037	392,0

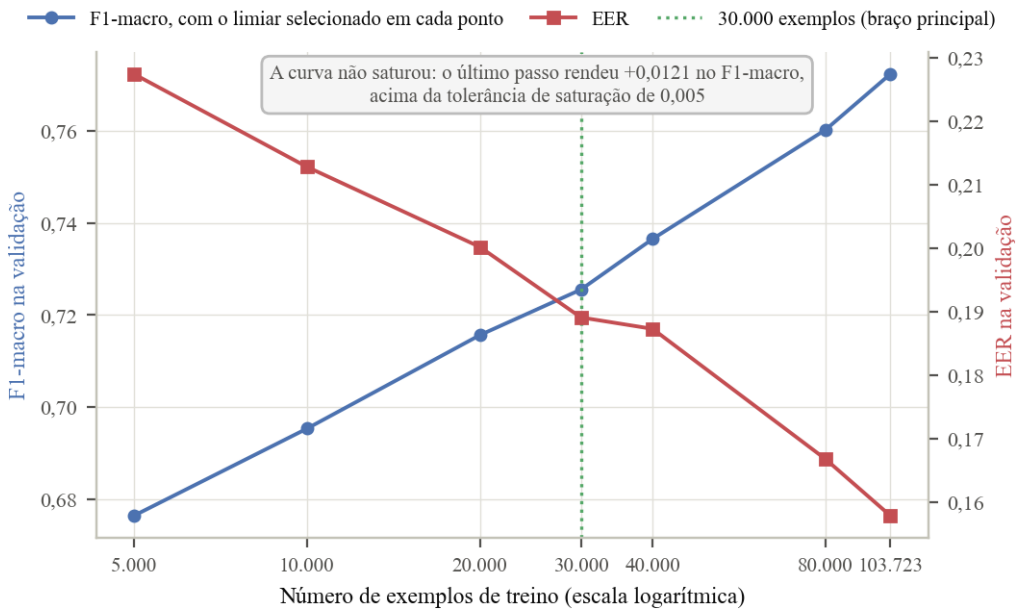
Fonte: Elaborado pelo autor (2026).
Nota: 30 configurações sorteadas em cada busca, com validação cruzada estratificada em cinco partições; pontuação pelo EER, com a ROC-AUC registrada ao lado.

Os modelos finais (o RF nos dois braços, com 30.000 e 103.723 exemplos, e o SVM só no principal) tiveram o limiar selecionado na validação. O do RF principal foi 0,6516, entre

22.207 valores distintos de *score* em 22.226 amostras, sinal de que o mínimo de 5 amostras por folha acabou com a granularidade grossa da linha de base; o do SVM foi −0,0329, entre 22.226 candidatos na escala da função de decisão. O SVM reteve 7.605 vetores de suporte (2.177 *bonafide* e 5.428 *spoof*), 25,4% do treino.

A curva de aprendizado do RF ajustado (Figura 4), com a configuração vencedora e o limiar selecionado na validação em cada ponto, foi de 5.000 a 103.723 exemplos de treino. O F1-macro subiu de 0,6764 para 0,7723, e o último passo (de 80.000 para 103.723 exemplos) ainda rendeu +0,0121, acima da tolerância de 0,005 que define saturação: a curva não saturou no maior tamanho disponível. O último ponto reproduz o braço de referência (F1-macro de 0,7723 e EER de 0,1579), e a extrapolação da curva é discutida na Seção 5.10.

Figura 4 – Curva de aprendizado do RF ajustado na validação, com as características congeladas e o limiar selecionado em cada ponto



Fonte: Elaborado pelo autor (2026).

A estabilidade foi medida em duas fontes de variação (Tabela 6). Com cinco sementes (de 42 a 46), o F1-macro do RF ficou entre 0,7214 e 0,7225, com desvio-padrão de 0,0004, embora o limiar tenha ido de 0,6463 a 0,6681; o SVM, sem calibração de probabilidade, tem treino determinístico. Com três subamostras alternativas de 30.000 (sementes 43 a 45), geradas pela mesma função de estratificação e treinadas com os mesmos hiperparâmetros, o desvio-padrão do F1-macro entre as quatro subamostras foi de 0,0023 no RF e de 0,0017 no SVM; antes, a subamostra oficial foi regerada e conferida por MD5.

Tabela 6 – Dispersão do F1-macro na validação por fonte de variação, com os hiperparâmetros vencedores fixos

Fonte de variação	Modelo	Pontos	F1-macro (mínimo a máximo)	Desvio-padrão
semente do treino (42 a 46)	RF	5	0,7214 a 0,7225	0,0004
semente do treino	SVM	não se aplica	não se aplica	treino determinístico
subamostra de 30.000 (sementes 42 a 45)	RF	4	0,7225 a 0,7272	0,0023
subamostra de 30.000 (sementes 42 a 45)	SVM	4	0,7946 a 0,7987	0,0017

Fonte: Elaborado pelo autor (2026).
Nota: A subamostra de semente 42 é a oficial do braço principal e reproduz os resultados publicados.

A etapa terminou com a comparação pareada entre os dois modelos na validação, antes de qualquer resultado da CNN. Com 1.000 reamostragens *bootstrap* e os limiares fixos, a diferença SVM − RF foi de +0,0762 no F1-macro (IC95 [0,0663; 0,0856]) e de −0,0468 no EER (IC95 [−0,0560; −0,0377]), com o SVM melhor em 100% das reamostragens nas duas métricas. A comparação com a CNN está nas Seções 5.1 e 5.4.

4.4 Geração dos Espectrogramas e Treinamento da CNN

A etapa da CNN começou pelo registro das sete decisões sobre o espectrograma e a rede (Seção 3.5) no arquivo de configuração, antes de qualquer tensor, com os valores padrão da biblioteca que afetam o resultado escritos explicitamente. Uma auditoria que encerra com erro diante de uma invariante violada conferiu a largura de 251 *frames* contra o *pipeline* e as 148.176 linhas do universo, dimensionou o lote (74.453 tensores, 8,91 GiB em precisão simples) e mediu a fração de *padding* por classe (47,11% no *bonafide* e 47,25% no *spoof*), o que desfez a assimetria que a primeira versão do documento de decisões havia deduzido da proporção de fala, o mesmo engano corrigido na Seção 4.2.

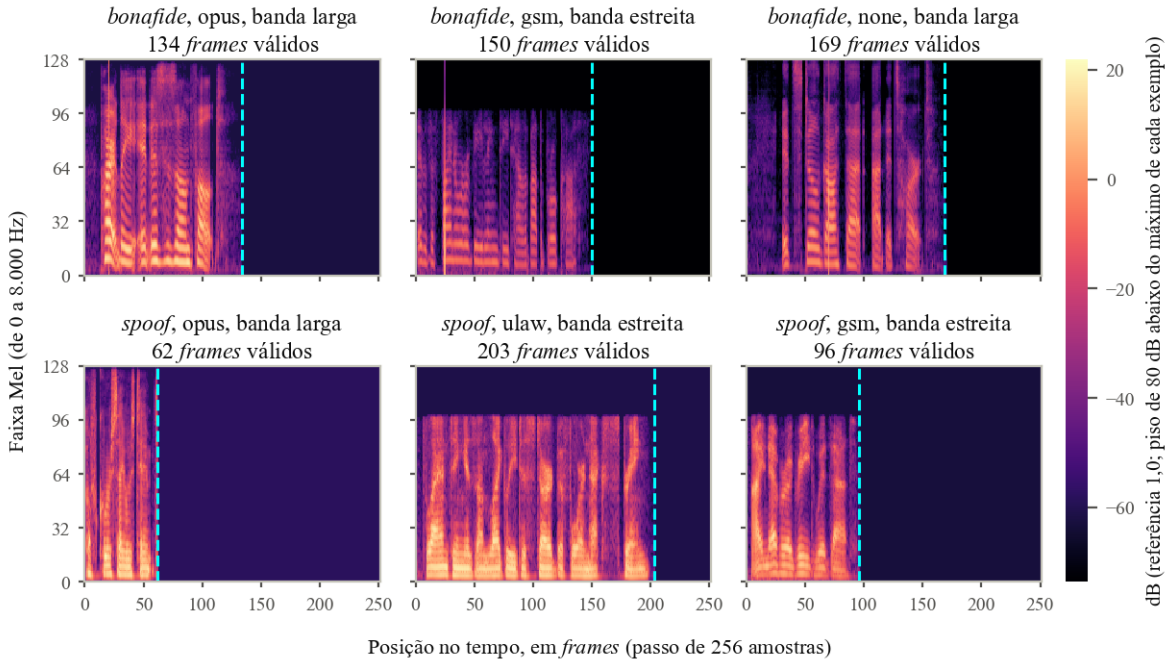
O gerador foi validado num piloto de 201 tensores (81 do treino, 60 da validação e 60 do teste; 20 *bonafide*) por 12 asserções, nenhuma reprovada (Quadro 5). Três expectativas iniciais caíram diante da medição. O máximo em dB foi positivo em 201 de 201 exemplos, porque a potência da transformada de Fourier de tempo curto (STFT) soma 400 amostras janeladas e ultrapassa 1 mesmo com o sinal em [−1, 1]. O primeiro *frame* inválido não foi constante em 120 de 187, porque a janela de 400 amostras, maior que o passo de 256, ainda alcança áudio real. Por fim, o topo constante do espectro apareceu em 65 de 118 exemplos de banda estreita e em apenas 2 de 83 de banda larga, o que o identifica como o teto de 4 kHz dos *codecs* telefônicos (Figura 5).

Quadro 5 – As 12 asserções que validaram o gerador de espectrogramas no piloto

Item	O que se asseriu	Medido no piloto
1	formato de 128 × 251 em todos os tensores	201 de 201 conformes
2	largura = 1 + ⌊64.000 / 256⌋ = 251, igual no arquivo de configuração, nos metadados e no *pipeline*	251 nas quatro fontes
3	nenhum valor NaN (*Not a Number*) ou infinito	0 NaN, 0 infinitos
4	os 24 parâmetros dos metadados iguais aos do arquivo de configuração, e os *hashes* de entrada iguais aos do disco	nenhuma divergência
5	faixa dinâmica ≤ 80 dB e piso igual ao máximo − 80 dB onde há *padding* (efeito do piso de 80 dB, não do valor mínimo de amplitude)	nenhuma violação; faixa máxima de 80,0 dB
6	ganho de 10× desloca todo o espectrograma em 20,0 dB (o que distingue a referência 1,0 da referência no máximo de cada exemplo)	24 áudios; erro máximo de 0,00023 dB
7	número de *frames* válidos recalculado igual ao da matriz de características	201 de 201 (100%)
8	platô constante no valor máximo − 80 dB a partir de um *frame* após o último válido	fronteira provada; *frame* de transição não constante em 120 de 187
9	o valor do platô varia entre exemplos (o piso é relativo ao máximo de cada um)	187 valores distintos em 187, de −73,78 a −52,78 dB
10	banco Mel com FFT de 1.024 pontos: suporte mínimo ≥ 2 *bins*, nenhum pico duplicado, no máximo um filtro de até 2 *bins*	2 / 0 / 1 (com FFT de 512: 1 / 12 / 61)
11	o número de *frames* independe do tamanho da FFT	251 com 512, 1.024 e 2.048 pontos
12	plausibilidade visual: *bonafide* e *spoof*, banda do *codec* visível	6 painéis; topo no platô em 65 de 118 (banda estreita) e 2 de 83 (larga)

Fonte: Elaborado pelo autor (2026).

Figura 5 – Espectrogramas Log-Mel crus do piloto do gerador, com três exemplos *bonafide* (acima) e três *spoof* (abaixo), de banda larga e estreita; a linha tracejada marca o fim dos *frames* válidos



Fonte: Elaborado pelo autor (2026).
Nota: A escala de cor é a mesma nos seis painéis; acima de cada painel, a classe, o *codec*, a banda e o número de *frames* válidos do exemplo.

O lote só foi disparado com as asserções do piloto aprovadas e o espaço em disco conferido (8,91 GiB previstos, 1.071 GiB livres), e gerou os 74.453 tensores de 128 × 251 sem erro. Suas 9 asserções, sem reprovação, conferiram formas, índices, disjunção entre os conjuntos e identidade com a partição, ausência de NaN e de infinitos em 2.000 linhas por conjunto e paridade do número de *frames* válidos e do rótulo com a matriz de características em 500 linhas por conjunto. Os espectrogramas ficaram então congelados, e os do teste nunca foram lidos por módulo de treino ou de validação (Seção 4.5).

A divisão interna (27.000 e 3.000 exemplos) usou a estratificação em 98 estratos e a função de alocação da subamostra, importadas; a razão *spoof*:*bonafide* ficou em 9,0 nos dois lados, com desvio máximo de 0,0167 ponto percentual por estrato. A normalização por faixa Mel foi calculada só nos 27.000 e só sobre os *frames* válidos (3.585.140 dos 6.777.000 que existiriam sem a máscara), e nenhuma faixa precisou de piso no desvio-padrão, cujo menor valor foi de 8,42 dB. Com os tensores crus em disco, o retreinamento final pôde recalculá-la sem regenerar o lote.

Antes do primeiro treino, a máscara temporal passou por uma checagem própria. O teste previsto (dois exemplos iguais na parte válida e com *padding* diferente, comparados com e sem máscara) não discriminou, com razão entre as duas diferenças de 0,71 na semente 42 e de no

máximo 4,0 nas demais, porque a convolução leva o conteúdo do *padding* às posições válidas de fronteira. Refeita com o alcance medido (no máximo 27 *frames*) e uma zona de guarda de 32, a checagem deu diferença com máscara igual a zero exato e diferença sem máscara positiva em todas as sementes, com o modelo em modo de avaliação. O que a máscara não protege no treino, a normalização em lote, é discutido na Seção 5.10.

A CNN de linha de base (arquitetura de quatro blocos, 240.866 parâmetros, *dropout* de 0,3, taxa de aprendizado de 0,001 e lotes de 128) treinou 30 épocas nos 27.000 exemplos internos. No conjunto de 3.000, o F1-macro chegou a 0,9018, mas a melhor época foi a última: a rede apenas esgotou as épocas, e o critério de parada da etapa seguinte foi definido com essa informação. A perda nesse conjunto oscilou entre 0,20 e 2,92 enquanto F1-macro e EER melhoravam, oscilação atribuída à perda de calibração, não de desempenho, e por isso a parada passou a usar o F1-macro. Esses números vêm da própria subamostra de treino e não se comparam aos do RF e do SVM.

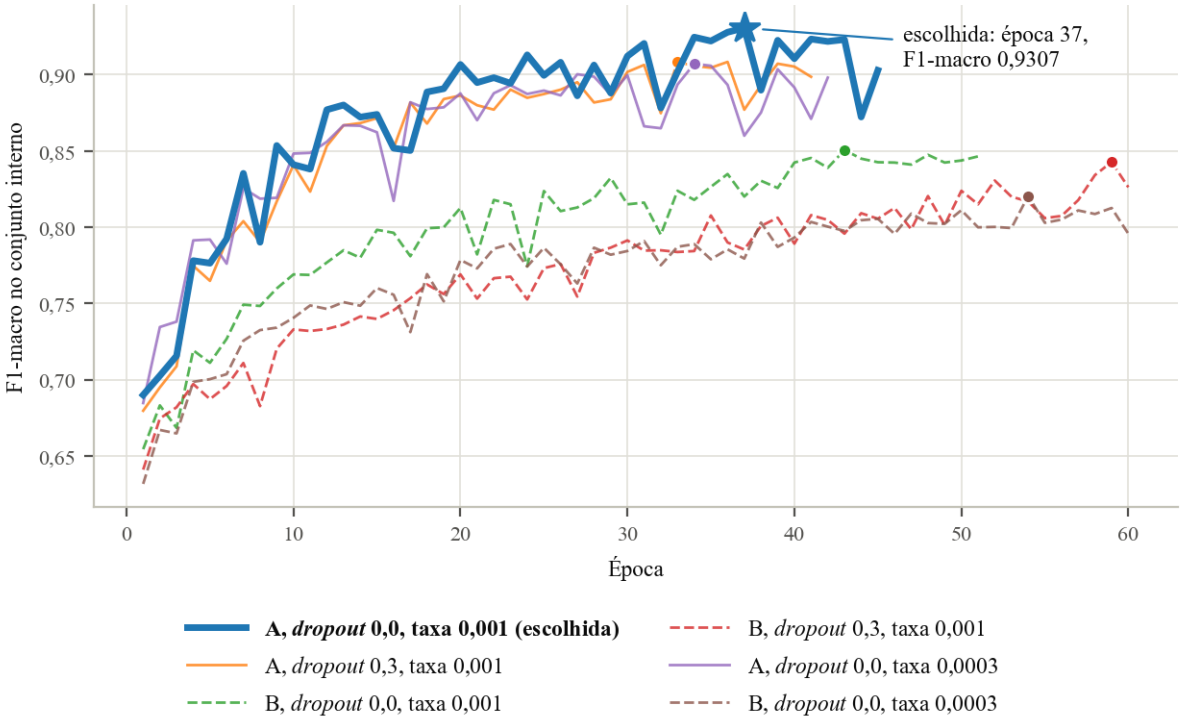
Na grade de seis configurações (Seção 3.7; Tabela 7), o *dropout* de 0,0 teve o maior F1-macro médio na primeira fase (0,8906 contra 0,8756). Venceu a arquitetura de quatro blocos sem *dropout* e com taxa de 0,001 (Figura 6): melhor época 37, parada na época 45, F1-macro de 0,9307 e EER de 0,0600 no conjunto interno, com as duas métricas de acordo e sem desempate. Na curva da vencedora (Figura 7), a perda de treino tende a cair, com oscilações marcadas nas últimas épocas, e a do conjunto interno oscila bem mais, enquanto o F1-macro se estabiliza. Os pesos recarregados do disco reproduziram o F1-macro da melhor época (diferença de 0,0), o que confirma que o modelo salvo é o daquela época, e não o da última.

Tabela 7 – Grade de configurações da CNN, avaliada no conjunto interno de parada antecipada (3.000 exemplos)

Fase	Arquitetura (canais)	Parâmetros	*Dropout*	Taxa de aprendizado	Melhor época	Parada	F1-macro	EER
1	**A: 4 blocos (32, 64, 128, 128)**	240.866	0,0	0,001	37	paciência, época 45	0,9307	0,0600
1	A: 4 blocos (32, 64, 128, 128)	240.866	0,3	0,001	33	paciência, época 41	0,9084	0,0637
1	B: 3 blocos (16, 32, 64)	23.538	0,0	0,001	43	paciência, época 51	0,8506	0,0998
1	B: 3 blocos (16, 32, 64)	23.538	0,3	0,001	59	teto de 60 épocas	0,8427	0,1072
2	A: 4 blocos (32, 64, 128, 128)	240.866	0,0	0,0003	34	paciência, época 42	0,9072	0,0770
2	B: 3 blocos (16, 32, 64)	23.538	0,0	0,0003	54	teto de 60 épocas	0,8205	0,1109

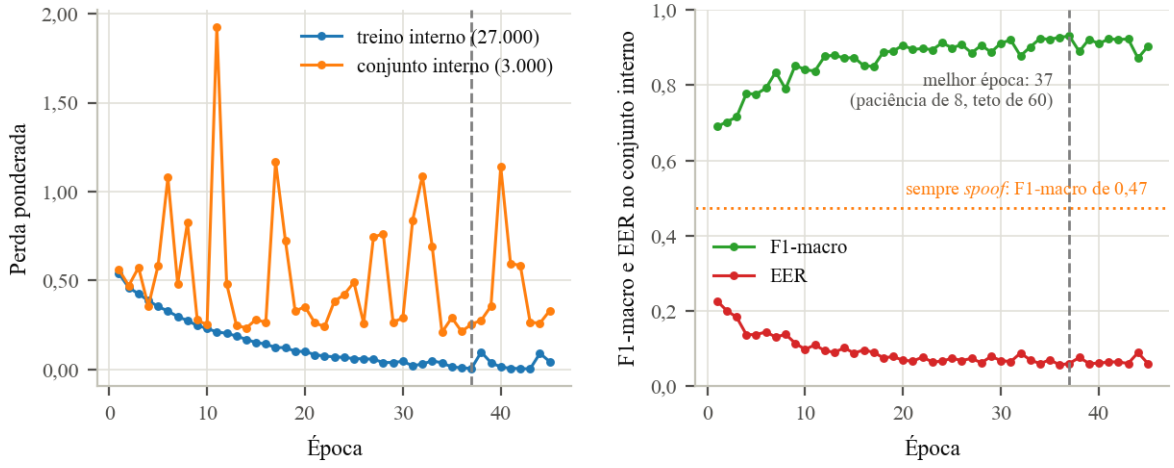
Fonte: Elaborado pelo autor (2026).
Nota: Em negrito, a configuração vencedora. Os valores vêm de exemplos da própria subamostra de treino, com o limiar escolhido neles, e não se comparam aos da validação externa.

Figura 6 – F1-macro no conjunto interno de parada antecipada, por época, nas seis configurações da grade da CNN (a estrela marca a melhor época da configuração escolhida)



Fonte: Elaborado pelo autor (2026).
Nota: A: quatro blocos (32, 64, 128, 128), linha cheia; B: três blocos (16, 32, 64), linha tracejada (Tabela 7). O círculo marca a melhor época de cada configuração e a estrela, a da configuração escolhida. Parada por paciência de 8 épocas, com teto de 60.

Figura 7 – Curvas de treino da configuração escolhida da CNN: perda ponderada no treino interno (27.000) e no conjunto interno (3.000), à esquerda, e F1-macro e EER no conjunto interno, à direita



Fonte: Elaborado pelo autor (2026).
Nota: A linha tracejada marca a melhor época (37), escolhida pelo F1-macro no conjunto interno; a pontilhada, o F1-macro de um classificador que respondesse sempre *spoof* nesse conjunto (0,47).

No retreinamento final, a vencedora treinou 37 épocas fixas nos 30.000 áudios do braço principal, sem parada antecipada, para ver as mesmas amostras que o RF e o SVM, com a

normalização recalculada (diferença máxima de 0,016 dB na média e de 0,012 dB no desvio em relação aos 27.000) e os pesos da perda derivados das contagens do próprio conjunto (3.000 *bonafide* e 27.000 *spoof*). Sem acesso à validação externa, a única conferência legítima foi a perda de treino na mesma época: 0,0136 no retreinamento contra 0,0049 na seleção, razão de 2,77. Os três módulos de treino auditados mantiveram o MD5, e o treino levou 887,52 s. A escolha de fixar épocas, em vez de atualizações de gradiente, é discutida na Seção 5.10.

A validação externa deu à CNN o mesmo tipo de registro do RF e do SVM: o modelo persistido e o limiar selecionado na validação, 0,3252, entre 9.322 valores distintos de *score*, sem nenhuma métrica reimplementada, com a normalização do retreinamento exigida por asserção e o TF32 desligado (Seção 4.5). Antes de aceitar o limiar, o módulo conferiu o alinhamento entre *score* e rótulo: rótulos e máscaras idênticos aos do índice nas 22.226 posições, e sete posições reprocessadas isoladamente com desvio máximo de $3{,}6 \times 10^{-7}$, contra $1{,}0 \times 10^{-3}$ de um deslocamento de uma posição (margem de 2.846,7, para um mínimo exigido de 100). O registro mostra ainda a *softmax* saturada: 5.101 dos 22.226 *scores* são exatamente 1,0, o que dá ao *score* da CNN granularidade mais grossa que a do RF. As métricas da CNN estão na Seção 5.1.

4.5 Problemas Encontrados e Controles de Integridade

Esta seção reúne os problemas técnicos que a execução encontrou, com o sintoma, a forma de detecção e o controle que permaneceu (Quadro 6), e os controles que protegeram o conjunto de teste. Nenhum dos problemas se revelou pela leitura do código: todos foram encontrados por medição ou por uma guarda automática.

Quadro 6 – Problemas técnicos encontrados na execução: sintoma, detecção e controle permanente

Problema	**Sintoma**	**Como foi detectado**	**Controle que ficou**
leitura de FLAC no Windows	falhas com mensagem de erro vazia numa rodada de extração	leitura direta pela biblioteca soundfile, que expôs a exceção do decodificador; arquivos conferidos por um decodificador independente	versão da biblioteca fixada; leitura direta na função de carga; varredura de legibilidade antes da extração definitiva
predição paralela do RF	número de *scores* distintos diferente entre reexecuções do mesmo modelo	guarda de reprodução; predições repetidas em paralelo e com um único processo	função de predição do RF que força um único processo
TF32 nas convoluções da CNN	*score* dependente do tamanho do lote	conferência do alinhamento entre *score* e rótulo, com critério comparativo	precisão simples plena imposta dentro da função que produz os *scores*, com registro no artefato
tempo em GPU sem sincronização	latência da CNN muito menor que a real	medição lado a lado, com e sem sincronização	sincronização dentro da função cronometrada; demonstração publicada no registro
GPU em estado de baixo consumo	latência da CNN inflada na primeira execução da validação externa	medição com a placa aquecida e ociosa	aquecimento explícito antes do cronômetro; número frio publicado ao lado

Fonte: Elaborado pelo autor (2026).

O primeiro problema foi a leitura dos áudios. Uma rodada antiga de extração registrou 79.645 falhas com mensagem de erro vazia (número registrado no dossiê da extração definitiva). A causa ficava escondida porque a função de carga do librosa descartava a exceção do soundfile e recorria a um leitor alternativo que, no Windows sem o ffmpeg, só produz esse erro. Lida diretamente pelo soundfile, a exceção apontou o decodificador FLAC da libsndfile embutida nas versões 0.13 e 0.14 para Windows, que neste ambiente apresentou falha determinística em cerca de 44% dos arquivos válidos, conforme o arquivo de dependências do projeto (Seção 3.10). O controle foi fixar a versão 0.12.1 e ler diretamente pelo soundfile; antes da extração definitiva, os 148.176 arquivos do universo foram lidos sem falha, e o arquivo de proveniência mista foi preservado como evidência, fora de qualquer análise.

O segundo e o terceiro problemas foram variações no último bit do *score*, invisíveis nas métricas e visíveis no número de candidatos a limiar, que conta os valores distintos do *score*. Na revisão do ramo clássico, a guarda de reprodução acusou esse número mudando de 22.103 para 22.104 no RF de referência. Repetida a predição, a contagem variou com a predição paralela e ficou idêntica, bit a bit, com um único processo, porque a soma paralela das árvores tem ordem variável e a soma em ponto flutuante não é associativa. A maior diferença de *score* foi de $4{,}4 \times 10^{-16}$, sem efeito no F1-macro nem no EER. O controle foi uma função de predição

do RF que força um único processo e passou a ser o único caminho de predição do RF no repositório.

Na CNN, a primeira execução da validação externa encontrou um desvio de $1{,}9 \times 10^{-5}$ entre o *score* de um exemplo calculado em lote e isoladamente. O critério comparativo indicou arredondamento em vez de desalinhamento, e a origem foi o TF32, precisão reduzida que a biblioteca cuDNN usa por padrão nas convoluções. Medido nos 256 primeiros exemplos da validação, o TF32 fazia o *score* depender do tamanho do lote em até $8{,}6 \times 10^{-5}$ ($2{,}7 \times 10^{-6}$ sem ele), e a diferença para a precisão simples plena chegava a $2{,}3 \times 10^{-3}$, da mesma ordem do espaçamento mediano entre *scores* vizinhos, de cerca de $3{,}5 \times 10^{-3}$, valores registrados na documentação dos módulos de validação e de inferência da CNN. Como o limiar é escolhido entre os valores distintos do *score*, o TF32 podia deslocá-lo. O controle foi impor a precisão simples plena dentro da própria função que produz os *scores*, com o estado anterior registrado nos artefatos da validação e do teste.

Os dois últimos problemas foram armadilhas de medição do tempo da CNN em GPU, em direções opostas. Sem sincronização antes e depois do trecho cronometrado, o relógio mede o enfileiramento das operações assíncronas: num lote de 128 exemplos, 0,9076 ms sem sincronização contra 28,6961 ms com ela, um fator de 31,6. A sincronização foi posta dentro da função cronometrada, sem alterar o módulo de cronometragem comum aos três modelos. Na direção contrária, os 3 descartes de aquecimento não tiram a placa do estado de baixo consumo depois de um trecho longo em CPU, e a primeira execução da validação externa publicou a latência fria. A correção foi um aquecimento explícito de 2,0 s (1.979 iterações) antes do cronômetro, com o número frio publicado ao lado: mediana de 7,66 ms sem aquecer contra 1,11 ms depois do aquecimento.

Os controles que protegeram o teste ficaram no código. A função que carrega os modelos ajustados, ponto único dos diagnósticos e do teste, recusa com exceção um limiar que não tenha sido selecionado na validação, sem recalculá-lo nem usar um valor padrão; assim, os limiares provisórios da CNN, escolhidos nos 3.000 exemplos internos, nunca poderiam chegar ao teste. Auditorias da árvore sintática do código-fonte procuraram, nos módulos de treino da seleção e do retreinamento da CNN, os nomes dos arquivos da validação externa e do teste; na validação externa, os do teste; e, na execução do teste, qualquer chamada de treino ou de seleção de limiar nos módulos que pontuam e avaliam. Nenhuma ocorrência proibida foi encontrada.

Antes do teste, um ensaio completo rodou sobre a validação, gravando fora da pasta de resultados, e, na execução real, cada um dos quatro modelos teve de reproduzir na validação as métricas, a matriz de confusão, a ROC-AUC e o número de *scores* distintos do seu registro,

com tolerância de $1{,}0 \times 10^{-9}$, antes da abertura do teste. O *script* do teste recusa uma segunda execução (reexecutá-lo exige as opções `--forcar` e `--razao`, com a razão gravada no artefato), e toda análise posterior, inclusive a comparação estatística do Capítulo 5, leu os *scores* gravados, sem pontuar o teste de novo.

Por fim, a guarda de reprodução comparou campo a campo os registros regenerados com cópias anteriores, sem as medidas de relógio, e confirmou os oito artefatos do ramo clássico e os quatro da CNN que dependem só de inferência, regenerados em arquivos à parte para não medir de novo os tempos citados no texto. Os artefatos de treino da CNN não foram regenerados, o que retreinaria a rede e sobrescreveria o modelo congelado; a reprodutibilidade do treino apoia-se na semente fixa e no modo determinístico estrito (Seção 3.10).

5 RESULTADOS E DISCUSSÃO

Este capítulo apresenta os resultados do protocolo descrito no Capítulo 3 e os discute à luz da pergunta de pesquisa. As Seções 5.1 a 5.3 trazem o desempenho dos quatro modelos na validação, onde os limiares foram escolhidos, e no teste, avaliado uma única vez, e a variação entre os dois conjuntos. A Seção 5.4 compara os modelos do braço principal pelo *bootstrap* pareado, e a Seção 5.5 mede, com o braço de referência, o custo da subamostragem. As Seções 5.6 a 5.8 reúnem as análises complementares: o desempenho por ataque e por *codec*, o custo computacional e a importância das características no RF. A Seção 5.9 responde à pergunta de pesquisa, e a Seção 5.10 discute as limitações.

5.1 Resultados na Validação

A Tabela 8 reúne o desempenho dos quatro modelos na validação, o conjunto em que cada limiar foi escolhido. Entre os modelos do braço principal, a ordem foi a mesma nas quatro métricas: a CNN teve F1-macro de 0,8975 e EER de 0,0738; o SVM, 0,7987 e 0,1462; e o RF, 0,7225 e 0,1930. Entre a CNN e o SVM, a distância é grande: o EER da CNN ficou em cerca da metade do EER do SVM. O *recall* de *bonafide*, a fração dos áudios reais reconhecida como real, mostra onde a diferença se concentra: 0,5441 no RF, 0,6625 no SVM e 0,8101 na CNN, enquanto o *recall* de *spoof* ficou acima de 0,93 nos três modelos.

Tabela 8 – Desempenho dos quatro modelos na validação (n = 22.226), com o limiar selecionado nesse conjunto

Modelo	n de treino	Limiar	F1-macro	EER	ROC-AUC	*Recall* de *bonafide*
RF (principal)	30.000	0,6516	0,7225	0,1930	0,8873	0,5441
RF (referência)	103.723	0,6196	0,7723	0,1579	0,9191	0,6008
SVM	30.000	−0,0329	0,7987	0,1462	0,9289	0,6625
CNN	30.000	0,3252	0,8975	0,0738	0,9799	0,8101

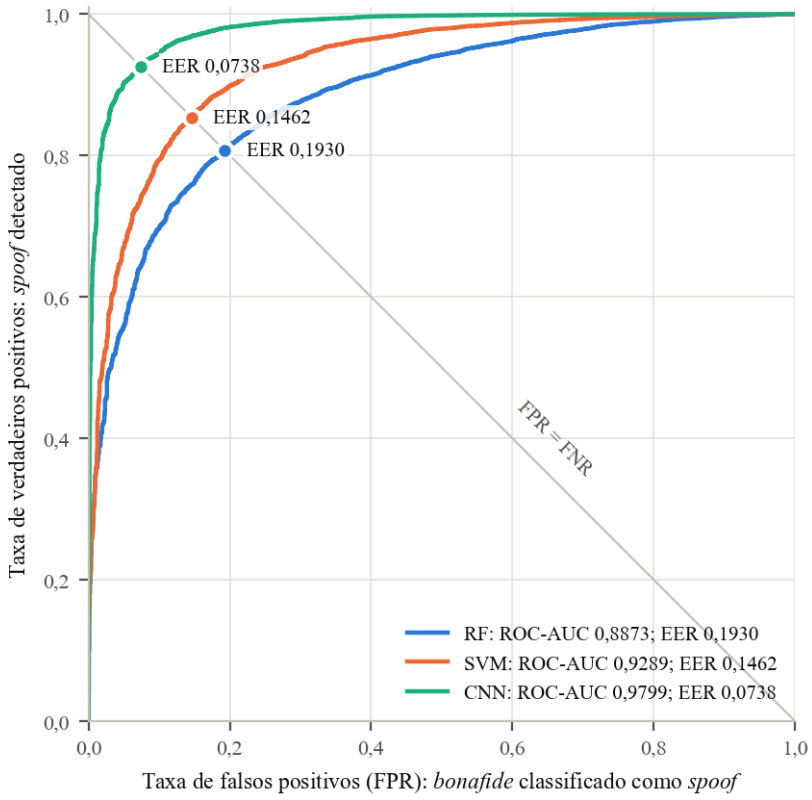
Fonte: Elaborado pelo autor (2026).
Nota: Regra de decisão: *spoof* se *score* ≥ limiar, com o limiar selecionado na própria validação pelo F1-macro. O limiar do SVM está na escala da função de decisão (real, centrada em zero); os do RF e da CNN, na escala de probabilidade, entre 0 e 1. O RF de referência, treinado nas 103.723 amostras do treino completo, mede o custo da subamostragem e não é concorrente do SVM nem da CNN, treinados nas mesmas 30.000 amostras do braço principal. EER e ROC-AUC não dependem de limiar.

Os limiares selecionados ficaram longe de 0,50 nos dois modelos com *score* de probabilidade, e em direções opostas: 0,6516 no RF e 0,3252 na CNN. Com o limiar padrão, cada um desses modelos teria decidido num ponto diferente da sua curva, e a comparação dependeria de qual deles tem os *scores* mais bem centrados; a seleção do limiar na validação, comum aos três (Seção 3.8), removeu essa dependência.

O RF de referência, treinado nas 103.723 amostras do treino completo, ficou entre o RF principal e o SVM (F1-macro de 0,7723 e EER de 0,1579). Ele não é concorrente do SVM nem da CNN: existe para medir quanto desempenho a subamostragem custou ao RF (Seção 5.5). A comparação entre modelos é a do braço principal, em que os três usaram as mesmas 30.000 amostras de treino.

A Figura 8 mostra as curvas ROC dos três modelos do braço principal, traçadas na validação. A curva da CNN ficou acima das outras duas em toda a extensão, e a do SVM acima da do RF, exceto no trecho inicial de taxa de falsos positivos muito baixa, em que as duas se cruzam. Fora desse trecho, portanto, a ordem entre os modelos não depende do ponto de operação escolhido, e a ROC-AUC resume o mesmo resultado (0,9799, 0,9289 e 0,8873). Para o teste, a ROC-AUC está na Tabela 9.

Figura 8 – Curvas ROC dos três modelos do braço principal na validação (n = 22.226), com o EER marcado no cruzamento com a reta FPR = FNR



Fonte: Elaborado pelo autor (2026).

5.2 Resultados no Teste Lacrado

No teste, avaliado uma única vez com os limiares da validação (Tabela 9), a ordem se repetiu. A CNN teve F1-macro de 0,8883 e EER de 0,0737; o SVM, 0,7981 e 0,1411; e o RF,

0,7210 e 0,1946. De novo, o EER da CNN ficou em cerca da metade do EER do SVM. O RF de referência, com 0,7602 e 0,1629, continuou entre o RF principal e o SVM.

Tabela 9 – Desempenho dos quatro modelos no teste (n = 22.227), com o limiar selecionado na validação

Modelo	n de treino	Limiar	F1-macro	EER	ROC-AUC	*Recall* de *bonafide*
RF (principal)	30.000	0,6516	0,7210	0,1946	0,8878	0,5326
RF (referência)	103.723	0,6196	0,7602	0,1629	0,9167	0,5794
SVM	30.000	−0,0329	0,7981	0,1411	0,9315	0,6635
CNN	30.000	0,3252	0,8883	0,0737	0,9783	0,7877

Fonte: Elaborado pelo autor (2026).
Nota: Conjunto de teste avaliado uma única vez, com o limiar que cada modelo recebeu na validação (Tabela 8); nenhum limiar foi selecionado no teste. Valem as observações da nota daquela tabela sobre a escala do limiar do SVM e sobre o RF de referência.

A Tabela 10 mostra por que a acurácia não serviu de critério de comparação. O RF principal teve acurácia de 0,8940 no teste, menor que os 0,9000 de um classificador que respondesse sempre *spoof* (Seção 3.8), e ainda assim teve F1-macro de 0,7210, contra 0,4737 desse classificador trivial. Como o *spoof* tem nove vezes mais exemplos que o *bonafide*, a acurácia é dominada pela classe majoritária; o F1-macro dá o mesmo peso às duas classes e registra o *recall* de *bonafide* do RF, de 0,5326, enquanto o classificador trivial não reconhece nenhum áudio real. Com o limiar escolhido pelo F1-macro, o RF trocou parte dos acertos de *spoof* (*recall* de 0,9342) por acertos de *bonafide*, troca que a acurácia contabiliza como perda.

Tabela 10 – Acurácia e métricas por classe dos quatro modelos no teste (n = 22.227)

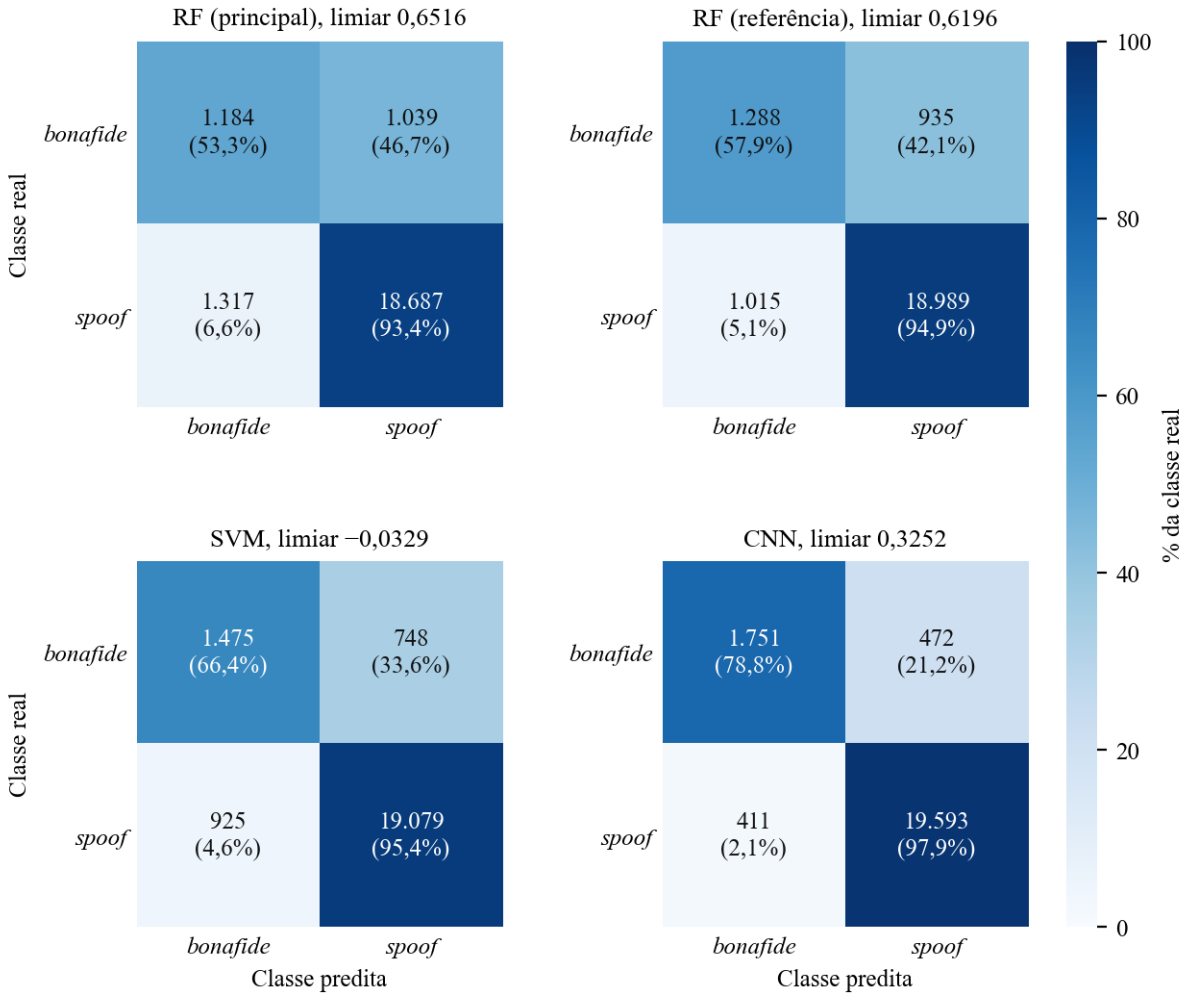
Modelo	Acurácia	Precisão *bonafide*	*Recall* *bonafide*	F1 *bonafide*	Precisão *spoof*	*Recall* *spoof*	F1 *spoof*
RF (principal)	0,8940	0,4734	0,5326	0,5013	0,9473	0,9342	0,9407
RF (referência)	0,9123	0,5593	0,5794	0,5692	0,9531	0,9493	0,9512
SVM	0,9247	0,6146	0,6635	0,6381	0,9623	0,9538	0,9580
CNN	0,9603	0,8099	0,7877	0,7986	0,9765	0,9795	0,9780

Fonte: Elaborado pelo autor (2026).
Nota: *Bonafide* é a classe 0, com 2.223 áudios no teste, e *spoof* a classe 1, com 20.004 (somas das linhas das matrizes de confusão). Mesmos limiares da Tabela 9.

As matrizes de confusão (Figura 9) mostram que a diferença entre os modelos está sobretudo na classe minoritária. O *recall* de *spoof* ficou acima de 0,93 nos quatro modelos, enquanto o de *bonafide* foi de 0,5326 no RF, 0,6635 no SVM e 0,7877 na CNN: o RF classificou como *spoof* quase metade dos áudios reais, o SVM cerca de um terço e a CNN cerca de um quinto. A precisão de *bonafide* seguiu a mesma ordem (0,4734, 0,6146 e 0,8099). No RF, menos da metade dos áudios aceitos como reais era de fato *bonafide*, porque os 1.317 *spoof* que o

modelo deixou passar superaram os 1.184 *bonafide* que ele reconheceu. Na CNN, o veredito de áudio real passou a acertar em cerca de quatro de cada cinco casos.

Figura 9 – Matrizes de confusão dos quatro modelos no teste (n = 22.227), com o limiar selecionado na validação: contagem e percentual da classe real em cada célula



Fonte: Elaborado pelo autor (2026).

5.3 Da Validação para o Teste

A Tabela 11 compara cada modelo nos dois conjuntos. As quatro variações, em F1-macro e em EER, ficaram dentro do ruído de amostragem estimado para a diferença entre dois conjuntos independentes desse tamanho, e a ordem dos três modelos do braço principal foi a mesma na validação e no teste (Figura 10). O desempenho medido na validação, com os limiares escolhidos nela, se transportou para o teste dentro do que o tamanho dos conjuntos permite distinguir. Esse ruído é uma heurística de aproximação normal; a comparação entre os modelos é feita pelo *bootstrap* pareado da Seção 5.4.

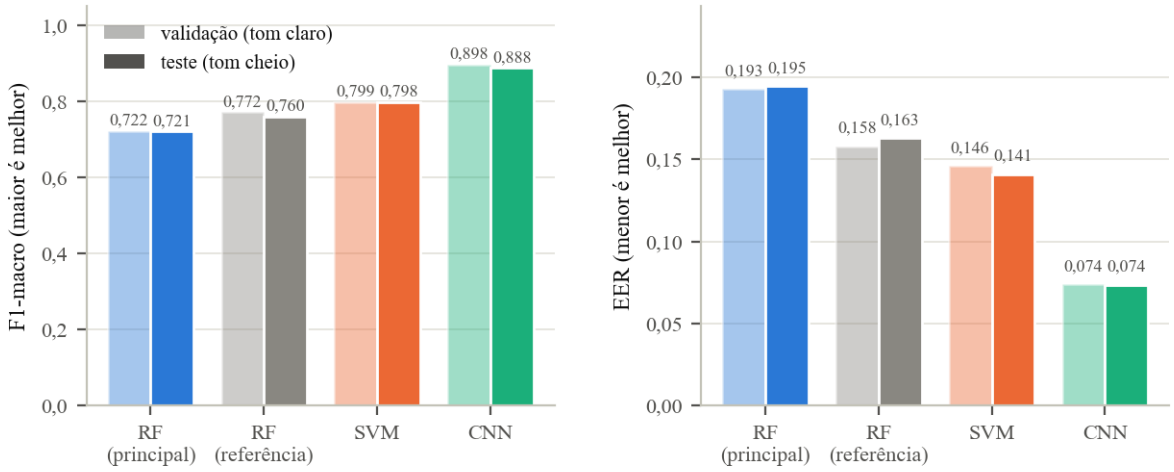
Tabela 11 – Variação do F1-macro e do EER da validação para o teste, por modelo, com o ruído de amostragem heurístico da diferença

Modelo	F1-macro na validação	F1-macro no teste	ΔF1-macro	Ruído ΔF1-macro	EER na validação	EER no teste	ΔEER	Ruído ΔEER
RF (principal)	0,7225	0,7210	−0,0015	±0,0133	0,1930	0,1946	+0,0016	±0,0115
RF (referência)	0,7723	0,7602	−0,0121	±0,0132	0,1579	0,1629	+0,0050	±0,0105
SVM	0,7987	0,7981	−0,0007	±0,0126	0,1462	0,1411	−0,0051	±0,0108
CNN	0,8975	0,8883	−0,0092	±0,0098	0,0738	0,0737	−0,0001	±0,0079

Fonte: Elaborado pelo autor (2026).
Nota: Δ = teste − validação. Ruído = 1,96 vezes a raiz da soma dos quadrados dos desvios-padrão do *bootstrap* individual da validação e do teste (1.000 reamostragens): heurística de aproximação normal para duas amostras independentes, diferente do teste pareado da Seção 5.4. O EER não depende de limiar; o F1-macro usa o limiar selecionado na validação.

Os quatro ΔF1-macro são negativos, e esse sinal é esperado pela construção do protocolo. O limiar de cada modelo foi o que maximizou o F1-macro na validação, de modo que o F1-macro da validação é o máximo de uma busca feita no próprio conjunto; transportado para outro conjunto, esse máximo tende a recuar. Trata-se do otimismo da seleção de limiar, que não indica perda de capacidade de separar as classes. Os recuos mais próximos do limite do ruído foram os do RF de referência (−0,0121, para um ruído de ±0,0132) e da CNN (−0,0092, para ±0,0098). No caso da CNN, o EER, que independe de limiar, variou apenas −0,0001: a capacidade de ordenar os áudios foi a mesma nos dois conjuntos, e o recuo do F1-macro se deve ao limiar transportado.

Figura 10 – F1-macro e EER dos quatro modelos na validação e no teste, com o limiar selecionado na validação



Fonte: Elaborado pelo autor (2026).
Nota: Tons claros: validação; tons cheios: teste. O RF de referência, treinado nas 103.723 amostras do treino completo, mede o custo da subamostragem e não é concorrente direto do SVM nem da CNN, treinados nas mesmas 30.000 amostras do braço principal.

5.4 Comparação Estatística: *Bootstrap* Pareado

As diferenças entre os modelos do braço principal foram avaliadas pelo *bootstrap* pareado descrito na Seção 3.8. O pareamento é necessário porque os três modelos foram avaliados nos mesmos áudios: um áudio difícil tende a ser difícil para todos, e os erros dos modelos são correlacionados. Comparar os intervalos de confiança individuais de cada modelo ignoraria essa correlação e superestimaria o ruído da diferença. Por isso, em cada reamostragem, o mesmo vetor de índices foi aplicado aos *scores* dos três modelos, e o intervalo foi calculado sobre a diferença entre eles.

Tabela 12 – Diferenças pareadas entre os modelos do braço principal na validação e no teste: Δ observado e IC95 do *bootstrap* pareado

Conjunto	Diferença	ΔF1-macro	IC95 ΔF1-macro	ΔEER	IC95 ΔEER	Reamostragens com o primeiro modelo melhor (F1-macro / EER)
Validação	SVM − RF	+0,0762	[0,0663; 0,0856]	−0,0468	[−0,0560; −0,0377]	100,0% / 100,0%
Validação	CNN − RF	+0,1750	[0,1650; 0,1853]	−0,1192	[−0,1275; −0,1102]	100,0% / 100,0%
Validação	CNN − SVM	+0,0988	[0,0887; 0,1093]	−0,0724	[−0,0812; −0,0635]	100,0% / 100,0%
Teste	SVM − RF	+0,0771	[0,0671; 0,0868]	−0,0535	[−0,0630; −0,0444]	100,0% / 100,0%
Teste	CNN − RF	+0,1673	[0,1568; 0,1775]	−0,1209	[−0,1304; −0,1118]	100,0% / 100,0%
Teste	CNN − SVM	+0,0902	[0,0801; 0,1005]	−0,0674	[−0,0763; −0,0582]	100,0% / 100,0%

Fonte: Elaborado pelo autor (2026).
Nota: Δ = métrica do primeiro modelo − métrica do segundo, calculada sobre o conjunto inteiro, diferente da média das reamostragens. IC95 pelos percentis 2,5 e 97,5 das 1.000 reamostragens (semente 42); em cada reamostragem, um único vetor de índices é aplicado aos *scores* de todos os modelos, com o limiar fixo no do protocolo. ΔEER negativo indica EER menor no primeiro modelo. n = 22.226 (validação) e 22.227 (teste).

Os três pares tiveram IC95 sem zero, em F1-macro e em EER, na validação e no teste (Tabela 12). No teste, a CNN superou o SVM em 0,0902 de F1-macro (IC95 [0,0801; 0,1005]) e teve EER menor em 0,0674 (IC95 do ΔEER [−0,0763; −0,0582]); a CNN superou o RF em 0,1673 de F1-macro (IC95 [0,1568; 0,1775]) e teve EER menor em 0,1209 (IC95 do ΔEER [−0,1304; −0,1118]); e o SVM superou o RF em 0,0771 de F1-macro (IC95 [0,0671; 0,0868]) e teve EER menor em 0,0535 (IC95 do ΔEER [−0,0630; −0,0444]). Em todas as 1.000 reamostragens, nos dois conjuntos, o primeiro modelo de cada par foi melhor nas duas métricas. A ordem CNN, SVM e RF é, portanto, distinguível neste protocolo. Na validação, o par SVM

e RF reproduziu exatamente o resultado obtido antes do treino da CNN (Seção 4.3), porque o sorteio dos índices é o mesmo.

O *bootstrap* reamostra a avaliação, não o treino. O intervalo mede a incerteza que vem de a validação e o teste serem amostras, mas não a variação que outro treino do mesmo modelo produziria. Essa variação foi medida à parte: no RF, o desvio-padrão do F1-macro entre sementes foi de 0,0004 (Seção 4.3), e o SVM tem treino determinístico. A CNN foi treinada com uma única semente, limitação discutida na Seção 5.10.

5.5 Custo da Subamostragem: o Braço de Referência

O braço de referência mede quanto o RF perdeu por ter sido treinado em 30.000 exemplos em vez dos 103.723 do treino completo, com os mesmos hiperparâmetros (Tabela 13). No teste, o RF de referência teve F1-macro maior em 0,0392 (IC95 [0,0329; 0,0451]) e EER menor em 0,0317 (IC95 do ΔEER [−0,0368; −0,0252]); na validação, as diferenças foram de 0,0498 e −0,0351. A subamostragem teve, portanto, um custo real e distinguível neste protocolo para o RF.

Tabela 13 – Custo da subamostragem para o RF: diferença pareada entre o RF de referência (103.723) e o RF principal (30.000), na validação e no teste

Conjunto	ΔF1-macro	IC95 ΔF1-macro	ΔEER	IC95 ΔEER	Reamostragens com a referência melhor (F1-macro / EER)
Validação	+0,0498	[0,0438; 0,0561]	−0,0351	[−0,0406; −0,0292]	100,0% / 100,0%
Teste	+0,0392	[0,0329; 0,0451]	−0,0317	[−0,0368; −0,0252]	100,0% / 100,0%

Fonte: Elaborado pelo autor (2026).
Nota: Δ = RF de referência (103.723 amostras de treino) − RF principal (30.000), com o mesmo algoritmo e os mesmos hiperparâmetros. Mesmo procedimento da Tabela 12. Não é um par da comparação entre modelos.

Esse custo não altera a comparação entre os modelos. Com 3,5 vezes mais dados, o RF de referência (F1-macro de 0,7602 e EER de 0,1629 no teste) ficou abaixo do SVM (0,7981 e 0,1411) e da CNN (0,8883 e 0,0737) treinados em 30.000. O par formado pelo RF de referência e pelo SVM não foi submetido ao *bootstrap* pareado, por não ser um par da comparação, mas os intervalos de confiança individuais dos dois no teste não se sobrepõem em nenhuma das duas métricas (no F1-macro, [0,7512; 0,7689] contra [0,7890; 0,8062]), critério mais conservador que o pareado. O braço de referência, porém, fala só do RF: não diz quanto o SVM ou a CNN ganhariam com o treino completo, porque nenhum dos dois foi treinado nele.

A curva de aprendizado do RF (Figura 4, Seção 4.3) mostra o mesmo custo por outro ângulo: o F1-macro ainda crescia no último passo, de 80.000 para 103.723 exemplos, e a curva não saturou no maior tamanho disponível. Mais dados continuariam a ajudar o RF; até onde, e

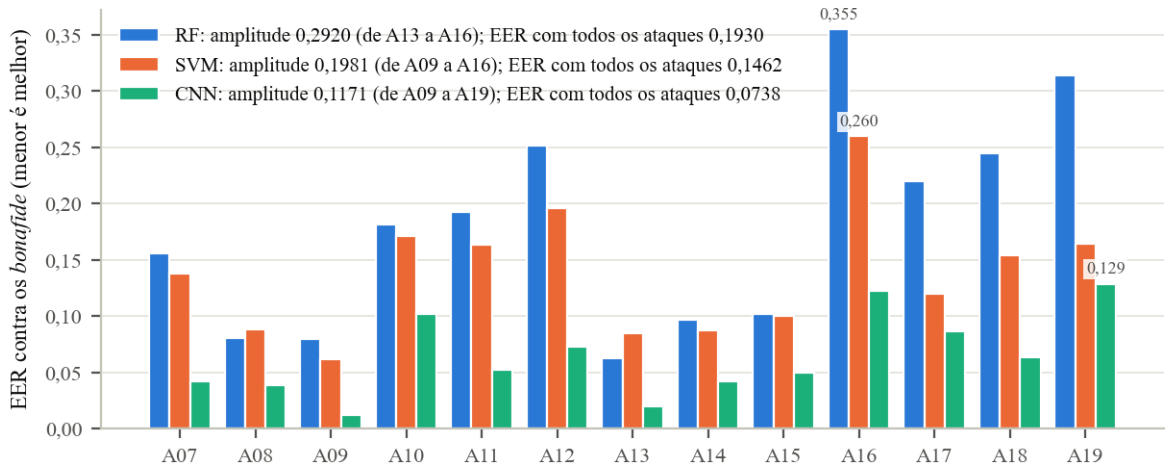
se o suficiente para alcançar o SVM, a curva medida não responde, e a extrapolação dela é discutida na Seção 5.10 como ordem de grandeza.

5.6 Análise por Ataque e por *Codec*

As análises por ataque e por *codec* foram feitas na validação, com os modelos persistidos e os limiares do protocolo, sem nenhum treino. O teste ficou reservado às métricas principais das Seções 5.2 a 5.4 e não foi usado para diagnóstico.

O EER de cada sistema de ataque, calculado contra os mesmos *bonafide* da validação, variou muito dentro de cada modelo (Figura 11 e Tabela 14). No RF, foi de 0,0628 no ataque A13 a 0,3548 no A16, uma amplitude de 0,2920; no SVM, de 0,0617 (A09) a 0,2598 (A16), amplitude de 0,1981. Nos dois modelos clássicos, essa amplitude superou a distância agregada de EER para os outros dois modelos (Tabela 15), isto é, a dificuldade dependeu mais do sistema de síntese que produziu o áudio do que do classificador que o avaliou.

Figura 11 – EER por sistema de ataque (A07 a A19) dos três modelos do braço principal na validação (n = 22.226)



Fonte: Elaborado pelo autor (2026).
Nota: O número sobre a barra marca o ataque de maior EER de cada modelo; na legenda, a amplitude entre ataques (Tabela 14) e o EER com todos os ataques juntos. O EER independe de limiar.

Tabela 14 – Resumo do EER por sistema de ataque na validação (n = 22.226), por modelo do braço principal

Modelo	Ataque de menor EER (EER)	Ataque de maior EER (EER)	Amplitude do EER	Razão maior / menor	Três ataques de maior EER
RF (principal)	A13 (0,0628)	A16 (0,3548)	0,2920	5,65	A16, A19, A12
SVM	A09 (0,0617)	A16 (0,2598)	0,1981	4,21	A16, A12, A10
CNN	A09 (0,0116)	A19 (0,1287)	0,1171	11,09	A19, A16, A10

Fonte: Elaborado pelo autor (2026).
Nota: EER de cada ataque contra os *bonafide* da validação, independente de limiar. Amplitude = EER do ataque de maior EER − EER do ataque de menor EER; razão = maior / menor. Ataques A07 a A19.

Na CNN, a amplitude caiu para 0,1171 (de 0,0116 no A09 a 0,1287 no A19), da mesma ordem da distância para o RF (0,1192; diferença de 0,0021, abaixo do piso de ruído de ±0,0086) e maior que a distância para o SVM (0,0724). Com a CNN, a variação entre ataques deixou de dominar a diferença entre os modelos, e a vantagem dela apareceu em toda a lista: nos treze ataques, o EER pontual da CNN foi menor que o do SVM e o do RF. Em razão entre o maior e o menor EER, a CNN foi a mais desigual (11,09, contra 5,65 do RF e 4,21 do SVM), mas o número se explica pelo denominador: o ataque mais fácil para a CNN ficou com EER de 0,0116, perto de zero.

Tabela 15 – Distância agregada de EER entre os modelos do braço principal na validação e piso de ruído heurístico

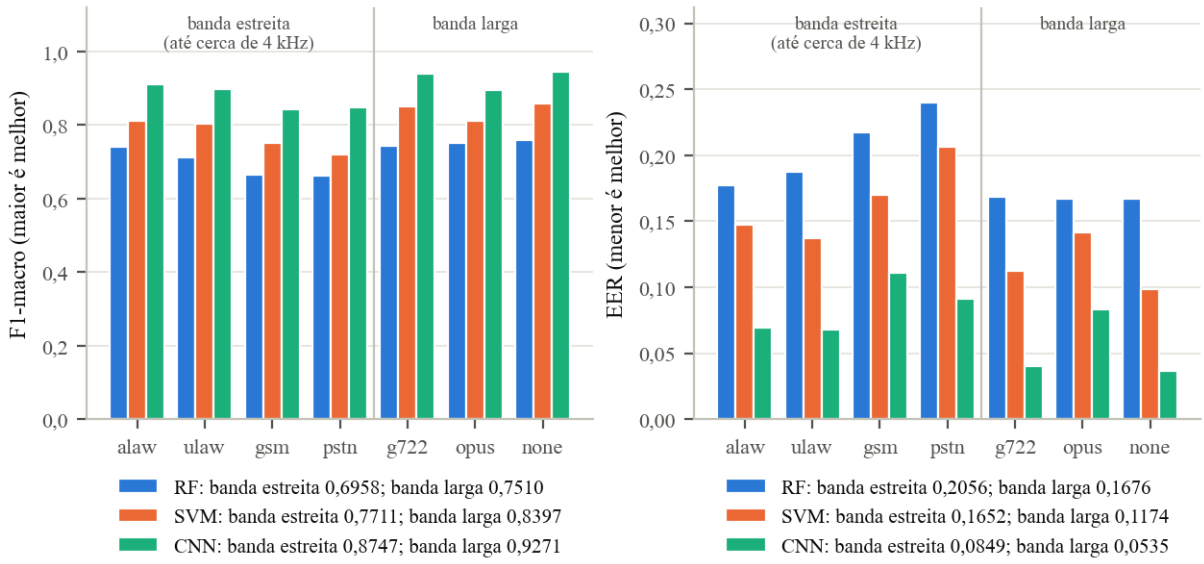
Par	|ΔEER| agregado	Piso de ruído
SVM − RF	0,0468	±0,0092
CNN − RF	0,1192	±0,0086
CNN − SVM	0,0724	±0,0086

Fonte: Elaborado pelo autor (2026).
Nota: Distância agregada = |ΔEER| observado na validação (Tabela 12). Piso de ruído = 1,96 × desvio-padrão do ΔEER pareado na validação: heurística que não é um intervalo de confiança da amplitude entre ataques, porque esta vem de subconjuntos menores (Seção 5.10).

O ataque A16 esteve entre os três de maior EER nos três modelos, o que indica que parte da dificuldade pertence ao sistema de síntese, qualquer que seja o classificador. Todos os ataques estavam presentes no treino, porque a partição foi aleatória por áudio, e mesmo assim o desempenho variou muito entre eles. O comportamento diante de ataques ausentes do treino não foi medido e é discutido na Seção 5.10.

Por *codec* (Figura 12 e Tabela 16), os três modelos foram melhores na banda larga que na banda estreita, com F1-macro maior e EER menor. Como o EER independe de limiar, o contraste não é efeito do limiar escolhido. O resultado é coerente com a observação de Shi *et al.* (2025), discutida na Seção 2.2, de que a compressão por *codecs* e a transmissão degradam o desempenho dos detectores: os *codecs* de banda estreita cortam o espectro perto de 4 kHz e eliminam o conteúdo acima dessa faixa. O efeito pesa no universo deste trabalho, em que 57,9% dos áudios são de banda estreita (Seção 3.2). A comparação é por banda agregada; *codec* a *codec*, a separação não é completa, pois o opus, de banda larga, teve EER maior que o alaw e o ulaw, de banda estreita, na CNN, e maior que o ulaw no SVM.

Figura 12 – F1-macro e EER por *codec* dos três modelos do braço principal na validação (n = 22.226), com o limiar do protocolo



Fonte: Elaborado pelo autor (2026).
Nota: Na legenda, a métrica de cada modelo sobre o conjunto de cada banda (Tabela 16). O F1-macro usa o limiar do protocolo; o EER independe de limiar.

A hipótese de que a CNN, por preservar a estrutura tempo-frequência até 8 kHz, ganharia mais com a banda larga ficou não decidida. Em termos absolutos, o ganho da CNN foi o menor dos três (redução de 0,0314 no EER, contra 0,0380 do RF e 0,0478 do SVM); em termos relativos, foi o maior (37,0% do EER da banda estreita, contra 18,5% e 28,9%). As duas escalas apontam em sentidos opostos, e falta um intervalo de confiança para a diferença entre os ganhos dos modelos (Seção 5.10). O que a CNN acrescenta com segurança é que o contraste entre as bandas aparece também num modelo que não usa ZCR nem centroide espectral, o que indica que ele está no próprio sinal, independentemente dessas duas características.

Tabela 16 – F1-macro e EER por banda de *codec* na validação, por modelo do braço principal

Modelo	F1-macro estreita	F1-macro larga	ΔF1-macro	EER estreita	EER larga	ΔEER	Redução relativa do EER
RF (principal)	0,6958	0,7510	+0,0552	0,2056	0,1676	−0,0380	18,5%
SVM	0,7711	0,8397	+0,0686	0,1652	0,1174	−0,0478	28,9%
CNN	0,8747	0,9271	+0,0524	0,0849	0,0535	−0,0314	37,0%

Fonte: Elaborado pelo autor (2026).
Nota: Banda estreita: alaw, ulaw, gsm, pstn (12.884 áudios); banda larga: g722, opus, none (9.342 áudios). Δ = larga − estreita. Redução relativa do EER = |ΔEER| / EER da banda estreita. Não há intervalo de confiança para a diferença entre os ganhos dos modelos (Seção 5.10).

5.7 Custo Computacional

Neste trabalho, custo computacional refere-se ao tempo de inferência (latência de um áudio isolado e vazão em lote), medido conforme o protocolo da Seção 3.9; custo de treinamento, uso de memória e consumo de energia não foram comparados formalmente. A Tabela 17 decompõe o tempo de inferência ponta a ponta de um áudio isolado nos quatro arranjos, medidos na mesma execução. O total de cada arranjo é a soma das medianas das etapas (a base comum de cada ramo foi medida uma vez e somada à predição de cada modelo, Seção 3.9), e não a mediana do tempo total medido por áudio. Em CPU, os dois modelos clássicos apresentaram menor latência que a CNN: 4,92 ms no SVM e 10,23 ms no RF, contra 16,80 ms da CNN. Com GPU, a CNN caiu para 4,58 ms, e CNN em GPU e SVM tiveram tempos da mesma ordem de grandeza: a diferença de 7,5% entre os dois fica dentro da variação de 4,1% a 13,7% medida nos tempos de predição da CNN entre duas execuções na mesma máquina (Seção 3.9 e nota da Tabela 17), de modo que não há ordenação estável entre eles, e isso também não constitui teste de equivalência. Sem GPU, a CNN levou 3,7 vezes o próprio tempo em GPU e foi o arranjo de maior latência. A vantagem dos modelos clássicos em latência de inferência está, portanto, em não exigirem GPU.

Tabela 17 – Tempo de inferência ponta a ponta de um áudio isolado (lote unitário), por etapa, nos quatro arranjos, em milissegundos (mediana)

Arranjo	Leitura	VAD e *padding*	Representação	Predição	Total	Predição (% do total)
RF (CPU)	0,57	0,34	3,43	5,88	10,23	57,5%
SVM (CPU)	0,57	0,34	3,43	0,57	4,92	11,6%
CNN (GPU)	0,58	0,35	2,48	1,17	4,58	25,5%
CNN (CPU)	0,58	0,35	2,48	13,39	16,80	79,7%

Fonte: Elaborado pelo autor (2026).
Nota: Os quatro arranjos foram medidos na mesma execução, sobre 200 áudios da validação (semente 42): mediana de 10 repetições após 3 de aquecimento, com lote unitário. Representação = extração das 44 características (RF e SVM, mesma base) ou geração do Log-Mel (CNN); na CNN, a normalização por faixa Mel está dentro da etapa de predição. As etapas comuns foram cronometradas separadamente na base clássica e na da CNN. Em CPU, uma única *thread*; GPU NVIDIA GeForce RTX 5060 Ti. Entre duas execuções feitas em momentos diferentes, na mesma máquina, os tempos de predição da CNN variaram de 4,1% a 13,7%. O total de cada arranjo é a soma das medianas das etapas (leitura, VAD e *padding*, representação e predição), conforme a Seção 3.9, e não a mediana do tempo total por áudio; como a soma é feita antes do arredondamento, no RF e no SVM a soma dos valores exibidos difere do total em 0,01 ms.

O peso do pré-processamento mudou com o arranjo. Leitura, VAD e extração das características custaram 4,35 ms no ramo clássico, e leitura, VAD e geração do Log-Mel, 3,41 ms no ramo da CNN. No SVM e na CNN em GPU, o classificador foi a menor parte do custo: a predição respondeu por 11,6% e 25,5% do total, e a representação dominou (a extração das

características, com 69,8% no SVM, e a geração do Log-Mel, com 54,1% na CNN em GPU). No RF e na CNN em CPU, ao contrário, a predição dominou, com 57,5% e 79,7% do total. Nos dois arranjos mais rápidos, portanto, a maior parte do tempo foi gasta antes do classificador.

A vazão em lote, medida só na predição, a partir da representação já extraída (Tabela 18), ordenou os modelos de outra forma. O RF, o clássico de maior latência para um áudio isolado, foi o mais rápido em lote, com 0,0204 ms por áudio (cerca de 49 mil áudios por segundo), seguido da CNN em GPU (0,2383 ms) e do SVM (0,4025 ms); a CNN em CPU ficou em 15,41 ms por áudio (cerca de 65 áudios por segundo), 64,6 vezes o tempo da GPU sobre os mesmos áudios. Na medição só da predição (Figura 13), um áudio isolado custou ao RF 6,11 ms (valor de uma medição diferente da Tabela 17, feita junto com a vazão em lote, em outra execução; na Tabela 17, a etapa de predição do RF aparece com 5,88 ms), e a mesma predição diluída no lote, 0,0204 ms por áudio, o que indica que quase todo o custo unitário é custo fixo da chamada, repartido entre os áudios quando eles chegam juntos. No SVM, cada áudio exige calcular o *kernel* contra os 7.605 vetores de suporte, e o custo por áudio pouco se diluiu (0,63 ms isolado, 0,40 ms em lote). Latência de um áudio e vazão em lote respondem a cenários de uso diferentes e não ordenam os modelos da mesma forma; por isso, as duas foram reportadas.

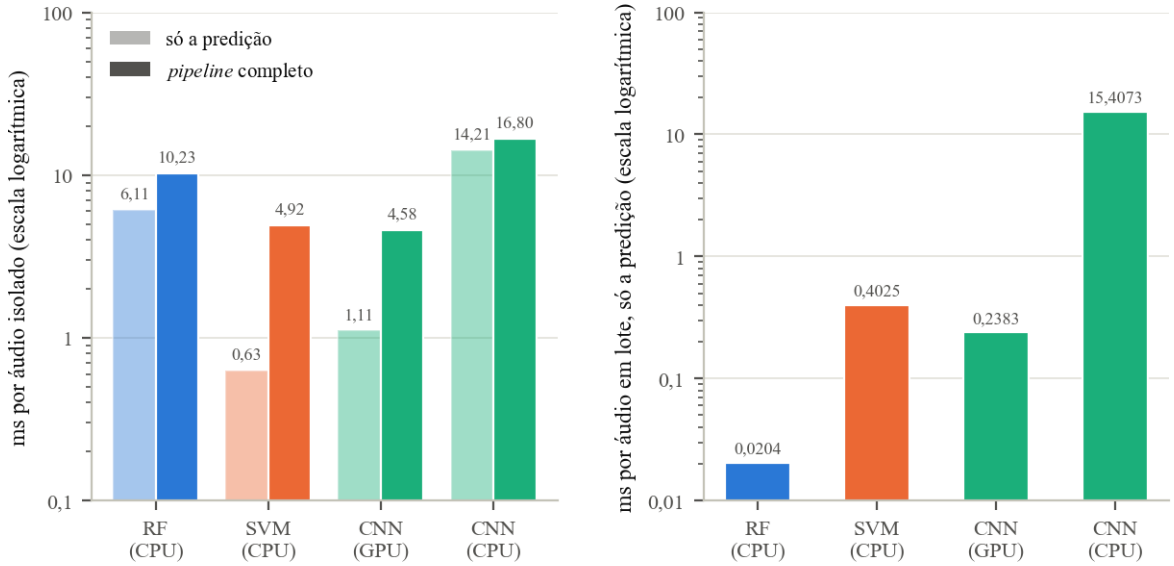
Tabela 18 – Vazão em lote da predição nos quatro arranjos, a partir da representação já extraída (mediana)

Arranjo	Áudios na medição	ms por áudio	Áudios por segundo
RF (CPU)	22.226	0,0204	48.943,6
SVM (CPU)	22.226	0,4025	2.484,2
CNN (GPU)	22.226	0,2383	4.197,1
CNN (CPU)	1.280	15,4073	64,9

Fonte: Elaborado pelo autor (2026).
Nota: Medição distinta da Tabela 17: só a predição, a partir das características ou do Log-Mel já extraídos, sem leitura, VAD nem representação. Mediana de 10 repetições após 3 de aquecimento. RF e SVM recebem os áudios de uma só vez; a CNN os processa em lotes de 128. Em CPU, a CNN foi medida sobre 1.280 dos 22.226 áudios da validação.

Esses resultados retomam o que a fundamentação antecipava (Seção 2.4.1). O custo de predição do RF, apontado ali como limitação, apareceu: a predição respondeu por 57,5% do tempo ponta a ponta do RF e o tornou o clássico de maior latência por áudio isolado, ainda que muito abaixo da duração do trecho analisado. O custo de treinamento do SVM, apontado como sua desvantagem em conjuntos grandes, pesou no desenho, em que a subamostra de 30.000 exemplos foi fixada *a priori* como orçamento experimental comum, tendo em vista a escalabilidade dele (Seção 3.6); na inferência de um áudio isolado, o SVM ficou entre os arranjos de menor latência. A relevância prática dessas diferenças para o uso de cada modelo é discutida na Seção 5.9.

Figura 13 – Custo de inferência nos quatro arranjos: por áudio isolado (só a predição e *pipeline* completo) e em lote (só a predição), em escala logarítmica

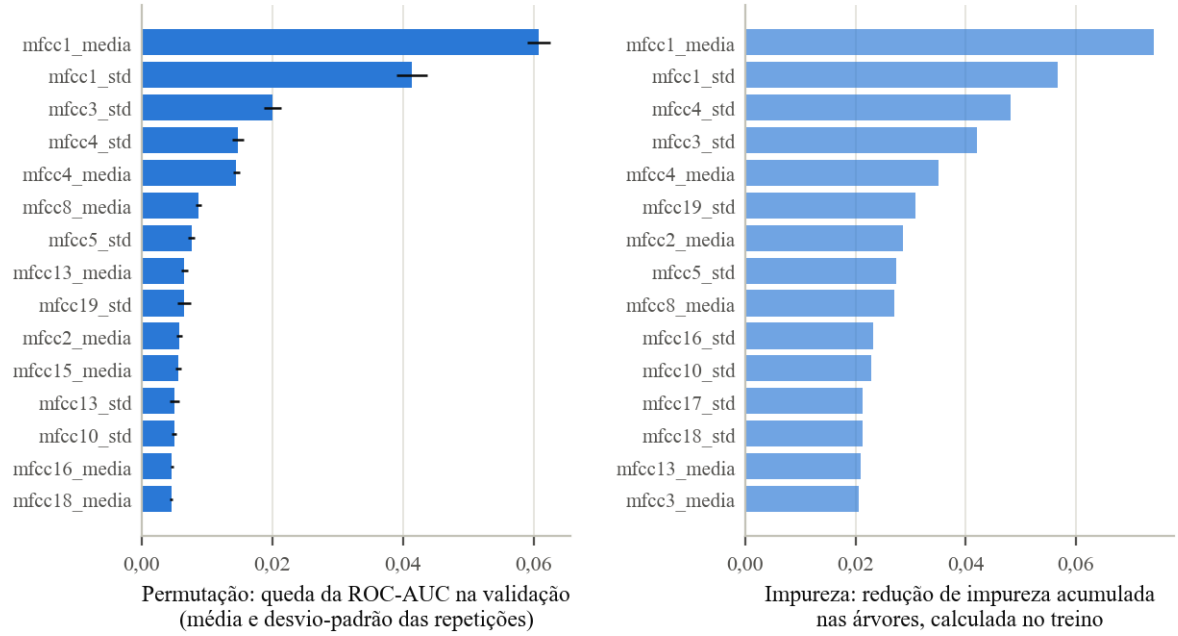


Fonte: Elaborado pelo autor (2026).
Nota: Barras claras: só a predição, medida junto com a vazão em lote; barras cheias: *pipeline* completo (soma das medianas das etapas, Tabela 17), medido sobre 200 áudios da validação. São medições distintas e não se somam. À direita, a vazão em lote (Tabela 18). Em CPU, uma única *thread*; GPU NVIDIA GeForce RTX 5060 Ti. A diferença entre CNN (GPU) e SVM no *pipeline* completo (4,58 e 4,92 ms) fica dentro da variação entre execuções (4,1% a 13,7%, Seção 3.9) e não sustenta ordenação entre os dois.

5.8 Importância das Características no *Random Forest*

A importância das características foi medida no RF principal por dois critérios (Figura 14 e Tabela 19): a redução de impureza acumulada nas árvores, calculada no treino, e a queda de ROC-AUC na validação quando os valores de uma característica são embaralhados, chamada importância por permutação. O primeiro critério tende a favorecer características contínuas com muitos valores distintos; o segundo não tem esse viés e é medido fora do treino. Os dois concordaram: a correlação de Spearman entre as duas ordenações das 44 características foi de 0,8516, e 9 das 10 primeiras coincidem. A análise foi feita só no RF, e a leitura vale para o que esse modelo usou, sem implicar causalidade acústica.

Figura 14 – Importância das características no RF principal: por permutação na validação e por redução de impureza



Fonte: Elaborado pelo autor (2026).
Nota: Em cada painel, as 15 primeiras das 44 características pelo critério. Nos rótulos, “media” e “std” indicam a média e o desvio-padrão da série, e o número, o coeficiente MFCC.

Os coeficientes MFCC dominaram os dois critérios. As 10 primeiras características, por permutação e por impureza, são todas médias ou desvios-padrão de coeficientes MFCC, com a média e o desvio-padrão do 1º coeficiente nas duas primeiras posições. As características de centroide espectral e de ZCR ficaram longe do topo: a mais bem colocada delas, o desvio-padrão do centroide, foi a 18ª por permutação (19ª por impureza), e a média da ZCR foi a última pelos dois critérios. Em todas as 44, a queda média de ROC-AUC ficou acima do desvio-padrão das repetições, de modo que nenhuma característica deixou de contribuir. Esse resultado atende ao objetivo específico de estudar as características acústicas relevantes (Seção 1.1.2): no RF, a informação útil concentrou-se nos coeficientes MFCC, sobretudo nos primeiros, e as séries de ZCR e de centroide contribuíram pouco.

Tabela 19 – As 10 características de maior importância por permutação no RF principal, com a importância e a posição por redução de impureza

Posição (permutação)	Característica	Queda de ROC-AUC (média)	Desvio-padrão	Importância por impureza	Posição (impureza)
1	MFCC 1, média	0,06084	0,00176	0,07419	1
2	MFCC 1, desvio-padrão	0,04135	0,00233	0,05672	2
3	MFCC 3, desvio-padrão	0,02006	0,00133	0,04200	4
4	MFCC 4, desvio-padrão	0,01476	0,00091	0,04815	3
5	MFCC 4, média	0,01450	0,00056	0,03504	5
6	MFCC 8, média	0,00867	0,00041	0,02702	9
7	MFCC 5, desvio-padrão	0,00762	0,00052	0,02733	8
8	MFCC 13, média	0,00653	0,00052	0,02100	14
9	MFCC 19, desvio-padrão	0,00649	0,00098	0,03093	6
10	MFCC 2, média	0,00579	0,00043	0,02866	7

Fonte: Elaborado pelo autor (2026).
Nota: Permutação: queda de ROC-AUC na validação (n = 22.226) ao embaralhar os valores da característica, com 10 repetições e semente 42; média e desvio-padrão das repetições. Impureza: redução de impureza acumulada nas árvores do RF, calculada no treino. Correlação de Spearman entre as duas ordenações das 44 características: $\rho = 0{,}8516$. Coincidem nos dois critérios 9 das 10 primeiras; entre as 10 primeiras por impureza, fora das 10 por permutação: MFCC 16, desvio-padrão (posição 10 por impureza e 16 por permutação).

A segunda característica pelos dois critérios, o desvio-padrão do 1º coeficiente MFCC (`mfcc1_std`), é também a única que manteve correlação acima de 0,3 com a proporção de fala depois do mascaramento ($|r| = 0{,}320$, Seção 4.2). Para medir quanto o RF dependia dela, o modelo foi retreinado sem essa característica, com a mesma configuração, a mesma semente, o mesmo treino e o limiar selecionado de novo na validação. Sem ela, o F1-macro na validação passou de 0,7225 para 0,7148 e o EER, de 0,1930 para 0,1980. O *bootstrap* pareado entre os dois modelos deu IC95 sem zero nas duas métricas, a favor do modelo com a característica ([0,0016; 0,0140] no F1-macro e [0,0006; 0,0111] na redução do EER), de modo que a contribuição existe; a magnitude, porém, é pequena: 2,6% do EER de referência e 10,7% da distância de EER entre RF e SVM na validação. A ablação não decide se a característica carrega um resíduo do atalho do silêncio ou informação acústica genuína, e, como as características correlacionadas com ela podem compensar a sua retirada, também não isola o efeito desse resíduo. O que ela mostra é que o RF perde pouco sem essa característica, e essa perda é pequena diante das diferenças entre os modelos.

5.9 Discussão: Resposta à Pergunta de Pesquisa

Neste protocolo, com 30.000 exemplos de treino e partição aleatória por áudio, os modelos clássicos não mantiveram desempenho preditivo competitivo (F1-macro e EER): a

CNN superou o SVM e o RF com diferenças estatisticamente distinguíveis. Em custo computacional, medido como tempo de inferência, a vantagem dos clássicos depende do *hardware*: sem GPU, eles apresentam menor latência de inferência ponta a ponta por áudio isolado (4,92 ms no SVM e 10,23 ms no RF, contra 16,80 ms da CNN em CPU); com GPU, CNN e SVM tiveram tempos da mesma ordem de grandeza, com diferença dentro da variação medida entre execuções, sem ordenação estável nem equivalência formal.

No eixo do desempenho preditivo, a resposta segue a definição operacional da Seção 3.8. No teste, a diferença entre a CNN e o SVM foi de +0,0902 no F1-macro (IC95 [0,0801; 0,1005]) e de −0,0674 no EER (IC95 [−0,0763; −0,0582]); entre a CNN e o RF, foi de +0,1673 no F1-macro (IC95 [0,1568; 0,1775]) e de −0,1209 no EER (IC95 [−0,1304; −0,1118]) (Tabela 12). Nenhum dos quatro intervalos contém zero, e o mesmo ocorreu na validação. Pela definição adotada, o modelo clássico só poderia ser considerado competitivo se algum desses intervalos contivesse zero, em uma das duas métricas (e, ainda assim, isso não provaria equivalência); como isso não ocorreu em nenhuma delas, a conclusão não depende da escolha entre F1-macro e EER. A distância também não é marginal: o EER da CNN ficou em cerca da metade do EER do SVM, e o *recall* de *bonafide* foi de 0,7877 na CNN, contra 0,6635 no SVM e 0,5326 no RF (Seção 5.2). Entre os clássicos, o SVM superou o RF, também com IC95 sem zero, e o RF de referência, com 3,5 vezes mais dados de treino, continuou abaixo do SVM e da CNN (Seção 5.5).

No eixo do custo computacional, entendido como tempo de inferência (Seção 5.7), a resposta depende do regime de uso (Tabela 17 e Tabela 18). Sem GPU, os dois clássicos tiveram menor latência ponta a ponta por áudio isolado que a CNN; com GPU, a diferença entre a CNN e o SVM ficou dentro da variação medida entre execuções, e o RF levou cerca do dobro do tempo de cada um deles. A vantagem dos modelos clássicos em latência de inferência está, portanto, em não exigirem GPU. Em termos práticos, com lote de um áudio, os quatro arranjos ficaram abaixo de 17 ms para classificar um trecho de 4,0 s, tempo muito menor que a duração do próprio trecho, o que indica capacidade de processamento mais rápida que o tempo real depois que o segmento de 4,0 s está disponível. A latência de aquisição do áudio e a inferência contínua em fluxo (*streaming*) não foram avaliadas, de modo que esse resultado, por si só, não demonstra a adequação a um sistema em tempo real. A diferença de tempo de inferência pesa em dois cenários: no processamento em massa, em que, na predição em lote, o RF processou cerca de 49 mil áudios por segundo, contra cerca de 65 da CNN em CPU; e nos ambientes sem GPU, em que a CNN foi o arranjo de maior latência.

Os EER deste trabalho não se comparam ao de 1,32% obtido pelos melhores sistemas na tarefa LA do ASVspoof 2021 (Yamagishi *et al.*, 2021), nem às acurácias de 90% a 96% relatadas por Bhukya, Raj e Kumar (2025) para classificadores clássicos (Seção 2.5). O protocolo é outro: no desafio, os sistemas são treinados no ASVspoof 2019 LA e avaliados em ataques e *codecs* em maioria ausentes do treino, enquanto neste trabalho a partição foi aleatória por áudio, com os mesmos ataques e *codecs* no treino e na avaliação (Seção 3.2). Essa diferença tende a favorecer os números deste trabalho, e ainda assim o EER da CNN no teste, de 0,0737, ficou acima daquele valor. O número absoluto mais alto tem motivos no próprio desenho: a CNN é pequena, com 240.866 parâmetros; é um modelo único, sem combinação de classificadores; foi treinada sem aumento de dados; usou o Log-Mel, que Müller *et al.* (2022) consideraram inferior à CQT (Seção 3.5); e recebeu o áudio com o silêncio removido pelo VAD, o que mitiga, por desenho, o atalho discutido a seguir. No caso das acurácias, há ainda a questão da métrica: com nove áudios *spoof* para cada *bonafide*, um classificador que responde sempre *spoof* já acerta 90,00% do teste deste trabalho (Seção 5.2), razão pela qual o F1-macro foi o critério de comparação.

Esse atalho é conhecido nos dados do ASVspoof. Müller *et al.* (2021) mostraram, no ASVspoof 2019 LA, que a duração do silêncio basta para separar boa parte dos áudios *bonafide* e *spoof* (Seção 3.3), e este trabalho encontrou o mesmo sinal: a proporção de fala retida pelo VAD difere entre as classes, e uma árvore de decisão treinada só com ela chegou a EER de 0,1933 numa partição anterior à delimitação do universo. Esse valor é da mesma ordem de grandeza do EER de 19,93% que o modelo baseado na duração do silêncio inicial obteve no ASVspoof 2021 LA (Tabela 1 de Müller *et al.*, 2021). A comparação é só de ordem de grandeza, porque a medida do silêncio (lá, a duração do silêncio inicial; aqui, a proporção de fala retida pelo VAD), o modelo e a partição são diferentes, e o número do artigo é um resultado preliminar, da fase *progress* do desafio. Diante desse sinal, o atalho foi tratado por desenho: o silêncio foi removido no pré-processamento e a proporção de fala ficou fora das entradas dos modelos.

Uma comparação que a leitura dos números sugere é entre essa árvore e o RF principal, cujo EER no teste foi de 0,1946, próximo do dela. Os dois números medem coisas diferentes. A árvore mede um artefato da construção do conjunto de dados, a diferença de silêncio entre as classes; o RF mede o que sobra depois de o atalho ser mitigado por desenho, com o silêncio removido pelo VAD e a proporção de fala fora das entradas (e com correlação residual em `mfcc1_std`, Seção 5.8), e teve F1-macro de 0,7210 no teste, contra 0,6503 da árvore. Os valores vêm de partições diferentes e não se comparam diretamente. Dois resultados apoiam

essa leitura, cada um com a sua ressalva. Na etapa de diagnóstico, com as características anteriores ao mascaramento, um RF treinado e avaliado com as classes pareadas em faixas da proporção de fala, arranjo em que essa variável não separa as classes, ainda obteve EER de 0,1631 (Seção 4.2). Com as características definitivas, a característica mais correlacionada com a proporção de fala foi retirada numa ablação, e o RF perdeu pouco sem ela (Seção 5.8).

A evidência deste trabalho não alcança quatro situações. Ela não diz como os modelos se comportam diante de ataques ausentes do treino, já que os treze ataques estavam presentes nos três conjuntos; não diz se a ordem entre os modelos se mantém em outros conjuntos de dados; não mede a variação que outro treino da CNN produziria, porque a rede foi treinada com uma única semente; e não diz como os três modelos se ordenariam com mais de 30.000 exemplos de treino, a não ser para o RF, pelo braço de referência. Essas lacunas são discutidas como limitações na Seção 5.10, e as que podem ser atacadas diretamente aparecem como trabalhos futuros na Seção 6.2.

5.10 Limitações

Esta seção reúne as limitações do trabalho, agrupadas em quatro frentes: o desenho do protocolo e os dados, as representações, o treinamento dos modelos e, por fim, as análises complementares e a medição do custo. Para cada limitação, indica-se o que ela é, por que aconteceu e que efeito pode ter sobre os resultados. Elas delimitam o alcance das conclusões da Seção 5.9, que valem para o protocolo aqui descrito.

A primeira frente reúne o desenho do protocolo e os dados, e nela está a principal limitação do trabalho: a partição aleatória por áudio (Seção 3.6). Cada um dos treze ataques, cada *codec* e cada locutor aparecem no treino, na validação e no teste, de modo que um modelo pode aprender a assinatura de um sistema de síntese ou de um locutor específico e reconhecê-la depois. As métricas deste trabalho são, por isso, potencialmente otimistas em relação a um cenário com ataques novos. Também não se comparam ao EER de 1,32% de Yamagishi *et al.* (2021), obtido no protocolo oficial do desafio, que treina no ASVspoof 2019 LA e avalia no 2021 LA, com ataques em maioria ausentes do treino. A escolha do ASVspoof 2021 LA com partição interna vem do planejamento inicial do trabalho (Seção 1.3) e atende à pergunta de pesquisa, que compara famílias de modelos no mesmo ambiente. O preço é que nada se afirma sobre ataques não vistos, nem sobre se a ordem entre CNN, SVM e RF se manteria nesse cenário. A análise por ataque e por *codec* (Seção 5.6) atenua a limitação, ao mostrar como o desempenho varia entre os ataques presentes, mas não a substitui.

A avaliação com um ataque deixado de fora do treino por vez (*leave-one-attack-out*), prevista como análise complementar, não foi executada, porque a fase experimental foi encerrada ao fim da comparação principal. É o experimento que mediria, dentro do próprio universo, a generalização a ataques não vistos; sem ele, a limitação anterior fica sem medida própria.

O trabalho usou um único conjunto de dados. O ASVspoof 2021 LA não disponibiliza partições próprias de treino e de desenvolvimento (Liu *et al.*, 2023), e treino, validação e teste saíram todos da fase *eval* (Seção 3.2). Não houve avaliação em outro conjunto, como o conjunto *in-the-wild* de Müller *et al.* (2022), e os resultados não dizem como os modelos se comportariam com gravações de outra origem, com outros sistemas de síntese ou em outras condições de canal.

O universo foi delimitado à fase *eval*, com a exclusão das fases *progress*, de 16.464 áudios, e *hidden*, de 16.926 (Seção 3.2). A fase *hidden* tem o silêncio cortado na origem, e misturá-la ao universo contaminaria qualquer análise baseada na proporção de fala; a *progress* ficou fora para que o universo fosse o conjunto oficialmente pontuado do desafio. Os resultados valem, portanto, apenas para os 148.176 áudios da fase *eval*, e nada se mede sobre áudios que já chegam sem o silêncio.

A segunda frente trata das representações. A duração de 4,0 s foi fixada por decisão de protocolo e não foi tratada como eixo experimental (Seção 3.3). Depois do VAD, os áudios com mais fala que isso foram cortados, e os mais curtos, completados com zeros: no universo, a mediana foi de 120 *frames* válidos em 251, e 53,07% dos áudios tinham mais da metade do vetor em *padding* (Seção 3.4). Outra duração mudaria a quantidade de sinal real vista pelos modelos e a fração de *padding*, e o efeito dessa escolha sobre a comparação não foi medido.

O piso de 80 dB da conversão do espectrograma para decibéis é relativo ao máximo de cada exemplo (Seção 3.5). O *padding* torna-se, assim, um platô constante dentro de cada espectrograma, com valor que varia de um exemplo para outro, conforme o máximo daquele exemplo. A agregação mascarada impede que esse platô entre diretamente na saída da CNN, mas a convolução leva o conteúdo do *padding* às posições válidas de fronteira (Seção 4.4), e a normalização em lote o inclui nas suas estatísticas, como se discute adiante. Se o máximo dos espectrogramas diferir sistematicamente entre as classes, o valor do platô também diferirá; esse efeito não foi medido.

No ramo clássico, o MFCC usou o banco de 128 filtros Mel padrão da biblioteca sobre a FFT de 512 pontos (Seção 3.4). Com 257 *bins* de frequência, 61 dos 128 filtros leem no máximo dois *bins* e 12 pares de filtros compartilham o mesmo *bin* de pico, o regime degenerado

que levou a CNN a usar a FFT de 1.024 pontos (Seção 3.5). O problema não foi corrigido no ramo clássico porque as características já estavam congeladas quando ele foi identificado, e uma nova extração exigiria refazer o ajuste e a avaliação de RF e SVM. O efeito é atenuado pela transformada discreta do cosseno (DCT) do MFCC, que resume os 128 filtros em 20 coeficientes, mas pode ter reduzido a qualidade das características do ramo clássico em relação à representação da CNN, cuja FFT de 1.024 pontos evita esse regime.

A terceira frente trata do treinamento dos modelos. Os três modelos do braço principal foram treinados em 30.000 dos 103.723 exemplos do treino, e a subamostragem custou desempenho real ao RF (Seção 5.5): a sua curva de aprendizado não saturou (Figura 4). Um ajuste log-linear dessa curva, feito sobre a faixa medida (de 5.000 a 103.723 exemplos), estima que o RF precisaria de cerca de 276 mil exemplos para alcançar o F1-macro do SVM na validação, 1,86 vezes o universo inteiro da fase *eval*. Por ser uma extrapolação fora da faixa medida, o número deve ser lido como ordem de grandeza e como limite otimista: curvas de aprendizado tendem a achatar à medida que os dados crescem, e o número real seria provavelmente maior. O braço de referência, além disso, só mede o custo para o RF; quanto o SVM e a CNN ganhariam com o treino completo não foi medido. A subamostra foi fixada *a priori* como orçamento experimental comum, tendo em vista a escalabilidade do SVM com *kernel* RBF, cujo custo de treinamento cresce entre $O(n^2)$ e $O(n^3)$ com o número de exemplos (Seção 3.6); o SVM não foi treinado nos 103.723 exemplos, de modo que a viabilidade desse treino não foi testada.

Na busca do RF, dois hiperparâmetros da configuração vencedora ficaram na borda do espaço explorado (Seção 4.3): o mínimo de amostras por folha no limite inferior da faixa, 5 (faixa de 5 a 20), e o mínimo de amostras para dividir um nó no maior valor da lista, 10. Nessas duas dimensões, a busca entregou o melhor ponto dentro da faixa, que não é necessariamente o ótimo do hiperparâmetro. A faixa do mínimo por folha foi uma decisão de protocolo, tomada para impedir folhas puras (Seção 3.7), e foi mantida. O efeito possível é um RF abaixo do que ele alcançaria num espaço de busca maior, em magnitude não medida.

O *kernel* do SVM foi fixado em RBF, sem busca do tipo de *kernel*, embora o planejamento inicial previsse incluí-lo no espaço de busca (Seção 3.7), e os *kernels* polinomial e sigmoide não foram avaliados. O indício disponível vem da própria busca: os sete candidatos no regime quase linear ($\gamma \leq 0{,}001$) tiveram EER médio de validação cruzada entre 0,187 e 0,290, contra 0,153 do melhor candidato, indício que se limita ao regime próximo do *kernel* linear e não substitui uma comparação de *kernels*. A omissão não se explica por custo, já que a

busca inteira do SVM levou 392 s (Seção 4.3), e um SVM com outro *kernel* poderia ter desempenho diferente do relatado.

A CNN final foi treinada com uma única semente (42), sem análise de variância entre sementes. Essa análise estava prevista como complementar, fora do caminho crítico do trabalho, mas não foi executada, porque a fase experimental foi encerrada ao fim da comparação principal. O *bootstrap* pareado reamostra apenas a avaliação, de modo que os IC95 da Seção 5.4 não incluem essa fonte de variação. Para contexto, o F1-macro do RF variou com desvio-padrão de 0,0004 entre cinco sementes, e o SVM tem treino determinístico; no braço principal, a fonte de variação que domina é a subamostra de 30.000 sorteada, com desvio-padrão de 0,0023 no RF e de 0,0017 no SVM (Tabela 6). A variância de inicialização da CNN é uma fonte não quantificada neste trabalho. A diferença de 0,0902 no F1-macro entre a CNN e o SVM no teste é muito maior que essas dispersões, o que torna improvável que outra semente invertesse a ordem; essa leitura, porém, é uma inferência feita a partir dos modelos clássicos, sem medida na própria CNN.

O retreinamento final da CNN nos 30.000 exemplos fixou o número de épocas na melhor época da seleção (37), sem igualar o número de atualizações de gradiente (Seção 3.7). Como o conjunto do retreinamento é 11,1% maior que o de 27.000 exemplos da seleção, com o mesmo tamanho de lote, a CNN final recebeu cerca de 11,1% mais atualizações que a configuração selecionada. A alternativa de fixar o número de atualizações foi considerada e descartada, porque o protocolo definido antes do treino estabelecia um número fixo de épocas e a alternativa resultaria num número fracionário de épocas. Além disso, a configuração vencedora não usa *dropout* nem *weight decay* (Seções 3.7 e 4.4), de modo que, na fase de seleção, a parada antecipada era o único mecanismo de regularização em operação. O retreinamento retira esse mecanismo, já que treina por número fixo de épocas e sem conjunto de acompanhamento, e ao mesmo tempo dá à rede as atualizações a mais; os dois efeitos tendem na mesma direção, a de um modelo final um pouco mais ajustado aos dados de treino que o modelo cuja época foi selecionada.

A perda de treino por época oscilou bastante nos dois treinos. Nas épocas de 28 a 40 da configuração vencedora da seleção, ela ficou entre 0,0049 e 0,0973; nas épocas de 28 a 37 do retreinamento, entre 0,0136 e 0,0526. A diferença entre as perdas da época 37 nos dois treinos (0,0136 no retreinamento contra 0,0049 na seleção, Seção 4.4) fica dentro dessa oscilação, e a comparação serviu apenas para confirmar que o retreinamento terminou com perda da mesma ordem de grandeza da seleção. A perda de treino não mede sobreajuste, o que exigiria um conjunto de acompanhamento que o retreinamento, por desenho, não teve. O efeito sobre o

desempenho também ficou sem medida, porque avaliá-lo exigiria usar a validação externa para escolher o ponto de parada, privilégio que os modelos clássicos não tiveram (Seção 3.6).

Na CNN, a máscara temporal protege a agregação, mas não a normalização em lote (Seção 3.5). Cada camada de normalização calcula a média e a variância por canal sobre o lote inteiro, com o *padding* incluído, e essas estatísticas normalizam também as posições válidas. Como a estatística é calculada sobre o lote inteiro, ela desloca e escala da mesma forma todos os exemplos do lote e não cria, exemplo a exemplo, um sinal de onde o áudio termina; ainda assim, a quantidade de *padding* do lote influencia a normalização, e o efeito não foi medido. Corrigi-lo exigiria uma normalização que respeitasse a máscara e um novo treino da CNN.

A redução da máscara segue uma semântica de teto (Seção 3.5): uma posição reduzida conta como válida se qualquer dos *frames* originais que ela agrega era válido. Na arquitetura adotada, cada uma das 15 posições finais agrega 16 *frames*, de modo que a posição de fronteira pode ser formada em sua maior parte por *padding* e ainda entrar na agregação com o mesmo peso das demais. O efeito fica limitado a uma posição por exemplo, com peso relativo maior nos áudios curtos, que têm menos posições válidas. A alternativa, a semântica de piso, descartaria a fronteira do áudio, que é informação real, e encurtaria ainda mais o sinal justamente nos áudios curtos; o teto é também coerente com o arredondamento para cima na contagem de *frames* válidos (Seção 3.4).

A quarta frente trata das análises complementares e da medição do custo. Na análise por *codec* (Seção 5.6), a hipótese de a CNN ganhar mais que os modelos clássicos com a banda larga ficou não decidida: o ganho da CNN foi o maior em termos relativos e o menor em termos absolutos, e falta um intervalo de confiança para a diferença entre os ganhos dos modelos. Decidir exigiria um *bootstrap* pareado dessa diferença, que não foi feito; como a análise foi feita na validação, o teste não foi usado para isso.

Na análise por ataque (Seção 5.6), a comparação entre a amplitude do EER entre ataques e a distância agregada entre os modelos usou um piso de ruído heurístico (1,96 vezes o desvio-padrão do ΔEER pareado), que não é um intervalo de confiança da amplitude. A amplitude vem de subconjuntos menores, um por ataque, e tem incerteza maior que a da distância agregada, de modo que as leituras de que ela supera a distância, ou fica da mesma ordem dela, são indicativas.

Todas as medições de tempo foram feitas num único equipamento, com a GPU NVIDIA GeForce RTX 5060 Ti (Seção 3.10). Os tempos indicam ordem de grandeza e não constituem um *benchmark*: entre duas execuções na mesma máquina, os tempos de predição da CNN variaram de 4,1% a 13,7% (Seção 3.9), razão pela qual não se atribuiu ordenação estável entre a CNN em GPU e o SVM, que apresentaram tempos da mesma ordem de grandeza. Em outro

equipamento, com outra GPU, outro processador ou mais *threads*, as razões entre os arranjos podem mudar, e a ordem entre arranjos de tempos próximos pode se inverter.

Um ponto que costuma figurar entre as limitações de trabalhos com redes neurais não se aplica aqui: a reprodutibilidade numérica do treino da CNN. O modo determinístico estrito do PyTorch, que interrompe a execução diante de uma operação sem versão determinística, esteve ativo no retreinamento e na inferência da CNN (Seção 3.10), e a reprodução dos artefatos de inferência foi conferida campo a campo (Seção 4.5). Trata-se de um controle do trabalho; o que permanece sem medida é a variação entre sementes, discutida acima.

6 CONCLUSÃO

Este capítulo retoma os objetivos do trabalho e a resposta à pergunta de pesquisa (Seção 6.1) e indica os desdobramentos sugeridos pelos resultados e pelas limitações discutidas no Capítulo 5 (Seção 6.2).

6.1 Considerações Finais

Este trabalho teve como objetivo geral desenvolver e avaliar modelos de *machine learning* capazes de classificar áudios como voz humana ou voz sintética, a partir de características acústicas extraídas dos sinais. O objetivo foi cumprido com três classificadores treinados e avaliados sob um mesmo protocolo: RF e SVM, alimentados por 44 características acústicas manuais, e uma CNN, alimentada por espectrogramas Log-Mel. Os três tiveram a mesma amostra final de treinamento no braço principal (30.000 áudios tirados da fase *eval* do ASVspoof 2021 LA) e foram avaliados, com os limiares escolhidos na validação, num conjunto de teste de 22.227 áudios, usado uma única vez.

Os objetivos específicos (Seção 1.1.2) foram atendidos ao longo do texto. A investigação das técnicas de processamento de áudio e de representação de sinais, iniciada na fundamentação (Capítulo 2), resultou na cadeia comum de pré-processamento (Seção 3.3) e nos dois ramos de representação (Seções 3.4 e 3.5). O estudo das características acústicas relevantes combinou a fundamentação das Seções 2.3.1 e 2.3.2 com a medida de importância da Seção 5.8, que mostrou a informação útil ao RF concentrada nos coeficientes MFCC, com a taxa de cruzamento por zero e o centroide espectral longe do topo. A seleção e a preparação do conjunto de dados estão nas Seções 3.2 e 3.6, com o universo delimitado a 148.176 áudios e uma partição única, versionada e compartilhada pelos três modelos. A extração das características e a geração dos espectrogramas foram implementadas sobre a mesma função de pré-processamento e executadas uma única vez cada, com os resultados congelados e identificados por *hash* (Seções 4.2 e 4.4).

O treinamento e a avaliação de RF, SVM e CNN estão descritos nas Seções 3.7, 4.3 e 4.4. A comparação do desempenho foi feita nas Seções 5.1 a 5.4, com acurácia, precisão, *recall*, F1-macro, EER e ROC-AUC, e na Seção 5.7, com o tempo de inferência. A análise crítica dos resultados, com as limitações e as melhorias possíveis, está nas Seções 5.9 e 5.10 e nos trabalhos futuros da Seção 6.2.

A pergunta de pesquisa (Seção 1.2) foi respondida nos dois eixos, com a resposta condicionada ao protocolo. Em desempenho preditivo, com 30.000 exemplos de treino e partição aleatória por áudio, os modelos clássicos não mantiveram desempenho competitivo:

no teste, a CNN teve F1-macro de 0,8883 e EER de 0,0737, contra 0,7981 e 0,1411 do SVM e 0,7210 e 0,1946 do RF, com diferenças distinguíveis nas duas métricas pelo *bootstrap* pareado (Seção 5.9). Em custo computacional, medido como tempo de inferência, a resposta depende do *hardware*: sem GPU, os modelos clássicos apresentaram menor latência de inferência ponta a ponta por áudio isolado (4,92 ms no SVM e 10,23 ms no RF, contra 16,80 ms da CNN em CPU), e, com GPU, a CNN e o SVM tiveram tempos da mesma ordem de grandeza, sem ordenação estável nem equivalência formal. A vantagem dos clássicos em latência de inferência está, portanto, em não exigirem GPU, e pesa no processamento em massa e nos ambientes sem esse recurso; para classificar um áudio por vez, os quatro arranjos ficaram abaixo de 17 ms, o que indica processamento mais rápido que o tempo real depois que o segmento de 4,0 s está disponível; a latência de aquisição e o *streaming* contínuo não foram avaliados.

A contribuição do trabalho está mais na forma da comparação do que nos valores absolutos, que não se comparam aos da literatura por diferença de protocolo (Seção 5.9). As duas famílias de modelos foram medidas sob um protocolo único e controlado, com pré-processamento comum, a mesma partição, a mesma amostra final de treinamento no braço principal, a mesma regra de decisão e o mesmo critério de escolha do limiar, o teste usado uma única vez e as diferenças avaliadas por *bootstrap* pareado. O tempo de inferência foi medido ponta a ponta e por regime de *hardware*, o que mostrou que a pergunta sobre qual modelo é mais rápido na inferência só tem resposta quando o regime de uso é especificado. Além disso, o atalho da duração do silêncio, já apontado na literatura para esses dados, foi identificado e mitigado por desenho, sem que se afirme sua eliminação: restaram uma correlação residual em `mfcc1_std` e possíveis efeitos de *padding* na CNN (Seção 5.10), e as verificações das Seções 4.2 e 5.8 indicam que o RF, o modelo em que isso foi medido, perde pouco desempenho sem a característica mais correlacionada com a proporção de fala.

6.2 Trabalhos Futuros

Os desdobramentos a seguir decorrem das limitações discutidas na Seção 5.10. O primeiro é repetir a comparação no protocolo oficial do desafio, com treino no ASVspoof 2019 LA (Wang, X. *et al.*, 2020) e teste no ASVspoof 2021 LA (Liu *et al.*, 2023). Esse protocolo mediria a generalização a ataques e *codecs* ausentes do treino e permitiria comparar os EER com os da literatura. Dentro do próprio universo deste trabalho, uma avaliação *cross-attack*, em que cada ataque é deixado de fora do treino por vez (*leave-one-attack-out*), mediria a mesma generalização sem trocar de conjunto de dados e diria se a ordem entre CNN, SVM e RF se

mantém diante de ataques não vistos. Um teste *cross-dataset*, por fim, avaliaria os modelos em gravações de outra origem, como as do conjunto *in-the-wild* de Müller *et al.* (2022).

No ramo profundo, outras representações poderiam ser comparadas ao Log-Mel, como a CQT e o espectrograma log-linear, que Müller *et al.* (2022) consideraram superiores à escala Mel, com o cuidado de preservar a paridade entre os ramos discutida na Seção 3.5. No ramo clássico, o SVM poderia ser treinado no treino completo de 103.723 exemplos, com aproximações de *kernel* que reduzem o custo de treinamento, o que diria se ele ganha com mais dados, como o RF ganhou (Seção 5.5). Na CNN, o treino com várias sementes mediria a variância de inicialização que ficou sem medida neste trabalho.

Dois parâmetros fixados no desenho também merecem ser revistos. O MFCC do ramo clássico poderia ser reextraído com FFT de 1.024 pontos, a mesma da CNN, para eliminar o regime degenerado do banco de filtros descrito na Seção 5.10. Outras durações do trecho analisado, hoje fixado em 4,0 s, poderiam ser tratadas como eixo experimental, para medir quanto do resultado depende da quantidade de fala que cada modelo recebe.

REFERÊNCIAS

BATEMAN, J. **Deepfakes and Synthetic Media in the Financial System: Assessing Threat Scenarios**. Washington, DC: Carnegie Endowment for International Peace, jul. 2020. (Cyber Policy Initiative Working Paper Series, Cybersecurity and the Financial System, n. 7). Disponível em: https://carnegieendowment.org/research/2020/07/deepfakes-and-synthetic-media-in-the-financial-system-assessing-threat-scenarios. Acesso em: 7 out. 2026.

BERGSTRA, J.; BENGIO, Y. Random search for hyper-parameter optimization. **Journal of Machine Learning Research**, v. 13, p. 281-305, 2012.

BHUKYA, R. K.; RAJ, A.; KUMAR, A. ASVSpoof 2021: Detecting Spoofed Utterances Through Hybrid Features. **APSIPA Transactions on Signal and Information Processing**, v. 14, n. 1, e19, 2025. DOI: 10.1561/116.20250026.

BREIMAN, L. Random forests. **Machine Learning**, v. 45, n. 1, p. 5-32, 2001.

CHESNEY, R.; CITRON, D. K. Deep fakes: a looming challenge for privacy, democracy, and national security. **California Law Review**, v. 107, p. 1753-1820, 2019. DOI: 10.15779/Z38RV0D15J.

CORTES, C.; VAPNIK, V. Support-vector networks. **Machine Learning**, v. 20, n. 3, p. 273-297, 1995.

DAVIS, S.; MERMELSTEIN, P. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. **IEEE Transactions on Acoustics, Speech, and Signal Processing**, v. 28, n. 4, p. 357-366, 1980.

EFRON, B.; TIBSHIRANI, R. J. **An introduction to the bootstrap**. New York: Chapman & Hall, 1993.

FINCEN. **FinCEN Alert on Fraud Schemes Involving Deepfake Media Targeting Financial Institutions**. FIN-2024-Alert004. [*S. l.*]: Financial Crimes Enforcement Network, 13 nov. 2024. Disponível em: https://www.fincen.gov/sites/default/files/shared/FinCEN-Alert-DeepFakes-Alert508FINAL.pdf. Acesso em: 7 out. 2026.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. Cambridge: MIT Press, 2016.

HSU, C.-W.; CHANG, C.-C.; LIN, C.-J. **A practical guide to support vector classification**. Taipei: National Taiwan University, 2003. Relatório técnico.

IOFFE, S.; SZEGEDY, C. Batch normalization: accelerating deep network training by reducing internal covariate shift. *In:* INTERNATIONAL CONFERENCE ON MACHINE LEARNING, 32., 2015, Lille. **Proceedings** [...]. [*S. l.*]: PMLR, 2015. p. 448-456.

KEERTHI, S. S.; LIN, C.-J. Asymptotic behaviors of support vector machines with Gaussian kernel. **Neural Computation**, v. 15, n. 7, p. 1667-1689, 2003.

KINGMA, D. P.; BA, J. Adam: a method for stochastic optimization. *In:* INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS, 3., 2015, San Diego. **Proceedings**

[...]. [*S. l.*: *s. n.*], 2015. Disponível em: https://arxiv.org/abs/1412.6980. Acesso em: 10 out. 2026.

LIU, X. *et al.* ASVspoof 2021: towards spoofed and deepfake speech detection in the wild. **IEEE/ACM Transactions on Audio, Speech, and Language Processing**, v. 31, p. 2507-2522, 2023. DOI: 10.1109/TASLP.2023.3285283.

MAJID, T. **Defending Against Audio Deepfakes: Robust Detection in the Synthetic Speech Era**. 2026. Tese (Doutorado em Engenharia Informática) – Università degli Studi di Roma "La Sapienza", Roma, 2026. Disponível em: https://hdl.handle.net/20.500.14242/359101. Acesso em: 9 maio 2026.

McFEE, B. *et al.* librosa: audio and music signal analysis in Python. *In:* PYTHON IN SCIENCE CONFERENCE, 14., 2015, Austin. **Proceedings** [...]. [*S. l.*: *s. n.*], 2015. p. 18-25.

MÜLLER, N. M. *et al.* Speech is Silver, Silence is Golden: What do ASVspoof-trained Models Really Learn? *In:* ASVSPOOF 2021 WORKSHOP, 2021. **Proceedings** [...]. [*S. l.*]: ISCA, 2021. p. 55-60. DOI: 10.21437/ASVSPOOF.2021-9.

MÜLLER, N. M. *et al.* Does Audio Deepfake Detection Generalize? *In:* INTERSPEECH, 2022, Incheon. **Proceedings** [...]. [*S. l.*]: ISCA, 2022. p. 2783-2787. DOI: 10.21437/Interspeech.2022-108.

NEGREIRO, M. **Children and deepfakes**. [*S. l.*]: European Parliamentary Research Service (EPRS), jul. 2025. (Briefing, PE 775.855). Disponível em: https://www.europarl.europa.eu/thinktank/en/document/EPRS_BRI(2025)775855. Acesso em: 8 out. 2026.

OORD, A. van den *et al.* WaveNet: A Generative Model for Raw Audio. **arXiv preprint arXiv:1609.03499**, 2016. Disponível em: https://arxiv.org/abs/1609.03499. Acesso em: 29 abr. 2026.

PASZKE, A. *et al.* PyTorch: an imperative style, high-performance deep learning library. *In:* ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 32., 2019, Vancouver. **Proceedings** [...]. [*S. l.*: *s. n.*], 2019.

PEDREGOSA, F. *et al.* Scikit-learn: machine learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825-2830, 2011.

PYTORCH TEAM. **PyTorch 2.7 release**. [*S. l.*]: PyTorch Foundation, 23 abr. 2025. Disponível em: https://pytorch.org/blog/pytorch-2-7/. Acesso em: 30 set. 2026.

SHI, H. *et al.* Benchmarking Audio Deepfake Detection Robustness in Real-world Communication Scenarios. **arXiv preprint arXiv:2504.12423**, 2025. Disponível em: https://arxiv.org/abs/2504.12423. Acesso em: 9 maio 2026.

SINGH, R. *et al.* Audio Deepfake Detection in the Age of Advanced Text-to-Speech models. **arXiv preprint arXiv:2601.20510**, 2026. Disponível em: https://arxiv.org/abs/2601.20510. Acesso em: 9 maio 2026.

TENSORFLOW. **Install TensorFlow with pip**. [*S. l.*]: Google, 12 mar. 2026. Disponível em: https://www.tensorflow.org/install/pip. Acesso em: 30 set. 2026.

WANG, C. *et al.* Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers. **arXiv preprint arXiv:2301.02111**, 2023. Disponível em: https://arxiv.org/abs/2301.02111. Acesso em: 8 out. 2026.

WANG, X. *et al.* ASVspoof 2019: a large-scale public database of synthesized, converted and replayed speech. **Computer Speech & Language**, v. 64, 101114, 2020.

WANG, Y. *et al.* Tacotron: Towards End-to-End Speech Synthesis. *In:* INTERSPEECH, 2017, Stockholm. **Proceedings** [...]. [*S. l.*]: ISCA, 2017. p. 4006-4010.

YAMAGISHI, J. *et al.* ASVspoof 2021: accelerating progress in spoofed and deepfake speech detection. *In:* ASVSPOOF 2021 WORKSHOP, 2021. **Proceedings** [...]. [*S. l.*]: ISCA, 2021. p. 47-54. DOI: 10.21437/ASVSPOOF.2021-8.

ZHANG, B. *et al.* Audio Deepfake Detection: What Has Been Achieved and What Lies Ahead. **Sensors**, v. 25, n. 7, art. 1989, 2025. DOI: 10.3390/s25071989.